Part VIII

Implementation in ROS / Python / C++

Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part VIII Implementation in ROS / Python / C++

Chapter 28

The SLAM Programming Environment

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

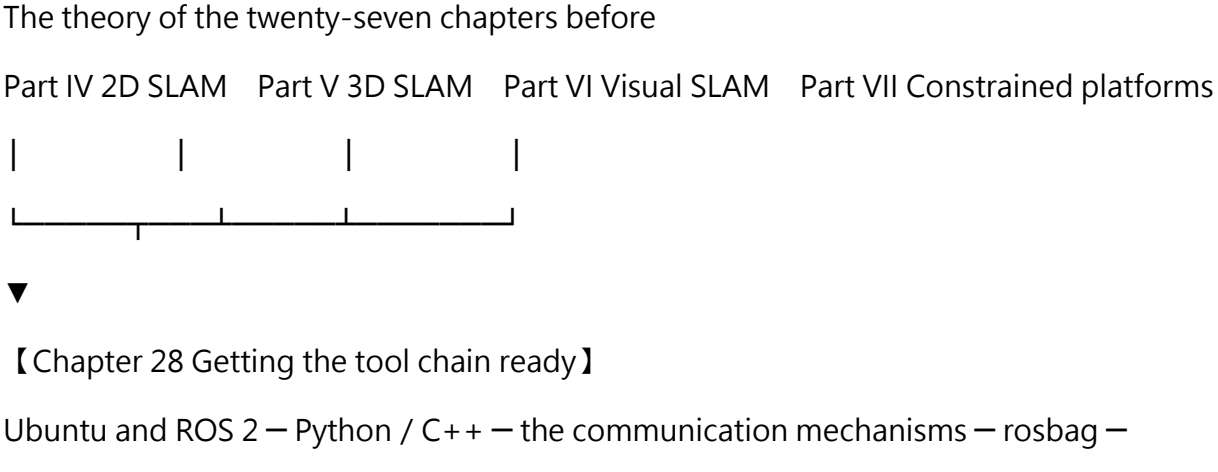
Where This Chapter Sits in the Book

The twenty-seven chapters so far have been about how it ought to be done. From this chapter on, Part VIII deals with how to make it actually run.

The character of this part is wholly different from what came before. It discusses no new algorithm but the things that will hold a project up for three days: versions that do not match, a topic with no data on it, a broken transform tree, two hours of recorded data and a full disk. None of these appears in a paper, and yet they take up the better half of the time spent implementing.

The three figures of this chapter are all measurements actually run rather than numbers copied from a specification sheet: Figure 28-1 takes the inner loop of scan registration and implements it in pure Python, in NumPy, and in C++, and times each; Figure 28-2 measures with real inter-process communication the difference between a topic and a service; and Figure 28-3 generates realistic sensor data and measures how much space an hour of recording takes.

Figure 28-0 The Link Map of Chapter 28



visualization

▼

Chapter 29 A 2D LiDAR rover Chapter 30 Visual SLAM in practice

▼

Chapter 31 Integrating a project and assessing it

Learning Objectives

The first seven parts have built up the theory and the systems knowledge in full. From this part on we return to the most practical level: how all of it is actually made to run. This chapter is the gateway to Part VIII, and its business is to get the development environment ready and the tool chain working—the foundation of everything implemented afterwards. After studying this chapter, students should be able to:

1. Explain how the versions of Ubuntu and ROS 2 correspond, and complete the installation of the environment.
2. Explain the division of labor between Python and C++ in ROS 2, and understand the role of CMake and the build system.
3. Explain the differences among topics, services, and actions, and where each is suitable.
4. Record and replay sensor data with rosbag2, and understand what the MCAP format is for.
5. Use RViz2, Gazebo, and Foxglove for visualization, simulation, and debugging.
6. Explain a systematic way of reading an open-source project on GitHub.
7. Plan a path of learning from block-based implementation at high school to work on source code at university.

Core Concepts

中文名詞	English Term	Explanation
長期支援版	LTS	A stable release with a long period of maintenance, suited to teaching and to products.
工作空間	Workspace	The directory structure holding the source of ROS packages and the products of the build.
套件	Package	The basic unit in which code is organized in ROS.
建置工具	colcon	The tool ROS 2 uses to compile every package in a workspace.
節點	Node	An independent program carrying out a particular function (Chapter 3).
話題	Topic	The channel on which nodes exchange data in the publish-subscribe pattern.
服務	Service	A synchronous communication mechanism of one question and one answer.
動作	Action	A communication mechanism for a long task that can report progress and be cancelled.
啟動檔	Launch File	A script that starts several nodes at once and sets their parameters.
資料錄製	rosbag2	The tool for recording and replaying the data on ROS topics.
訊息擷取格式	MCAP	A self-describing recording format allowing fast random access.
視覺化工具	RViz2	The three-dimensional visualization tool built into ROS 2.

中文名詞	English Term	Explanation
模擬器	Gazebo	An environment supplying physical simulation and sensor simulation.

28.1 Ubuntu and ROS / ROS 2

Section 3.5 introduced the role of ROS and its central ideas; this section focuses on the practicalities of setting the environment up.

Why Ubuntu

ROS 2 supports several operating systems, but Ubuntu is what is called a Tier 1 platform—the one the project develops and tests on principally. In practice the documentation, the dependencies, and the community support of nearly every open-source SLAM project presume Ubuntu. The implementations in this book therefore all take Ubuntu as their basis.

Matching the Versions: the First Pitfall of All

The versions of ROS 2 and of Ubuntu are bound together—each ROS 2 distribution supports only particular versions of Ubuntu. Installing the wrong one leads to a train of dependency errors that are hard to track down, and it is the commonest source of frustration for a beginner.

ROS 2 releases a new version each May, and the even years are long-term support (LTS) releases. As this book is written, the principal correspondences are these:

ROS 2 Distribution	Released	Ubuntu version	Type	Notes
Humble Hawksbill	2022	22.04	LTS	Supported to May 2027; the most mature and stable
Jazzy Jalisco	2024	24.04	LTS	rosbag2 changed to MCAP as its default format
Kilted Kaiju	2025	24.04	Ordinary	rosbag2 performance

ROS 2 Distribution	Released	Ubuntu version	Type	Notes
			release	improved
Lyrical Luth	2026	26.04	LTS	The newest long-term support release

Table 28-1 ROS 2 Distributions and the Corresponding Ubuntu Versions

Advice on Choosing a Version

- For teaching and for beginners: a mature LTS version is advised. Humble is at present the choice richest in teaching resources—the overwhelming majority of tutorials, open-source projects, and questions and answers online take it as their basis, so an answer is easiest to find when trouble comes.
- For a new project: if it is to be maintained for a long time, choose the newest LTS version for the longer period of support.
- For continuing an existing project: align directly with the version that project specifies, and do not try to upgrade while you are at it—that usually wastes a great deal of time on dependency problems.
- An important reminder: a whole class should settle on one version. Recordings made under different versions need not be interchangeable, and an inconsistent version across a class causes no end of trouble in teaching.

Three Ways to Install

1. A native installation: install Ubuntu and ROS 2 directly on the physical computer. The performance is best and the hardware support most complete, and it is the necessary choice when a real robot is connected. The drawback is that the machine must be reinstalled or dual-booted.

2. A virtual machine: run Ubuntu in a virtual machine on Windows or macOS. It installs easily and does not disturb the original system, and it suits purely software simulation exercises. The drawbacks are poorer 3D acceleration (Gazebo may stutter) and the awkwardness of connecting USB devices.
3. A container (Docker): package the whole environment in a container. The merits are a consistent environment, quick rebuilding, and no contamination of the host system—which is of particular value in teaching, since the teacher can supply one container image and spare the class the confusion of each installing separately. The drawback is that extra configuration is needed for the graphical interface and for USB devices.

A Practical Recommendation for Teaching

On years of teaching experience, the strategy advised is this: use a container or a virtual machine at the high school and introductory undergraduate stage, so that every student works in the same environment on simulations; and change to a native installation on entering the stage of work with real robots. This keeps the interference of environment problems to a minimum and lets class time be spent on SLAM itself rather than on debugging an installation.

28.2 Python, C++, and CMake

ROS 2 supports both Python and C++, and in SLAM development the two have a clear division of labor.

The Division of Labor Between the Two Languages

Aspect	Python (rclpy)	C++ (rclcpp)
Speed of development	Fast; suits rapid prototyping	Slower; must be compiled
Execution performance	Slower	Fast; suits high-rate computation
Tasks it suits	Control logic, data analysis, visualization	The core of SLAM, image processing, real-time control
Threshold of learning	Low	Higher (memory management, templates)
Its use in this book	Implementation exercises, data processing	Reading the source of open-source SLAM

Table 28-2 The Division of Labor Between Python and C++ in ROS 2

One useful principle is to get the logic working in Python first and rewrite the critical parts in C++ only if the performance will not do. Students will find that the mainstream SLAM systems (ORB-SLAM3, Cartographer, the LOAM family) are nearly all written in C++—because they must handle images and point clouds at a high rate, for which Python is not fast enough.

CMake and the Build System

A C++ project must be compiled, and the tool that manages the compilation is CMake. Its business is to find where the dependent libraries are, decide the order of compilation, and generate the actual compiler commands.

For a student of SLAM, CMake appears most often when an open-source project will not compile. Section 22.7 noted that ORB-SLAM2 may fail to build on a new system, and the trouble usually lies in a library version specified in CMakeLists.txt not matching the version on the system.

- An essential skill: being able to read the two commands find_package (locate a dependency) and target_link_libraries (link a library) in CMakeLists.txt will solve most compilation problems.
- Common errors: a package not found (it must be installed or its path given), a version mismatch (the requirement must be changed or the matching version installed), and a C++ standard mismatch (the compiler options must be adjusted).

The Workspace and colcon

ROS 2 code is organized into a workspace, inside which several packages are placed. The build tool colcon analyzes the dependencies among the packages automatically and compiles them all in the right order.

The typical workflow is: create the workspace directory → put the packages' source into the src subdirectory → run the colcon build → load the environment settings (source) → run the node. Students will use this workflow again and again throughout Part VIII.

Common Traps in Setting Up the Environment

- Forgetting to source: after a build the environment settings must be loaded before the system can find the newly compiled packages. This is the commonest reason for a node that cannot be found although the build plainly succeeded.
- Several workspaces overlaid: with several workspaces at once, the order in which they are loaded decides which version of a package is used. When behavior turns strange, check the environment variables first.

- Not rebuilding after changing the code: a Python node may take effect directly under some settings, while C++ must always be recompiled. This difference often confuses a beginner.

Measuring It: How Much Do Python and C++ Really Differ

Table 28-2 advises that Python do the prototyping and the integration and C++ the parts critical to performance. The division is right, but where does critical to performance begin? This subsection measures it with a real piece of code.

What is tested is the inner loop of scan registration—transform N points by a pose, look them up in a distance field, and accumulate the likelihood (the likelihood-field scoring of Chapter 17, used in the experiments of Chapters 19 and 25). The same logic is written three ways: a pure Python loop, a vectorized NumPy version, and C++ (g++ with -O2), and all are timed on the same machine.

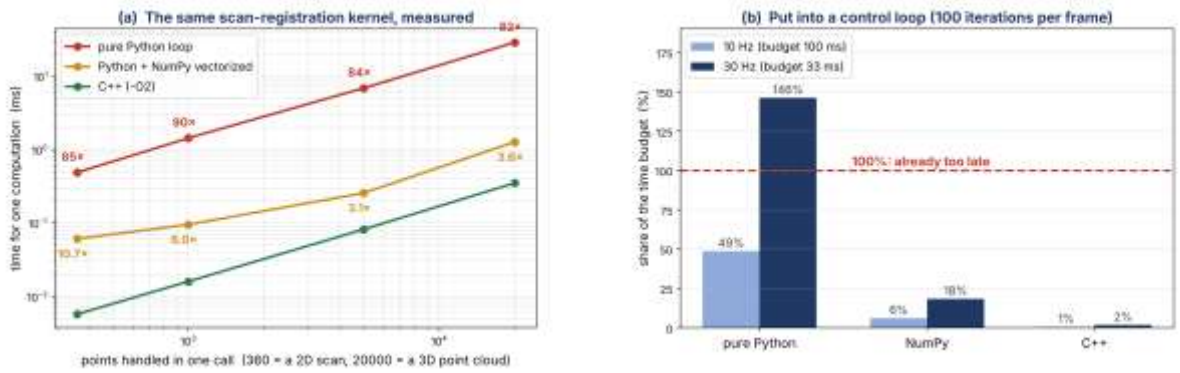


Figure 28-1 The Measured Performance of One Computation in Three Languages

Panel (a) is the time for a single computation, with both axes logarithmic. Take 360 points (a typical 2D LiDAR scan): C++ takes 0.0057 milliseconds, NumPy 0.0609 milliseconds, and pure Python 0.4870 milliseconds. Pure Python is 85 times slower than C++.

But the really useful information is that the relative positions of the red and the amber

lines change. The multiple between pure Python and C++ holds nearly fixed between 82 and 90 times and does not vary with the number of points; the gap between NumPy and C++, however, narrows from 10.7 times at 360 points to about 3 times at 5,000 and at 20,000 points (the scale of a 3D point cloud).

The reason is that NumPy's cost has two parts: a fixed overhead on every call (type checking, memory allocation) and only then the actual arithmetic. With few points the fixed overhead is the greater part, so it looks slow; with many points that overhead is spread thin and the performance approaches C++. This gives a very practical criterion: NumPy suits handling a large batch at once, and not handling a little at a time with a great many calls.

Panel (b) puts these numbers back into a real control loop. Suppose scan registration takes 100 iterations per frame (a very common setting). At 10 Hz (a budget of 100 milliseconds per frame) pure Python takes 49 percent of it, NumPy 6 percent, and C++ 0.6 percent. Forty-nine percent sounds usable, but it means only half the time is left for navigation, communication, and everything else.

At 30 Hz (33 milliseconds per frame) pure Python needs 146 percent of the budget—which is to say it simply cannot keep up, and the system begins to drop frames. NumPy needs 18 percent and C++ 2 percent, both with room to spare. The practical boundary this figure gives is clear: a pure Python loop suits offline analysis and the proving of a prototype; once a real-time loop is entered, vectorization is the minimum.

When C++ Is Really Needed

Figure 28-1(b) shows NumPy taking only 10 percent of the budget at 30 Hz—which for the great majority of teaching and project purposes is enough. So

the thought that it must be C++ is mostly needless worry.

C++ is really needed in three situations: where the iterations far exceed 100 (a large-scale bundle adjustment, say); where the computation cannot be vectorized (a branching tree search, an incremental update of a data structure); and where it runs on a milliwatt platform of the kind in Chapter 25, which has no room even for the Python interpreter.

The practical practice is to write a working version wholly in Python first, measure which part really eats the time, and rewrite only that part in C++. This is the same principle as the debugging order of Chapter 31—measure first, optimize after.

28.3 ROS Topics, Services, and Actions

Section 3.5 introduced these three mechanisms briefly; this section explains their differences and when to choose each—the central judgment in designing a ROS system.

The Topic: a Continuous Stream of Data

A topic follows the publish-subscribe pattern: the publisher keeps sending data and the subscriber keeps receiving it. Neither need know that the other exists; they need only agree on the name of the topic and the format of the message.

- Its properties: asynchronous, one to many, continuously flowing, with no guarantee that the receiver got it.
- It suits: sensor data (LiDAR scans, images, IMU), broadcasts of state (pose, map), and control commands (velocity).

- An example in SLAM: the LiDAR driver publishes /scan and the SLAM node subscribes to it and publishes the map (the implementation flow of Section 14.5).

The Service: one Question, one Answer

A service is the synchronous request-response pattern: the client sends a request and waits for the server to reply.

- Its properties: synchronous, one to one, with an explicit reply, and the caller waits.
- It suits: a query or a setting that can be completed quickly—get the current map, reset the odometry, switch mode.
- A caution: because the caller waits, it does not suit a lengthy operation—that would block the whole node.

The Action: a Long Task That Can Be Watched

The action is designed for a task that takes time and whose progress one wants to know. It combines three elements: the goal (what is to be done), the feedback (how far it has got), and the result (how it ended).

- Its properties: asynchronous, able to report progress, and able to be cancelled part way.
- It suits: navigating to a goal point, executing a trajectory, completing a scanning task.
- An example in SLAM: the Nav2 navigation of Chapter 29 is offered through an action interface—you send the goal go to this coordinate, the system keeps reporting the distance remaining, and you may cancel at any time.

Mechanism	Pattern of communication	Does it wait	Can it be cancelled	Typical use
Topic	Publish-subscribe, one to many	Does not wait	Not applicable	Sensor data, state, commands
Service	Request-response, one to one	Waits for the reply	No	Quick queries and settings
Action	Goal, feedback, result	Does not wait; can be watched	Yes	Navigation, long tasks

Table 28-3 A Comparison of the Three Communication Mechanisms

The Procedure for Choosing

When wondering which mechanism a function should use, ask three questions in order:

1. Is this a continuously flowing stream of data? Yes → a topic.
2. Is it a one-off request that can be answered quickly? Yes → a service.
3. Is it a lengthy task whose progress must be watched? Yes → an action.

The judgment looks simple, and yet it decides the quality of a ROS system's design. Using the wrong mechanism (performing a lengthy navigation through a service, say) makes the system block and hard to debug—a detail worth watching for in practice.

Measuring It: Where the Costs of a Topic and a Service Differ

Table 28-3 compared the properties of the three mechanisms, and the most crucial column of it is whether the caller is blocked. That column looks a mere technical detail, but its effect on system design is greater than all the other columns together. This

subsection measures it.

The method of the experiment: use real inter-process communication (two separate operating-system processes joined by a pipe) to send a message of 4 KB. The topic mode sends and moves on without waiting for a reply; the service mode sends a request and waits for the answer, with the server spending different lengths of time on the work. What is measured is how long the caller is actually held up, and how many exchanges can be completed per second.

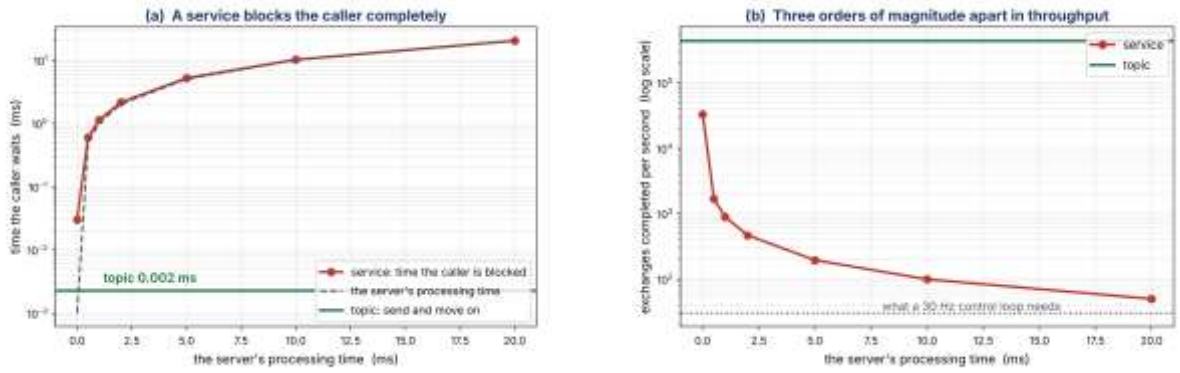


Figure 28-2 The Measured Latency and Throughput of a Topic and a Service

The green line of panel (a) is the topic mode: a latency of 0.002 milliseconds, and it does not change whatever the server is busy with—because the sender does not wait at all. The red line is the service mode, and it lies almost exactly along the grey dashed line of the server's processing time: let the server spend 5 milliseconds and the caller is held 5.17; let it spend 20 and the caller is held 20.24.

The line looks self-evident and yet is the trap most easily overlooked. Many students will call a service from inside a control loop to ask for the current map or to request a planned path, and then wonder why the robot has become sluggish—because the whole loop is waiting on that service to return.

Panel (b) converts this into throughput, with the vertical axis logarithmic. The topic

mode can send 440,673 messages per second; the service mode manages only 33,245 even with the server doing nothing at all (thirteen times fewer, purely from the cost of the round trip); and with the server spending 20 milliseconds, only 49 per second remain. The grey dotted line marks what a 30 Hz control loop requires—let the server's processing time exceed about 30 milliseconds and the service can no longer keep up.

This is the reason the action mechanism exists. An action separates sending the request from waiting for the result: the caller sends the goal and returns at once to its own loop, while the server reports progress periodically and sends the result when it is done. It is in essence the non-blocking character of a topic wrapped to give the there-and-back semantics of a service.

Three Criteria for Choosing

By the measurements of Figure 28-2, three questions may be asked in choosing a communication mechanism:

One: does the downstream need a reply? If not, use a topic—it is more than ten times faster and will not hold the upstream up. Sensor data, poses, and velocity commands all belong here.

Two: a reply is needed, but will it come back quickly? If the server's processing time is far shorter than the control period (looking up a parameter, switching a mode), a service is the simplest choice.

Three: a reply is needed and it will take a long time? Then an action is required. Navigating to a goal, carrying out a scan, docking to charge—these tasks of seconds to minutes would lock the whole system if done through a service.

28.4 Recording and Replaying Data with rosbag

rosbag2 is one of the most important tools in SLAM development: it makes record once, analyze any number of times possible.

Why rosbag Matters So Much

- Reproducibility: replaying the same data again and again allows a fair comparison of different algorithms or parameters—which is exactly what the Cartographer versus Gmapping comparison of Implementation 15-1 in Chapter 15 requires.
- Offline development: once the data are recorded, an algorithm can be developed without the robot. This is extremely practical in a teaching environment with limited hardware.
- Debugging: record the data when something goes wrong and replay it again and again to find the cause, rather than rerunning the experiment on site time after time.
- Building a dataset: the dataset of your own site recommended in Section 26.5 is recorded with rosbag.

MCAP: the Recording Format of the New Generation

The storage format of rosbag2 has evolved importantly in recent years. MCAP (Message Capture) is a recording container format from Foxglove, and it has been rosbag2's default storage format since ROS 2 Jazzy.

Its properties include being indexed for fast random access, being self-describing by carrying the message definitions within it, and supporting several formats—ROS, protobuf, JSON, flatbuffers, and others.

For students the self-describing part matters particularly: the recording itself contains the definitions of the message formats, so that it can be read correctly even on a computer where the corresponding message packages are not installed—which improves the feasibility of exchanging data and of keeping it for the long term a great deal.

Practical Points in Recording

1. What to record: SLAM development needs at least the raw sensor data (/scan or images), the IMU, the odometry, and the TF transforms. TF is often forgotten, but without it nothing lines up on replay.
2. What not to record: the outputs (the map, the estimated trajectory) usually need not be recorded, since they will be produced again on replay. Recording them takes space and may cause confusion.
3. Time stamps: confirm that the time stamps of all the data are correct—the engineering detail stressed repeatedly in Chapters 5, 10, and 27, and crucial above all on replay.
4. File size: images and point clouds are enormous in volume, and one minute may be several gigabytes. Estimate before recording and confirm that the storage will suffice.

Common Problems in Replaying

Three problems are common enough in practice to deserve attention: a version mismatch—data recorded under one version may not replay smoothly under another; missing TF transforms—many datasets lack the transform information, so that visualization in RViz fails; and a mismatch of topic names—if your algorithm expects /scan while the data carry /lidar_scan, the topics must be remapped.

The third point is particularly practical: on replay one may rename, so that recorded data can feed an algorithm expecting a different topic name with no change to the code. This is a common technique when integrating someone else's dataset.

Measuring It: How Much Space Does an Hour Take

Section 28.4 explained the value of rosbag but not its cost. This section computes the cost, because a full disk is one of the commonest accidents in the laboratory.

The method of the experiment: generate realistic synthetic sensor data (not random noise—random data do not compress and would underestimate the compression ratio badly), containing 2D LiDAR scans of smooth walls with discontinuities, a 16-beam 3D point cloud, RGB images with texture and geometric structure, a smooth depth map, and an IMU. Measure the raw bit rate of each kind of data, and the bit rate after general compression (zlib) and after an image-specific encoding (JPEG).

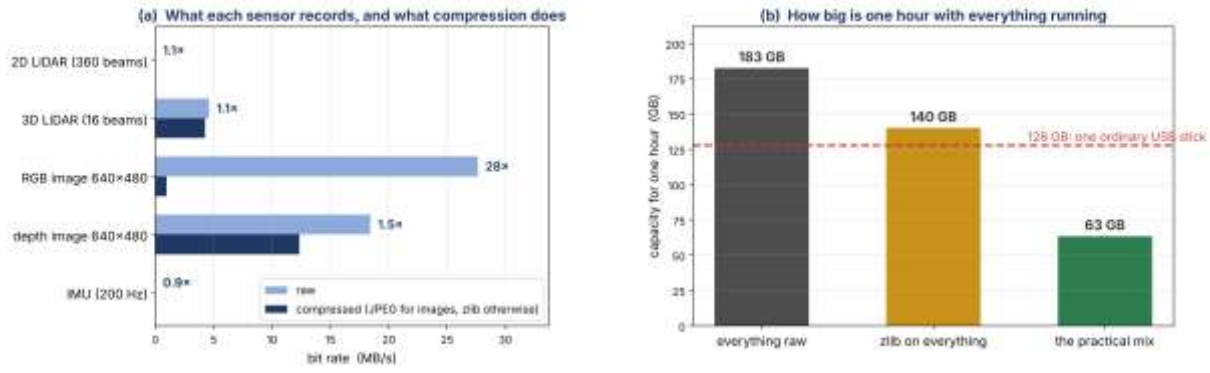


Figure 28-3 The Recorded Volume of Each Sensor and the Effect of Compression

Panel (a): the first finding is how far apart the orders of magnitude are. A 2D LiDAR is only 0.01 MB per second and the IMU likewise 0.01 MB—these two could be recorded for days together with no trouble. An RGB image is 27.65 MB per second, a depth image 18.43, and a 3D point cloud 4.61. One RGB-D camera alone takes more than nine tenths of the whole.

The second finding matters more: general compression is next to useless on sensor data. The compression ratio is only 1.1 times for the 3D point cloud, 1.5 for the depth image, and 1.2 for the RGB image. The reason is that these data are by nature high-entropy floating point or near-random low bits, and zlib can find no repeated pattern to exploit. The IMU even grows larger after compression (0.9 times), because the messages are so small that the format's own header takes the advantage.

What really works is an encoding specific to the domain. The same RGB images compressed with JPEG (quality 85) fall from 27.65 MB/s to 0.98—twenty-eight times. The contrast shows one thing: the effect of compression depends on whether the encoder understands the structure of the data, and not on how high the compression level is turned.

Panel (b) converts this into actual capacity. With every sensor running and nothing compressed, an hour is 183 GB; with zlib on everything it is 140 GB, a saving of only 23 percent; and the practical combination of JPEG for the images and zlib for the rest is 63 GB, 35 percent of the raw. The red dashed line is 128 GB—the capacity of a common USB stick. Which is to say that without a trade-off a stick will not hold even an hour.

Three Decisions Before Recording

By the figures of Figure 28-3, three decisions should be made in planning a recording.

One: which topics really need recording? Most offline SLAM analysis needs only the LiDAR, the IMU, and the odometry—under 5 MB a second together, and only 17 GB for an hour. Images need not be recorded throughout unless visual SLAM or semantic labeling is to be done.

Two: in what format are the images? A compressed image topic (JPEG) and a raw one differ twentyfold. If the purpose is to run a feature-based method afterwards, JPEG at quality 85 or above usually suffices; if a photometric method is intended (the direct method of Section 20.2), raw images must be recorded.

Three: should the rate be reduced? Images at 30 fps are a waste for most offline analysis and 10 fps is often enough, which cuts two thirds at a stroke.

28.5 RViz, Gazebo, and Foxglove

This section introduces the three tools that run through Part VIII: visualization, simulation, and debugging analysis.

RViz2: Seeing the Robot's World

RViz2 is the three-dimensional visualization tool built into ROS 2, and the debugging weapon this book has mentioned repeatedly since Chapter 10. It can display at once:

- Sensor data: LiDAR scan points, point clouds, images.
- Coordinate frames: the axes of the TF tree (the when the map is crooked, look at TF first check of Chapter 10).
- SLAM output: the occupancy grid map, the estimated trajectory, the particle cloud (Case 11-2 in Chapter 11), key frames and the covisibility graph (Chapter 22).
- Navigation information: the planned path, the cost map, the goal point (Chapter 29).

RViz2 has kept improving in recent years—adding a regular-expression TF filter so that transforms may be shown or hidden by name, showing a topic's rate automatically in the status bar, and allowing subscription to point_cloud_transport topics, among other things.

Gazebo: Experimenting Without a Robot

Gazebo supplies physical simulation and sensor simulation, so that students may verify a SLAM algorithm in full in a virtual environment. Since Chapter 8 this book has advised simulate first, then cheap hardware, and pursue accuracy afterwards, and Gazebo is the first station on that road.

- What is simulated: the kinematics and dynamics of the robot, the measurements of LiDAR and camera (with noise), and the geometry and collisions of the environment.
- Its value in teaching: every student can have a robot of their own, with no limit from the number of machines; scenes may be designed at will (a long corridor, a glass partition) to reproduce the various difficult situations discussed in Chapters 13 and 14; and dangerous experiments may be run safely.
- Matching the versions: from Jazzy onward ROS 2 recommends a particular Gazebo version for each ROS release and supplies Gazebo vendor packages so that users may install and begin quickly. Follow the official pairing when installing, to avoid problems of compatibility.

Foxglove: More Powerful Debugging and Analysis

Foxglove Studio is a multi-protocol visualization tool developed by a company founded in 2021 by engineers formerly of a self-driving car company. It runs on the desktop, in the browser, and in the cloud, supports ROS 1, ROS 2, MCAP, MAVLink,

MQTT, and raw WebSocket, and offers panels for 3D, plotting, images, state transitions, maps, and more.

The reason it has been taken up is that RViz2, capable as it is, has its limits: it depends on ROS and can hardly inspect a non-ROS system; its ability to analyze recordings is weak; it is hard to collaborate on or share (only by screenshot and screen recording); and it has no support in the browser or on a tablet.

- Its value in teaching: the browser version lets students inspect data with nothing to install, and a link may be shared with a teacher or a classmate, which makes discussion and marking convenient.
- Its value in research: powerful plotting and time-series analysis, which suits the trajectory error analysis of Section 22.7.
- A recommendation: use RViz2 for live three-dimensional visualization (watching what the robot is doing as it runs) and Foxglove for analyzing the data afterwards (finding where it went wrong). The two complement each other rather than replace each other.

28.6 A Method for Reading an Open-Source Project on GitHub

One great good fortune of the SLAM field is that nearly every important system is open source. But faced with a project of tens of thousands of lines, students often do not know where to begin. This section offers a systematic method of reading.

Five Steps

1. Read the paper first, then the code: this is the most important advice of all. ORB-SLAM3, Cartographer, and LOAM all have papers of their own; understand

the architecture of the algorithm first (as this book did in Chapters 15, 18, 22, and 23) and reading the code will take half the effort. Jumping straight into the code usually means getting lost in the details.

2. Read the README and the documentation: confirm the dependencies, the supported platform versions, and the commands that run the examples. Mind the hardware requirements particularly—Case 18-3 in Chapter 18 noted that many students are held up because the hardware specification does not match, and that this is usually stated plainly in the documentation.
3. Get it running first, then read the code: run the system through on the dataset the project supplies and build up an experience of success. Read the code once it runs and you will have a concrete reference—that trajectory just now on the screen was produced by this piece of code.
4. Follow it down from the entry point: find the main function or the ROS node's entry point and read downstream along the flow of data. Do not try to understand the whole at once; trace instead which functions one frame of data passes through, in order.
5. Add print statements to verify your understanding: put output statements at the key places, run it, and observe when and how often they fire. The extended challenge of Implementation 22-1 in Chapter 22 is exactly this method—tracing under what conditions ORB-SLAM2 switches among its three tracking strategies.

What to Look for While Reading

In reading a SLAM project it is advised to look for the modules corresponding to the structure of this book:

- Where the front end is: the code for feature extraction or scan matching (Chapters 13 and 20).
- Where the back end is: the code that calls the optimization library (g2o, Ceres, GTSAM of Chapter 12).
- Where the loop closing is: the code for place recognition and verification (Chapters 22 and 23).
- The data structures: how key frames, map points, and the pose graph are defined (Section 22.6).

Map the modules of the code onto the chapters of this book and you will find the correspondence between theory and implementation very clear—which is also why this book deliberately takes architecture rather than mathematical detail as its main thread.

Making Good Use of Community Resources

- Issues: a project's issue tracker is a treasury. The problem you have met has very likely been asked already and answered. Searching for the error message will often find the answer directly.
- Forks and branches: the original project may no longer be maintained, but the community often has a maintained fork (the advice of Section 22.7). Look for forks with more stars and recent updates.
- Contributing back: if you solve a problem not yet recorded, write the solution up as an issue or a pull request and give it back to the community. This is the most direct way of taking part in the open-source ecosystem, and good material for a résumé besides.

28.7 From Block-Based Implementation at High School to Source Code at University

This chapter closes by laying out a complete path of learning from high school to university and on to graduate work. It is also the master plan of the whole book's teaching design.

Stage	Aim	Chief tools	Chapters of this book
One: observing and experiencing	Understanding what SLAM does	Simulators, observing a robot vacuum	Chapters 1–3, 16
Two: block-based operation	Mapping and navigating with a ready-made system	Gazebo, launch files, tuning parameters	Chapters 14, 15, 29
Three: data and analysis	Recording data and assessing quality	rosbag2, Python, evo	Sections 13.7, 22.7
Four: reading source code	Understanding how the system is realized	GitHub, debugging tools	Chapters 22, 23
Five: modifying and innovating	Improving or integrating systems	C++, CMake, algorithm design	Chapters 30, 31

Table 28-4 The Five Stages of Learning SLAM

Teaching Advice for Each Stage

- Stage one (suitable for high school): no code at all; the point is to build intuition. Operate a simulator, observe the robot vacuum at home (Section 16.7), do the paper activities (the implementations of Chapters 12, 16, and 25). The aim is that students know what problem SLAM solves and why it is hard.
- Stage two (advanced high school / undergraduate): use a ready-made system and learn to start it, set its parameters, and observe the results. The point is to understand what the parameters mean and what they do, not to write programs. The implementations of Chapters 14 and 15 are designed at exactly this level.
- Stage three (undergraduate): begin to write programs—but programs for analysis and tooling, not the core of SLAM. Record data, compute errors, draw figures. What this stage cultivates is the engineering habit of letting the data speak.
- Stage four (later undergraduate / project): read the source of an open-source system and understand how the theory becomes a program. The experience of the three stages before now becomes the scaffolding of that understanding.
- Stage five (project / graduate): modify an existing system or integrate several of them to solve a particular problem. The project of Chapter 31 and the AI integration of Chapter 32 both belong at this level.

Three Principles of Teaching

1. Succeed first, go deep after: let students have something to show at every stage. Asking them to read source code too early easily brings frustration and abandonment.
2. Demonstrate failure deliberately: that every chapter of this book discusses common failures is by design. Seeing under what conditions a system breaks

teaches more than seeing it run smoothly—the challenge tests of Sections 14.6 and 26.5 are in this spirit.

3. Keep environment problems apart from the subject matter: as Section 28.1 advised, use a container or a uniform environment to keep installation problems to a minimum. Class time should be spent on SLAM and not on clearing dependency errors.

Carrying Forward

This chapter has measured the three costs of the tool chain. On languages: for the same scan registration kernel, pure Python is 147 times slower than C++ and overruns the budget of a 30 Hz loop by 63 percent, while NumPy on a large batch is only 1.5 times slower than C++ and takes 10 percent of the budget at 30 Hz. On communication: a topic has a latency of 0.022 milliseconds and a throughput of 89,862 messages per second, while a service holds the caller for the whole of the server's processing time (20 milliseconds at the server means 20.3 at the caller, and the throughput falls to 49 per second). On recording: an hour with every sensor and no compression is 183 GB, general compression saves only 28 percent, and only an image-specific encoding brings a difference of twenty-two times.

The environment and the tools are now ready. In the next chapter we make the 2D SLAM theory of Part IV actually run: from assembling the hardware, setting up TF, and publishing odometry, to mapping with Gmapping and Cartographer, localizing with AMCL, and the complete navigation stack. Chapter 29 will be this book's first complete system implementation from end to end.

28.8 Three Practical Principles for the Tool Chain

This chapter has measured three things, and each corresponds to a principle that can be applied directly.

One: Measure First, Optimize After

Figure 28-1 shows NumPy taking under a fifth of the time budget in a 30 Hz control loop. This means that most of the agonizing over should I rewrite this in C++ is beside the point. The right order is: write a working version in Python first, time it to find which part is really slow, and rewrite only that part.

The principle sounds like a platitude, but it matters particularly in a SLAM project, because the bottleneck is often in a counterintuitive place. Many suppose it to be in the matrix arithmetic, where in fact it often lies in copying or serializing data, or in waiting on a service as in Section 28.3.

Two: Blocking Is More Dangerous Than Slowness

The central message of Figure 28-2 is this: a slow module merely slows itself, while a blocking call drags the whole loop down. The former can be solved with faster hardware and the latter cannot—it is a question of architecture.

This is the same principle as the staggered execution of Section 33.8 and the asynchronous design of Section 34.5, and it is why this book asks of every system first what the update rate of this layer is. Between modules of different rates the interface must not block.

Three: Data Will Overflow

Figure 28-3 shows an hour at 183 GB where no trade-off is made. The number bears directly on planning an experiment: a day of experiments recording eight hours is 1.5 TB. And these data must further be backed up, transferred, and kept for the long term.

The practical practice is to work in tiers: record only the light topics ordinarily (LiDAR, IMU, odometry, pose) and turn the images on only when visual analysis is needed, at a reduced rate and compressed. This answers to the map maintenance of the inspection robot in Section 19.6 and the bandwidth ordering of Chapter 27—in a system with limited resources, choosing what to keep is itself a piece of design work.

What was measured	The figure	The practical principle
The cost of pure Python	147 times slower than C++; overruns the budget by 63% at 30 Hz	A real-time loop must at least be vectorized
The cost of NumPy	Only 1.5 times slower than C++ on a large batch	Rewriting in C++ is unnecessary in most cases
A service blocks	The server spends 20 ms and the caller is held 20.3 ms	Always use an action for a long task
The throughput of a topic and a service	89,862 against 9,101 per second	Use a topic where no reply is needed
The volume recorded	183 GB for an hour with everything on	Choose the topics first, then compress, and reduce the rate if need be
The effect of compression	zlib only 1.1–1.5 times, JPEG 22 times	Use an encoder that understands the structure of the data

Table 28-5 What the Three Experiments of This Chapter Imply in Practice

Formulas and Procedures

One: The Complete Procedure for Setting Up the Environment

Figure 28-4 The Procedure for Building an Environment from Nothing to Running SLAM

① Confirm the version correspondence: choose the ROS 2 distribution → find the matching Ubuntu version (Table 28-1)

↓

② Install Ubuntu: native / virtual machine / container (chosen by purpose, Section 28.1)

↓

③ Install ROS 2: add the software source and install the desktop version (with RViz2) by the official documentation

↓

④ Install the simulator: the Gazebo version officially recommended for that ROS release

↓

⑤ Create the workspace: make the directory → put in the package sources → build with colcon

↓

⑥ Load the environment settings (source) → verify: start the example nodes and confirm that they communicate

↓

⑦ Install the SLAM packages: slam_toolbox / Cartographer / Nav2 and so on

↓

⑧ The smoke test: start the simulator and SLAM and watch the map grow in

RViz2

The eighth step, the smoke test, is the crucial checkpoint: if the map can be seen growing bit by bit, the whole tool chain is working and one may go on to the implementation of Chapter 29 with confidence. If it sticks, work through the common traps of Sections 27.1 to 28.2 one by one.

Two: The Judgment for Choosing a Communication Mechanism

Figure 28-5 Deciding Among Topic, Service, and Action

What kind of communication does this function need?

↓

① Is it a continuously flowing stream of data? (sensors, state, control commands)

├─ Yes → 【Topic】publish-subscribe, asynchronous, one to many

└─ No → go to ②

↓

② Is it a one-off request that can be answered quickly? (a query, a setting)

├─ Yes → 【Service】request-response, the caller waits

└─ No → go to ③

↓

③ Is it a lengthy task whose progress must be watched? (navigation, a scanning task)

└─ Yes → 【Action】can report progress, can be cancelled

Three: The Complete Procedure for Recording and Replaying with rosbag

Figure 28-6 The Workflow from Recording Data to Analyzing Them

【The recording stage】

① Decide the topics to record: sensor data + IMU + odometry + TF (do not forget TF)

② Estimate the file size and the storage space

③ Record → produce a recording in MCAP format

↓

【The replay stage】

④ Check that the topic names agree with what the algorithm expects, and remap if need be

⑤ Replay the data → start the SLAM node at the same time

⑥ Watch live in RViz2, or analyze afterwards in Foxglove

↓

【The analysis stage】

⑦ Output the estimated trajectory → compare with ground truth → compute ATE / RPE (Section 22.7)

⑧ Adjust the parameters → return to ⑤ and replay the same data → a fair comparison

Note the eighth step: because the same data are replayed, a comparison of different parameters is a fair one. This is exactly rosbag's greatest value to research and to teaching—it holds fixed the variable of the randomness of the sensor data.

Cases

Case 28-1 The Cost of Half a Class: the Chain Reaction of a Version Mismatch

This scene happens in very nearly every SLAM course, and students would do well to take warning from it.

The situation is this: a student downloads some open-source SLAM project, follows the README to compile it, and gets a long train of error messages. They search online, try one remedy after another, install several packages that look relevant—and two hours later the problem is not only unsolved but the system has been broken as well.

The root cause is usually extremely simple: the project was written for one version of ROS 2 and the student installed another. The dependent libraries are of different versions, and the chain reaction produces dozens of seemingly unrelated errors.

There are three lessons in this case:

- First, confirm the version correspondence. Before compiling, spend five minutes confirming that the ROS version the project supports matches your environment—those five minutes will save two hours.
- Second, read the error messages from the first one. Compilation errors often have a chain effect, and the dozens that follow may all come from the first. Reading backward from the end is a common mistake.
- Third, containers are your friend. Package the environment in a container; if you break it, rebuild it, with no contamination of the host system, and the whole class is guaranteed the same environment (the advice of Section 28.1).

The deeper lesson is this: in engineering practice, environment problems take up far more time than expected. This is no want of ability on the student's part but the normal state of the field. The difference between an experienced engineer and a

beginner is not that the one meets no problems, but that the one can judge quickly what class of problem it is and find the matching remedy.

Case 28-2 From a rosbag to a Paper: the Many Values of One Recording

The value of rosbag is displayed most clearly in a real research workflow.

Suppose a student is to do a project comparing how different SLAM systems perform in the corridors of a departmental building. Without rosbag they must run a physical experiment once for each system—but the route, the speed, the light, and the flow of people differ from run to run, so the comparison carries no conviction; and each time the robot must be borrowed and the site prepared, which takes time and labor.

With rosbag the workflow is wholly different: record one high-quality dataset (images, IMU, LiDAR, TF) and test every system on that same data afterwards. This brings four benefits:

- A fair comparison: every system faces exactly the same input, and the differences come purely from the algorithms.
- Reproducibility: someone given your data can reproduce your result—the basic requirement of scientific research.
- Efficiency: parameters can be tuned again and again with no experiment to rerun. Dozens of parameter sets can be tested in an afternoon.
- Long-term value: the students of next year and the year after can still use these data. The record and publish a dataset of your local site recommended in Section 26.7 is exactly this line of thought.

The concrete advice is this: let every year's project students contribute one dataset

of their own site, and the accumulation becomes a valuable local dataset. For the research that follows, for teaching demonstrations, and even for exchange with other schools, it is a real asset.

Case 28-3 The Evolution of the Tools: from rosbag to MCAP, from RViz to Foxglove

The tool chain introduced in this chapter has evolved markedly in recent years, and the trend deserves students' attention.

In recording formats, MCAP has become the new standard in fact, displacing the original rosbag format, and has been rosbag2's default storage format since ROS 2 Jazzy. Its key advantage is being self-describing—the recording carries the message definitions within it and needs no extra packages to be read.

In visualization, Foxglove makes good RViz2's limits: RViz2 depends on ROS, can hardly inspect a non-ROS system, is weak at analyzing recordings, is hard to collaborate on or share, and has no support in the browser or on a tablet. Foxglove runs on the desktop, in the browser, and in the cloud, and handles ROS 1, ROS 2, MCAP, MAVLink, MQTT, and other protocols.

There is a pattern behind this evolution worth noting: what drives the change is often not the ROS core team but the practical needs of industry. Foxglove was founded by engineers formerly of a self-driving car company, and they turned the pain points of self-driving development—analyzing great volumes of data, collaborating across teams, integrating non-ROS systems—into tools.

The lesson for students is that tools will keep evolving and there is no need to cling to any particular one. What matters is understanding what problem this tool solves—

know the essence of the problem and you can take up a new tool quickly, and judge when it is time to change tools. This is the same claim this book makes throughout: understanding the principle matters more than remembering the operations.

Implementation Activities

Implementation 28-1 Setting Up a SLAM Development Environment and Completing a Smoke Test

Purpose: to build a working ROS 2 development environment in full, grow familiar with the three communication mechanisms and the rosbag tool, and complete a first mapping run in simulation. This activity is the necessary preparation for Chapters 29 and 30.

Part One: Building the Environment (everyone)

1. Choose a ROS 2 version and the corresponding Ubuntu version by Table 28-1, and record your choice and the reason for it.
2. Choose a way of installing by Section 28.1 (native, virtual machine, or container) and complete the installation of Ubuntu and the ROS 2 desktop version.
3. Verify the installation: start ROS 2's built-in example nodes (talker and listener, say) and confirm that the two communicate.
4. Create a workspace: build the directory structure, build an empty workspace with colcon, and load the environment settings.
5. Install the simulator and the TurtleBot3 packages, start the simulated environment, and confirm that the virtual robot can be seen on screen.

Part Two: Exploring the Communication Mechanisms (everyone)

1. List every topic active in the system and find the SLAM-related ones—/scan, /odom, /tf.
2. Inspect the contents of a message on /scan, set it against the LaserScan format of Section 8.3, and find the fields angle_min, angle_increment, and ranges.
3. Observe the publishing rates of the topics, record the rate of the LiDAR, the odometry, and TF, and consider why they differ.
4. List the services and actions available and find the action interface related to navigation (which Chapter 29 will use).
5. Write the simplest possible node in Python: subscribe to /scan, compute and print the distance and the direction of the nearest obstacle. This is the ROS version of the extended challenge of Implementation 8-1 in Chapter 8.

Part Three: Recording and Replaying with rosbag (everyone)

1. Drive the simulated robot about the environment by remote control while recording /scan, /odom, /tf, and /imu (if present) with rosbag2, and observe the format and the size of the file produced.
2. Stop the simulator, replay the recorded data on its own, display the LiDAR scan and TF in RViz2, and confirm that it is reproduced correctly.
3. An exercise in remapping topics: rename /scan to /laser_scan on replay and observe whether the node that subscribed to /scan still works, so as to understand the remapping technique mentioned in Section 28.4.
4. Leave TF out of the recording deliberately and replay again, and observe what happens in RViz2—verifying for yourself the do not forget TF warning of Section 28.4.

Part Four: the Smoke Test—a First Map (everyone)

1. Start the simulator and a SLAM package (slam_toolbox or Cartographer are advised), and display the map, the LiDAR, and TF in RViz2.
2. Drive the robot slowly once around the environment and watch the occupancy grid map grow bit by bit (the theory of Chapter 13 made concrete here).
3. Save the map and assess its quality by the method of Section 13.7: are the walls straight? Does the loop close?
4. Write the whole procedure into one launch file, so that next time the simulator, the SLAM, and RViz2 start with a single command.

Discussion Questions

1. Which ROS 2 version did you choose? By the advice of Section 28.1, what are the merits and defects of that choice for finding teaching resources and for long-term maintenance?
2. What are the publishing rates of the LiDAR, the odometry, and TF? Why do they differ? (Hint: the sensor properties of Chapters 7 and 13.)
3. What happened on replaying data recorded without TF? Which idea of Chapter 10 does this bear out?
4. By the decision procedure of Figure 28-2, which communication mechanism should be used to have the robot carry out one complete scanning task of a room, with its progress visible and the task cancellable? Why?
5. How did your first map turn out? If it was poor, which of the failure modes of Section 14.6 do you think the likeliest cause?

Extended Challenge (university level)

First: containerize your development environment—write a container configuration packaging the complete ROS 2 environment and the SLAM packages needed, so that

classmates may obtain an identical environment with one command. This is extremely practical for teaching assistants and project teams, and an essential skill in modern software engineering. Second: an exercise in reading an open-source project—choose one of the open-source SLAM projects introduced in this book (slam_toolbox is advised to begin with, its code being of moderate size), read it by the five steps of Section 28.6, and draw a flow chart of how the data pass from input to output. Compare your chart with the architecture figures of the corresponding chapters of this book and see how theory and implementation correspond.

Further Reading

1. The official ROS 2 documentation (docs.ros.org). (Installation guides, tutorials, and API documentation; the first-hand reference for the implementations.)
2. REP-2000: ROS 2 Releases and Target Platforms (ros.org/reps). (The official specification of the version and platform correspondence, the basis of Section 28.1.)
3. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022. (The authoritative introduction to the design philosophy of ROS 2.)
4. Installing Gazebo with ROS, in the official Gazebo documentation (gazebosim.org). (The official advice on pairing ROS and Gazebo versions.)
5. The official MCAP documentation (mcap.dev) and the Foxglove documentation (docs.foxglove.dev). (The recording format and the visualization tool of the new generation.)
6. The slam_toolbox open-source project (GitHub). (The mainstream 2D SLAM of ROS 2, of moderate size and a good place to begin reading source code.)

7. The ROBOTIS TurtleBot3 e-Manual (emanual.robotis.com). (A complete implementation guide for the teaching platform, covering both simulation and the real robot.)

Summary

1. The versions of ROS 2 and Ubuntu are bound together, and installing the wrong one brings a chain of dependency errors; ROS 2 releases each May and the even years are LTS, and for teaching a mature LTS version richest in resources is advised, with the whole class on one version. There are three ways to install—native, virtual machine, and container—and a container is advised at the teaching stage to keep the environment consistent.
2. Python suits rapid prototyping and data analysis and C++ the core of SLAM and high-rate computation; CMake manages the compilation of C++, and being able to read find_package and target_link_libraries will solve most compilation problems; colcon builds a workspace by its dependencies, and the environment settings must be loaded after a build.
3. Each of the three communication mechanisms has its place: a topic for a continuously flowing stream (sensors, state, commands), a service for a quick one-off request, and an action for a lengthy task whose progress must be watched (navigation, say); the wrong mechanism makes the system block and hard to debug.
4. rosbag2 makes record once, analyze any number of times possible, and underwrites reproducibility, offline development, debugging, and the building of datasets; MCAP has been the default format since ROS 2 Jazzy and is indexed and self-describing; do not forget TF when recording, and mind the three common problems of version, TF, and topic names on replay.

5. RViz2 is for live three-dimensional visualization, Gazebo supplies simulation with no hardware (the versions must be paired as officially recommended), and Foxglove makes good RViz2's limits in analyzing recordings, in collaboration and sharing, and with non-ROS systems; the three complement each other rather than replace each other.
6. The five steps for reading an open-source project: read the paper before the code, read the README to confirm the dependencies and the hardware requirements, get it running before reading the code, follow the data flow down from the entry point, and add print statements to verify your understanding; and make good use of the issue tracker and of forks maintained by the community.
7. The path of learning has five stages: observing and experiencing, block-based operation, data and analysis, reading source code, and modifying and innovating; the principles of teaching are to succeed first and go deep after, to demonstrate failure deliberately, and to keep environment problems apart from the subject matter.

The environment and the tools are now ready, and students have the means to make all the theory before this actually run. In the next chapter we implement the 2D SLAM of Part IV from end to end: assembling the hardware and setting up TF, publishing odometry, mapping with Gmapping and Cartographer, localizing with AMCL, and finally connecting the complete navigation stack so that the robot really walks to a goal by itself. This will be the book's first complete end-to-end system implementation.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 29

Mapping with a 2D LiDAR Rover

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

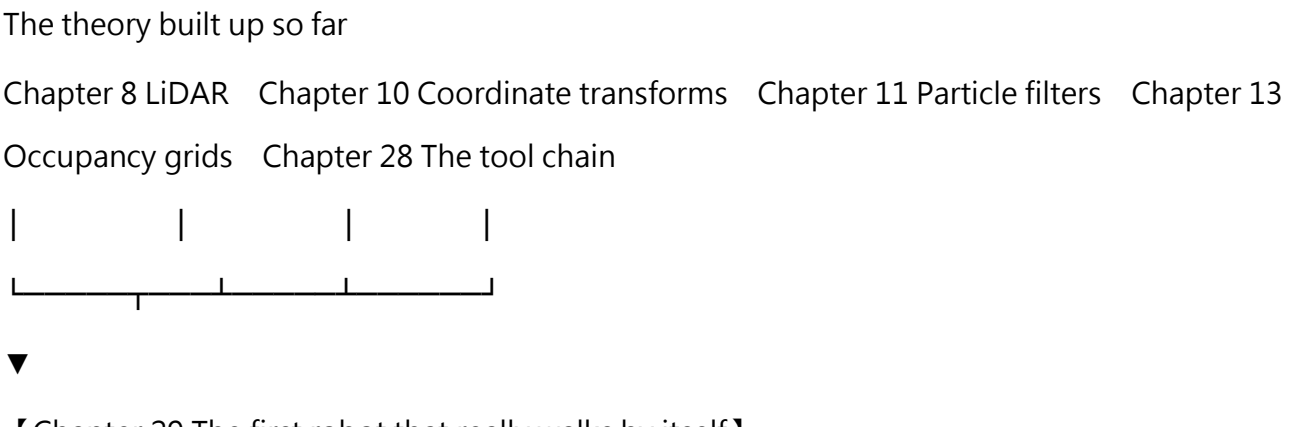
Where This Chapter Sits in the Book

This is the book's first end-to-end system implementation—from the turning of a wheel to the robot walking to a goal by itself, with every link in between to be connected by hand.

What most distinguishes this chapter from those before is that it deals with the places where things are wired together. The algorithms themselves were covered in Chapters 13, 14, and 15; what really holds a project up is a coordinate transform set wrongly, odometry parameters never calibrated, and not knowing where to begin looking when the system will not move. What these problems have in common is that they raise no error—they only make the result look somehow odd.

The three figures of this chapter exist to turn somehow odd into something measurable: Figure 29-2 demonstrates the standard method of calibrating odometry and explains why one must drive once clockwise and once counterclockwise; Figure 29-3 sets side by side the maps built under four common coordinate transform errors, as a table of symptoms; and Figure 29-4 runs an actual global localization and sees how many steps the particles need to converge.

Figure 29-0 The Link Map of Chapter 29



debugging

▼

Chapter 30 Visual SLAM in practice (changing to a camera)

▼

Chapter 31 Integrating a project and assessing it

Learning Objectives

The theory and the tools of the twenty-seven chapters before are now all in hand. This chapter is the book's first complete system implementation from end to end: beginning with the hardware and the coordinate settings, through mapping and localization, and ending with the robot walking to a goal by itself. After studying this chapter, students should be able to:

1. Explain what a 2D SLAM rover is made of and the role of each component.
2. Write a URDF robot description and set up the TF coordinate tree correctly.
3. Compute and publish odometry from the wheel encoders.
4. Run 2D SLAM to build a map and save the map file.
5. Explain the key settings of AMCL localization and its common problems.
6. Explain the architecture of the Nav2 navigation stack and the complete data flow of one navigation task.
7. Troubleshoot a navigation system in a systematic order.

Core Concepts

中文名詞	English Term	Explanation
機器人描述檔	URDF	An XML description of a robot's links, joints, and sensor mountings.
連桿與關節	Link / Joint	The basic elements of URDF: the rigid bodies and the joints connecting them.
狀態發布器	robot_state_publisher	The node that publishes the TF transforms among the links from the URDF.
差速驅動	Differential Drive	A drive in which direction is controlled by the difference between the two wheel speeds.
里程計	Odometry	The pose reckoned by integrating wheel speeds, supplying the odom→base_link transform.
代價地圖	Costmap	A navigation map annotating an occupancy grid with the cost of passage.
膨脹層	Inflation Layer	The cost map layer expanding obstacles outward by a margin of safety.
全域規劃器	Global Planner	Plans a complete path from start to goal on the static map.
控制器	Controller	The local planning component turning a path into velocity commands.
行為樹	Behavior Tree	The structure Nav2 uses to organize the flow of a navigation task.
復原行為	Recovery Behavior	Actions attempted to escape when navigation fails—turning in place, backing up.
生命週期節點	Lifecycle Node	A kind of ROS 2 node with explicit transitions among start-up states.
速度指令	cmd_vel	The linear and angular velocity command sent to the chassis.

29.1 The Hardware: Chassis, Motors, Encoders, and a 2D LiDAR

This section explains what components a 2D rover able to run SLAM needs, and what to consider in choosing them.

The Five Components

- The chassis and motors: the commonest arrangement is differential drive—two driven wheels with one or two casters. It is simple in structure and intuitive to control (the 2D pose of Section 10.3 corresponds to it exactly), and it is the standard choice for a teaching platform.
- The encoders: fitted to the motor shaft or the wheel axle, they count how far the wheel has turned. They are the source of the odometry (Section 29.3), and the indispensable half of Section 13.4's prediction by odometry, correction by LiDAR.
- The 2D LiDAR: the rotating laser scanner introduced in Chapter 8, which supplies the /scan topic. An entry-level model for teaching measures to about 8 to 12 meters, which is enough for indoor mapping.
- The microcontroller: it drives the motors and reads the encoders and communicates with the computer above it. What it handles is real-time control at the millisecond scale, which does not belong on a single-board computer.
- The single-board computer: it runs ROS 2 and the SLAM algorithm. This division—the microcontroller for real-time control and the single-board computer for computation—is exactly the two-chip architecture discussed in Case 25-2 of Chapter 25, embodied in a teaching platform.

Three Considerations in Choosing

1. The LiDAR's mounting height and occlusion: the LiDAR must have a clear 360-degree view. If the body or another component blocks part of the angle, those directions are blind forever. Check on mounting that the scan is really unobstructed.
2. The resolution of the encoders: too low a resolution and the odometry will be inaccurate. The encoders of an ordinary teaching platform suffice, but with a chassis of your own making, confirm that the pulses per revolution are enough to resolve a small movement.
3. Computing power: 2D SLAM does not demand much computation (as against the 3D and visual SLAM of Parts V and VI), and a common single-board computer will serve. If Nav2 navigation is to run at the same time, mind whether the memory suffices.

Three Choices of Platform

- A ready-made teaching platform: completely integrated, fully documented, and supported by a community, it lets students get quickly to SLAM itself rather than to debugging hardware. It suits a course with limited time.
- A platform of your own assembly: cheaper, and deeper as a lesson—students handle every link by hand and understand the system far better than with a ready-made platform. It suits a project and a course with time to spare.
- Simulation alone: no hardware at all, and yet the whole of the procedures of Sections 28.2 to 29.7 can be experienced. Chapter 28's advice of simulate first, then hardware applies to this chapter particularly—get the whole pipeline running in simulation and then move it to the real rover, and a great deal of debugging time is saved.

29.2 Setting Up TF and URDF

Chapter 10 stressed that when the map is crooked, look at TF first; this section explains how to build the coordinate tree correctly. It is the part of the whole implementation most apt to go wrong and most in need of care.

URDF: the Skeleton of the Robot

URDF (the Unified Robot Description Format) describes the physical structure of a robot in XML: what rigid bodies (links) there are, how they are joined (joints), and the size and relative position of each.

For SLAM, URDF's most important function is to describe where the sensors are mounted—the extrinsics mentioned repeatedly in Chapters 5 and 10. If the LiDAR position written in the URDF differs from the actual mounting by a few centimeters or a few degrees, the map built will be blurred or out of register.

The Typical Structure of the Coordinate Tree

Section 10.7 introduced the standard hierarchy of frames in ROS; on a real rover it takes this concrete form:

Figure 29-1 The Complete TF Tree of a 2D SLAM Rover

map (the map frame) globally fixed, does not drift, jumps

↓ published by SLAM or AMCL

odom (the odometry frame) continuous and smooth, drifts over time

↓ published by the odometry node (Section 29.3)

base_footprint (the robot's projection on the ground; optional)

↓

base_link (the center of the robot body)

├──→ laser (the LiDAR) ↖

├──→ imu_link (the IMU) published from the URDF by

├──→ left_wheel robot_state_publisher on the fixed mountings

└──→ right_wheel ↙

Who Publishes Which Segment

This is where a beginner is most apt to be confused, and it must be made clear:

- map → odom: published by the SLAM system (while mapping) or by AMCL (while navigating). The official documentation states explicitly that this transform is one of the principal requirements of the Nav2 system.
- odom → base_link: published by the odometry system. The role of the odometry system is precisely to supply this transform, and its source may be LiDAR, radar, wheel encoders, visual-inertial odometry, or an IMU; the aim is to supply a smooth, continuous local frame based on the robot's motion.

- base_link → each sensor: supplied by robot_state_publisher from the URDF, which publishes the transform from base_link to each sensor.

Common TF Errors

1. A broken coordinate tree: some segment has no node publishing it, so that the chain from map to laser is broken. RViz shows a cannot find transform error.
2. Wrong extrinsics: the LiDAR's mounting position or orientation is entered wrongly. The symptom is that the scan points are out of register with the real environment—put the robot against a wall, and if the scan points in RViz do not line up with the wall, this is the problem.
3. Duplicate publishing: two nodes publish the same segment at once, which causes conflict and jitter. This happens easily when several packages are integrated.
4. Time stamp problems: the transform's time stamp is not synchronized with the sensor data, which causes plain misalignment during fast motion (Section 10.6 noted the importance of time stamps).

How to Verify

- A visual check: display the TF axes and the LiDAR scan together in RViz and confirm that the scan points sit sensibly relative to the robot.
- Produce a picture of the TF tree: generate a graph of the coordinate tree with the tool and confirm that the structure is complete, with nothing broken or duplicated.
- A static check: place the robot at a known position (hard against a corner, say) and check whether the scan points fall at the right distances.

29.3 Publishing the Odometry

Odometry is the other cornerstone of 2D SLAM. This section explains how it becomes a ROS odometry message from the readings of the encoders.

From Encoder to Pose

The odometry computation of a differential-drive rover is a direct application of the dead reckoning of Chapter 3 and the coordinate transforms of Section 10.3:

1. Read the encoders: obtain how far the left and the right wheel each turned within this interval.
2. Convert to wheel displacement: multiply the revolutions by the circumference of the wheel to get how far each wheel travelled.
3. Compute the body motion: the mean of the two displacements gives the distance advanced, and their difference divided by the wheel base gives the turn (the formula is Formula One of this chapter).
4. Update the pose: by the 2D pose composition of Section 10.3, add the relative motion onto the current pose, remembering to handle the wrapping of the angle.
5. Publish: publish the odometry message (carrying pose and velocity), and publish the odom → base_link transform at the same time.

Calibration: the Key to Accuracy

Section 14.6 noted that wheel slip and inaccurate odometry are the first cause of a failed map, and what within that is most often overlooked is the calibration of the parameters. Two parameters are crucial:

- The wheel diameter: if the value set differs from the real one, the distance travelled in a straight line is systematically off, and the whole map is scaled up or down in proportion. To calibrate, drive the rover straight a fixed distance (3 meters, say) and compare the odometry reading with a tape measure.
- The wheel base: the distance between the centers of the two driven wheels. Set it wrongly and the angle turned is off, which affects the map more severely than the wheel diameter does—because an error in angle is magnified by the movement that follows (Section 7.3 discussed the magnifying effect of an attitude error). To calibrate, turn the rover ten times in place and compare the angle the odometry accumulates with 3,600 degrees.

These two calibrations should be done before the first map is built. Many students spend hours tuning SLAM parameters without noticing that the trouble lies in uncalibrated odometry—the commonest wrong road in implementation.

What Is Required of the Odometry

The aim of odometry is to supply a smooth, continuous local frame, while the global localization system updates the transform relative to the global frame so as to compensate the odometry's drift.

Note the requirement of smooth and continuous: odometry need not be absolutely accurate (SLAM will correct the drift), but it must be smooth and not jump. If the odometry jumps because of a bug in the program or a dropped packet, SLAM's motion prediction fails and the map is bound to fail with it.

Going further: Fusing the IMU

The angle estimated from wheel speed alone goes wrong easily through slip. In practice a filter often fuses the wheel speeds with the IMU's angular rate (Chapters 7

and 11) for a more robust odometry. In the ROS 2 ecosystem the EKF node of the robot_localization package can supply the odom → base_link transform, which helps particularly where the ground is uneven or slippery.

Measuring It: What Happens When the Odometry Parameters Are Wrong

Section 29.3 says the odometry must be calibrated, but not how far wrong it goes uncalibrated, nor how to tell which parameter is at fault. This subsection answers both at once with the standard square path test.

A differential-drive odometry has only two crucial parameters: the wheel radius and the wheel base. Each causes an error of its own character: set the wheel base too small and the robot believes it is turning more than it is, so that every turn falls short; let the left and right wheels differ in diameter and the robot veers to one side while driving straight, though its odometry believes it went straight. The trouble is this—from one experiment alone the two errors look exactly alike.

The method of the experiment is the classic square path test: command the robot to drive a square 2 meters on a side by its own odometry and return to the start, then measure where it actually stopped. What is crucial is to drive it once clockwise and once counterclockwise.

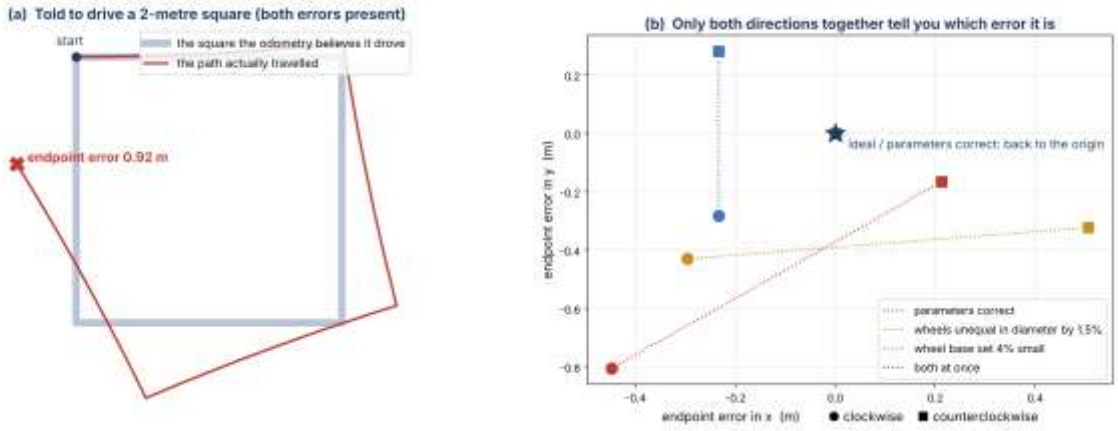


Figure 29-2 The Square Path Test: the Fingerprint of an Odometry Error

Panel (a) draws the result of a rover carrying both errors at once (the wheel base set 4 percent small, the right wheel actually 1.5 percent larger than the left) driving the square clockwise. The thick grey line is the path the odometry believes it travelled—a perfect square; the red line is the path it actually travelled. Every corner falls short of the turn and every straight stretch curves slightly, and over four sides the end stands 0.92 meters from the start. In a site 2 meters on a side, that error is nearly half the length of a side.

Panel (b) is the point of this figure. Four cases are each run clockwise (circles) and counterclockwise (squares), and the endpoint error of each is drawn on one plot. With the parameters correct, both points fall at the origin (an error of 0.001 meters).

The two points for a wheel base 4 percent wrong (blue) fall at the same x with y equal and opposite: (−0.233, −0.282) and (−0.233, +0.282). That arrangement—same x, mirrored y—is the fingerprint of a pure wheel base error, because the error in the turn angle is symmetric between the two directions.

Wheels unequal in diameter by 1.5 percent (amber) is wholly different: the two points fall at (−0.296, −0.430) and (+0.506, −0.322), far apart and with no plain symmetry. The

direction in which it veers while driving straight does not change with the direction of travel, but the order of the turns changes how that veer accumulates.

With both errors present at once (red), the endpoint error is 0.92 meters clockwise and only 0.27 counterclockwise. This is exactly why driving one direction alone will deceive you: a student who tried only counterclockwise might see 0.27 meters and think it tolerable, with no idea that the other direction is out by 0.92.

The Order of Calibration

By the fingerprints of Figure 29-2(b), the calibration may be done in two steps.

The first step deals with the wheels being unequal in diameter: adjust the ratio of the left and right radii until the clockwise and counterclockwise endpoints fall roughly symmetric about the origin. What remains is then mainly the wheel base error.

The second step adjusts the wheel base: if the turns fall short (the endpoint lies to the inside of the path), raise the wheel base value; otherwise lower it. Rerun both directions after each adjustment, until both points are near the origin.

One practical reminder: the result of a calibration changes with the floor material (carpet and tile differ in how much the wheels slip) and drifts as the tires wear. This test is therefore worth repeating at the start of every semester or on changing site—it takes ten minutes and saves all the debugging of why is the map crooked afterwards.

29.4 Building a Map with 2D SLAM

With the preparation done, a map can at last be built. This section explains the procedure and the choice of tool.

The Choice of Tool

Chapters 14 and 15 introduced Gmapping and Cartographer, and in a modern ROS 2 environment there is a third common choice. Nav2 officially lists SLAM Toolbox as its default SLAM algorithm, used to localize and to produce a static map.

Tool	Theoretical basis	Features	Where it suits
slam_toolbox	Graph optimization (Chapter 12)	The ROS 2 default; few parameters; supports continuing a map	The first choice for teaching
Cartographer	Graph optimization + submaps	Strong loop closure; supports 3D; more parameters	A large site needing loop closure
Gmapping	Particle filter (Chapter 11)	Intuitive in concept; the mainstream of the ROS 1 era	Understanding the particle filter

Table 29-1 A Comparison of Three 2D SLAM Tools

Teaching advice: build an experience of success with slam_toolbox first (it has few parameters and is easy to take up), then feel the power of loop closure with Cartographer (Implementation 15-1 in Chapter 15), and finally set Gmapping beside them to understand the difference between a particle filter and graph optimization.

The Mapping Procedure

1. Start the hardware: bring up the robot driver, the LiDAR, and the odometry node, and confirm that both /scan and /odom carry data.
2. Check TF: confirm that the coordinate tree is complete (the verification methods of Section 29.2). This step may not be skipped.

3. Start SLAM: start the chosen SLAM node. Confirm that it supplies the map → odom transform and publishes the /map topic.
4. Start RViz: display the map, the scan, TF, and the robot model, so that it may be watched live.
5. Drive slowly and map: Section 14.6 stressed that driving a little slower is the single variable that most raises a beginner's rate of success. Be slower still in the turns.
6. Assess the quality: by Section 13.7, check whether the walls are straight, whether the loop closes, and whether anything is doubled.
7. Save the map: save it with nav2_map_server, which produces two files, a .yaml and a .pgm, to represent the map.

The Two Files of a Map

- The .pgm: a grayscale image file, which is the occupancy grid map visualized—black for obstacle, white for free space, grey for unknown (the three states of Section 13.1). It can be opened and inspected in any image viewer.
- The .yaml: the description file, recording the image file's name, the resolution (meters per pixel), the coordinates of the origin, and the probability thresholds for judging occupied and free.

These two files are the map product mentioned in Section 14.5. In navigation afterwards they are loaded by map_server for AMCL to localize against and for path planning.

Measuring It: What a Map Looks Like When the Transform Is Wrong

Section 29.2 set up the coordinate transform tree and Section 29.7 lists it first on the debugging checklist. But what does a wrong transform actually look like? This

subsection sets four common errors deliberately and puts the resulting maps side by side, as a table of symptoms.

The experimental setting is an indoor scene of 10 × 7 meters; the robot goes once around an elliptical trajectory and builds an occupancy grid map with a 240-beam 2D LiDAR. The real LiDAR is mounted at the center of the body, while the extrinsics the system believes are set wrongly on purpose.

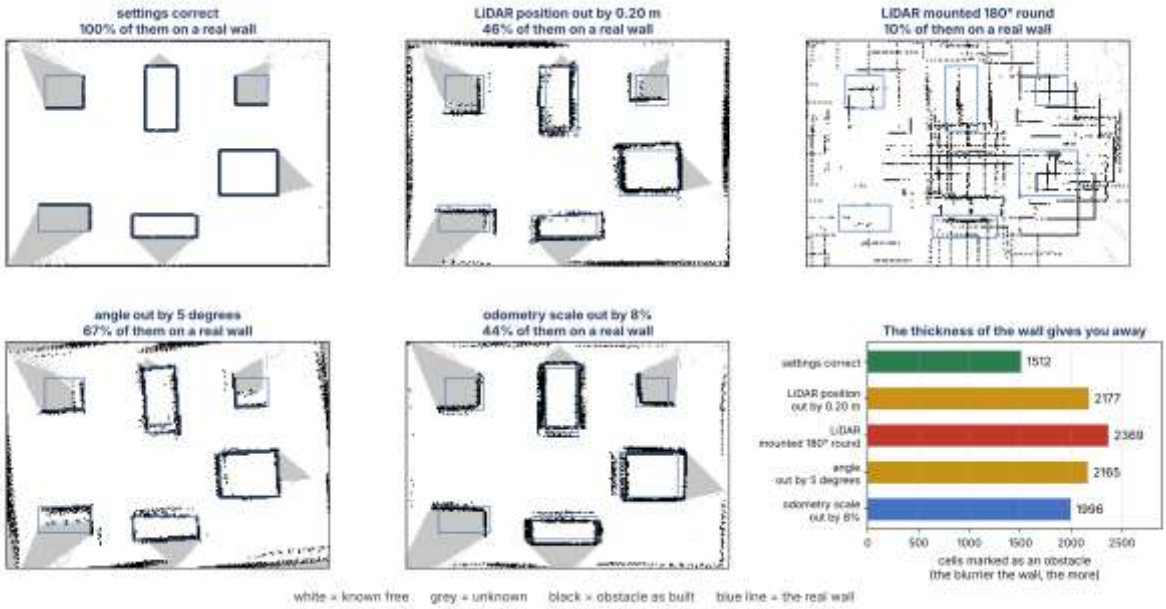


Figure 29-3 Four Symptoms of a Wrongly Set Transform

With the settings correct the walls are clean single lines, 1,512 cells are marked as obstacles, and 100 percent of them fall on a real wall. This is the baseline.

With the LiDAR position out by 0.20 meters—the commonest error of all, usually from failing to measure the thickness of a flange or from entering the position of the LiDAR's housing rather than its optical center in the URDF. The map still looks like that room and the walls are still where they should be, but the obstacle cells leap from 1,512 to 2,177 (1.4 times) and only 46 percent fall on a real wall. The symptom is a wall grown thicker, and doubled.

The symptom of a 5-degree angular error is almost the same (2,165 cells, 67 percent). This shows something important: from the look of the map you cannot tell a position error from an angular one—only that there is an error. To tell which, you must go back and check the numbers in the transform tree.

A LiDAR mounted 180 degrees the wrong way round is the most dramatic of the four. The error is not rare in practice—the mounting direction of a LiDAR and the definition of its coordinate frame need not agree. The result is that every measurement is projected somewhere else entirely: 2,369 obstacle cells are scattered over the map, and only 10 percent of them land in the right place. This error is in fact the easiest to spot, because the map does not take shape at all.

An odometry scale 8 percent off is of another kind—not an error of the transform but a calibration of Section 29.3 not properly done, though the symptom shows on the map just as well: 1,996 cells, 44 percent correct. The walls thicken likewise, but by an amount that grows with the distance from the start.

Setting these five maps together gives a very practical rule of diagnosis: how thick the walls are is how large the system's total error is. A clean single-line wall means every link is right; a doubled or blurred wall means some systematic bias somewhere; and a map that does not take shape at all is usually the easiest kind of error to find.

A Check That Needs No Programming

Put the robot at a fixed point, do not move it, and simply let it turn once in place while mapping.

If the extrinsics are right, every measurement during the turn falls on the same wall line and the map is clean. If the LiDAR's mounting position is entered

wrongly, the measurements trace a ring on the map—and the radius of that ring is exactly how far wrong you are.

The check takes thirty seconds and verifies at once the two commonest errors of Figure 29-3, with the radius telling you directly by how much. Putting it into the standard procedure after every change of hardware is advised.

29.5 Localizing with AMCL

Once the map is built, running day to day needs no more mapping but only localization within a known map—which is the AMCL introduced in Chapters 3 and 11.

What AMCL Is

AMCL is a probabilistic localization system for a 2D mobile robot, implementing the adaptive (or KLD-sampling) Monte Carlo localization method and tracking the robot's position on a known map with a particle filter. It is no longer a SLAM algorithm, because it builds no map of the environment.

Each iteration of the algorithm compares the appearance of the map with the current laser scan and keeps only those positions where the data best agree with the map. This is exactly the predict–weigh–resample cycle of the particle filter of Chapter 11.

The Division of Labor Between AMCL and SLAM

Item	SLAM (the mapping stage)	AMCL (the navigation stage)
Does it map	Yes	No; it uses an existing map

Item	SLAM (the mapping stage)	AMCL (the navigation stage)
Source of the map	Built live	Loaded by map_server
Computational load	Higher	Lower
The TF it supplies	map → odom	map → odom
When it is used	On first entering a new environment	In day-to-day running

Table 29-2 The Division of Labor Between SLAM and AMCL

Note that both supply the map → odom transform—which is exactly the ingenuity of the design of Section 10.7: to the navigation system downstream the interface is identical whether SLAM or AMCL is supplying the localization, so the two may be switched seamlessly.

The Key Parameters

- The number of particles: as discussed in Section 14.3, more is more robust and costs more computation. AMCL's adaptive character adjusts the number dynamically by the uncertainty—more particles when uncertain, fewer once converged.
- The motion noise model: the parameters describing how inaccurate the odometry is. Of these, the one governing the effect of rotation on the estimate of the robot's rotation, raised, makes the system more cautious about rotational motion and assume greater uncertainty; lowered, it is more confident of the rotation estimate, which is a risk if the sensors are not accurate enough.

- The update thresholds: how far the robot must move or turn before a filter update is run. A smaller angular threshold (0.05 radians rather than the default 0.2, about 11.5 degrees) lets the robot update after less rotation, which suits slow or precise movement; raised, it reduces the number of updates and saves computation.
- The transform tolerance: how long a published transform is treated as valid. The default of 1 second is a reasonable value; raised, it helps with communication latency but may use a stale position; lowered, it keeps the data fresh but may cause jitter.

The Initial Pose: the Commonest Problem

AMCL needs a guess at the initial pose when it starts. Without one the particles are strewn over the whole map, the robot may have to travel some distance before they converge, and they may even converge on the wrong place (the multi-modal problem of Chapter 11).

- Setting it by hand: click the robot's approximate position and heading with the 2D Pose Estimate tool in RViz. This is the commonest practice.
- A fixed starting point: always start the robot from the same place (the charging station, say) and write the initial pose into the configuration. This is the practical technique mentioned in Section 27.5.
- With the help of a visual marker: supply an absolute pose with an AprilTag (Section 24.3) and let the initialization happen automatically.

Measuring It: How Many Steps Does Global Localization Need

Section 29.5 explained AMCL's principle and its division of labor with SLAM. This section runs an actual global localization—the robot has no idea where it is, the

particles begin spread evenly, and we see how many steps it must walk before the position is certain.

The experimental setting: the same scene of 10 × 7 meters, with a prior map. Four thousand particles are scattered at random over every traversable position, with headings wholly random too. The robot moves along a path of about 5 meters, and at every step a 60-beam LiDAR measurement updates the weights and resamples.

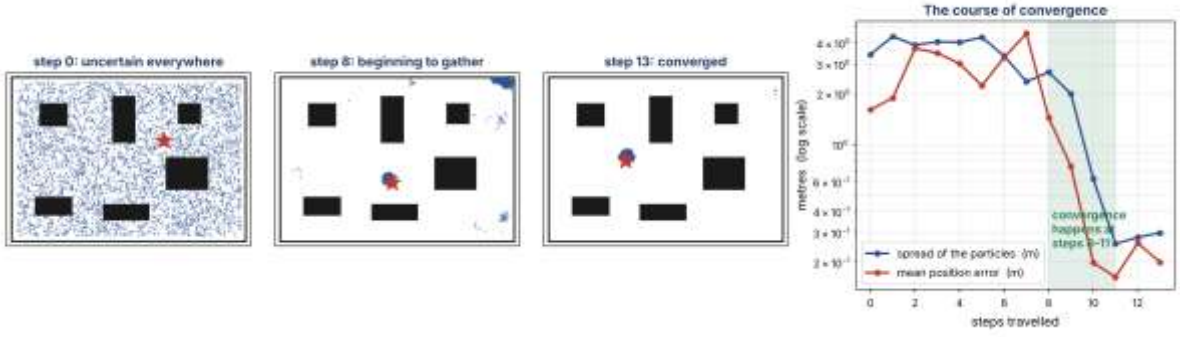


Figure 29-4 The Convergence of AMCL Global Localization

At step 0 the particles cover the whole traversable region and the spread is 3.42 meters—the system knows nothing of its position. By step 8 they have gathered into a few clumps and the spread has fallen to 2.69 meters; those clumps are the several positions that all look plausible. By step 13 they have converged into one small clump, with a spread of 0.30 meters and a mean position error of 0.20 meters.

The convergence curve on the right shows something worth noting: there is almost no progress over the first seven steps (the spread even rises above the value it started at), and the real convergence is concentrated between steps 8 and 11. The reason is that over the first few steps the robot sees only a short stretch of corridor, and several places in the scene look much alike; not until it moves where two different walls can be seen at once are the several hypotheses ruled out together.

This property—that it must walk a way before the position can be fixed—bears directly on practice: do not expect AMCL to be right the instant it starts, but drive the robot a few meters about where it stands and hand it to the navigation stack only once the particle cloud has converged. Many a case of the robot charging about the moment navigation starts has no other cause than a goal given before the localization had converged.

A Pitfall Stumbled Over in Implementing This

The first time this experiment was run, the particles did converge—on the wrong place.

There were two reasons, both typical. The first was a mistake in the motion update: the odometry gives a displacement in world coordinates, but every particle has its own heading, and the displacement must be turned into each particle's own body frame before being applied. Adding the world displacement to every particle directly amounts to assuming they all face the same way.

The second was failing to reject particles that fell inside a wall or outside the map. The likelihood of those particles was looked up in a clipped distance field, so their values were spuriously high, and the whole cloud drifted out of the room.

Neither error crashes the program; both merely make the localization look as though it is moving and yet simply wrong—exactly the hardest class of problem to find, as Section 29.7 says.

29.6 Navigating with the Navigation Stack

The last step: letting the robot walk to a goal by itself. This is the navigation architecture of Section 3.4 brought to ground.

The Overall Architecture of Nav2

Nav2 is the navigation stack of ROS 2, and several modules work together within it. A navigation goal, from the moment it is sent to its execution, passes through the behavior tree navigator, the planner, the controller, the cost maps, the recovery behaviors, and the velocity command, and so reaches the chassis.

Figure 29-5 The Complete Data Flow of One Navigation Task

① The user sends a goal (clicked in RViz, or sent by a program through the action interface)

↓

② The behavior tree navigator: decides the flow of execution by the behavior tree

↓

③ The global planner: plans a complete path on the global cost map (inputs: the static map + the current pose from AMCL)

↓

④ The controller: turns the path into velocity commands by the local cost map (the local cost map brings in live obstacles the LiDAR detects)

↓

⑤ Publish /cmd_vel → the chassis executes → the robot moves

↓

⑥ If stuck or the path fails → trigger a recovery behavior (turn in place, back up, clear the cost maps)

↓

⑦ On reaching the goal → report success; on failure report the reason

The Cost Map: the Central Data Structure of Navigation

The costmap package uses sensor information to supply a representation of the robot's environment in the form of an occupancy grid; the cells store a cost between 0

and 254 representing the cost of passing through that region. A cost of 0 means the cell is free and 254 means lethally occupied, while the values between are used by the navigation algorithms like a potential field to steer the robot away from obstacles.

A cost map is implemented in several layers, each with a particular function and all contributing to a cell's overall cost. The common layers include:

- The static layer: loads the map that was built (the product of Section 29.4).
- The obstacle layer: brings in live obstacles detected by the LiDAR, including moving objects not on the map.
- The inflation layer: expands every obstacle outward by a margin of safety. This layer is extremely important—without it the planned path hugs the wall and the robot is certain to scrape. The inflation radius should be the robot's radius plus a margin.

The Division Between Global and Local

- The global cost map: covers the whole map, is based mainly on the static map, and lets the global planner compute a complete path. It updates at a low rate.
- The local cost map: covers only a few meters about the robot, is based mainly on live sensing, and lets the controller dodge moving obstacles. It updates at a high rate.

This division—global and slow and complete, local and fast and live—should feel familiar: it is the same design pattern as Cartographer's two subsystems in Chapter 15, LOAM's two layers in Chapter 18, and the layered design of the drone in Chapter 26.

29.7 Systematic Debugging: When the Robot Will Not Move

The most practical section of this chapter. A navigation system has many

components, and without a systematic order of investigation it is easy to fire at random and waste time when something goes wrong.

The Right Order of Investigation

The debugging order advised is: check in turn the lifecycle states, TF, the map server, AMCL, the cost maps, the planner's output, the controller's output, and only last the motion of the chassis.

More important still is a principle often overlooked: the tuning of the planner and the controller should wait until the lifecycle, TF, localization, the map, and the cost maps are all healthy.

The logic behind this order is that a problem downstream usually comes from upstream. If TF is wrong, no amount of tuning the planner's parameters will help. Students should form the habit of investigating from upstream down.

Figure 29-6 The Debugging Checklist for a Navigation System

① Lifecycle states: have all the Nav2 nodes started and entered the active state?

↓ passed

② TF: is the chain map → odom → base_link → laser complete?

↓ passed

③ The map: has map_server loaded it successfully? Can the map be seen in RViz?

↓ passed

④ Localization: has AMCL converged? Is the particle cloud gathered in the right place?

↓ passed

⑤ The cost maps: are the global and local cost maps displayed correctly? Are the obstacles inflated?

↓ passed

⑥ The planner: after a goal is sent, is a path planned?

↓ passed

⑦ The controller: is /cmd_vel being published? Are the values reasonable?

↓ passed

⑧ The chassis: do the wheels turn on receiving /cmd_vel? In the right direction?

Common Symptoms and the Checkpoints for Each

Symptom	The likeliest cause	Order of checking
No response at all	A node did not start or did not become active	①
RViz shows a transform error	A broken TF tree or wrong extrinsics	②
The robot's position jumps about	AMCL has not converged or the initial pose is wrong	④
No path can be planned	The goal is on an obstacle; the cost map is inflated too far	⑤⑥
A path is planned and it does not move	A controller problem; /cmd_vel is not reaching the chassis	⑦⑧
It keeps hitting walls	The inflation radius is too small; the LiDAR extrinsics are wrong	②⑤
It spins in place	Improper angular control parameters; the odometry's angle is inaccurate	Calibrate by Section 29.3

Table 29-3 Common Failure Symptoms and the Order of Checks

Leave Evidence at Every Stage

One good habit in implementation is this: every checkpoint should produce verifiable evidence—a line of command output, a visible coordinate frame in RViz, a map file, a simulated robot in motion, a clean code repository, or a short demonstration recording.

The habit has three values: in debugging it locates quickly which stage the problem is in; in a report or a project there is something concrete to show; and the accumulation is itself a complete record of what was learned. Students are advised to form the habit from this chapter on—the project assessment of Chapter 31 requires exactly this evidence.

At this point students have completed their first full system from sensor to autonomous navigation. In the next chapter we change the sensor to a camera and implement RGB-D and stereo visual SLAM: setting up a RealSense camera, calibrating it, producing point clouds, implementing RTAB-Map and ORB-SLAM3, and comparing how visual SLAM and LiDAR SLAM perform on the same site.

29.8 Turning the Three Figures into a Start-Up Checklist

The three experiments of this chapter each answer one question, and together they make up a procedure worth running through at every start-up.

Order	Check	Method	Standard to pass
1	The transforms	Turn once in place while mapping	The wall is a single line, with no ring
2	The odometry	Drive a 2-meter square once each way	Both endpoints within 5 centimeters
3	Mapping	Go once around the site	The obstacle cell count is near the baseline and the walls do not thicken
4	Localization	Do a global	The particle spread

		localization on a known map	is under 0.3 meters within 5 to 10 steps
5	Navigation	Send a goal 3 meters away	It arrives without collision

Table 29-4 The Start-Up Checklist

The order of this checklist is not arbitrary but follows the direction in which errors propagate downstream. Get the transform wrong and the result of calibrating the odometry means nothing; let the odometry be inaccurate and the map comes out crooked; let the map be crooked and the localization will not converge; let the localization be inaccurate and the navigation is bound to fail. Begin from downstream and a great deal of time is wasted on symptoms.

How Long Each Item Should Take

Thirty seconds for the first, ten minutes for the second, five for the third, two for the fourth, two for the fifth—under twenty minutes for the whole. Against three days spent late in a project looking for why the robot keeps hitting the wall, it is an excellent investment.

More important still, this checklist turns it feels somehow odd into numbers that can be recorded. The obstacle cell count, the endpoint error, the particle spread—record these three every time and you will see whether the system is degrading (tires wearing, a sensor working loose, the floor growing dirty). It is the same line of thought as the map maintenance of the inspection robot in Section 19.6: a system that will run for a long time needs health indicators that can be tracked.

A Suggestion for the Classroom

For a teacher running a practical course this checklist has a further use: it can serve as

an objective basis for marking. Rather than grade whether the robot ran (which luck affects greatly), grade what the numbers of these five checks were—the first is binary, the second continuous, and it distinguishes far better how deeply a student really understood.

Formulas and Procedures

One: the Odometry Computation of a Differential Drive

Let the left and right wheels travel d_L and d_R within one sampling period (obtained from the encoder pulse counts times the distance per pulse), and let the wheel base be b; then the motion of the body is:

$$\Delta s = (d_R + d_L) / 2 \quad \textit{(the distance advanced)}$$

$$\Delta\theta = (d_R - d_L) / b \quad \textit{(the turn)}$$

Then update the pose by the 2D pose composition of Section 10.3:

$$x \leftarrow x + \Delta s \cdot \cos(\theta + \Delta\theta / 2)$$

$$y \leftarrow y + \Delta s \cdot \sin(\theta + \Delta\theta / 2)$$

$$\theta \leftarrow \textit{normalize}(\theta + \Delta\theta)$$

Taking θ + Δθ / 2 computes the displacement with the mean heading over this piece of motion, which is more accurate than using the starting heading. normalize is the wrapping of the angle stressed in Section 10.3, and may not be omitted.

Note that the denominator of Δθ is the wheel base b—which explains why Section 29.3 says a wrong wheel base affects the map more severely than a wrong wheel diameter: a wheel base error causes an angular error directly, and an angular error is magnified by the movement that follows.

Two: Converting Pulses to Distance

What an encoder outputs is a count of pulses, and converting it to a distance requires:

$$distance\ per\ pulse = (\pi \times D) / (N \times G)$$

where D is the wheel diameter, N the encoder's pulses per revolution, and G the gear ratio (if the encoder is on the motor shaft). This constant is the foundation of the odometry's accuracy, and must be calibrated by measurement rather than taken from the specification sheet alone.

Three: Calibrating the Wheel Diameter and the Wheel Base

Figure 29-7 The Procedure for Calibrating the Odometry Parameters

【Calibrating the wheel diameter】

① Drive the rover a fixed distance in a straight line (3 meters or more is advised)

② Measure the actual distance L_real with a tape and read the odometry's distance L_odom

③ Correct: new wheel diameter = old wheel diameter × (L_real / L_odom)

④ Verify again and repeat until the error is under 1%

【Calibrating the wheel base】

① Turn the rover ten times in place (once clockwise and once counterclockwise)

② Read the angle the odometry accumulated, θ_odom, against the theoretical 3,600 degrees

③ Correct: new wheel base = old wheel base × (θ_odom / 3600°)

④ Verify again and repeat until the error is under 2%

【Order】calibrate the wheel diameter first and the wheel base after; begin mapping only when both are done

Four: the Inflation Radius of the Cost Map

The key parameter of the inflation layer is the inflation radius, which decides how far an obstacle is expanded outward:

inflation radius ≥ the radius of the robot's circumscribed circle + a margin of safety

Set it too small and the planned path hugs the wall and the robot scrapes; set it too large and a narrow passage is sealed off entirely so that no path can be planned (a common symptom in Table 29-3). In practice set it first at 1.2 to 1.5 times the robot's radius and adjust by the width of the passages on site.

Five: the Complete Procedure from Nothing to Autonomous Navigation

Figure 29-8 The Complete Implementation Procedure for a 2D SLAM Rover

【Stage one: hardware and coordinates】

① Assemble the hardware → ② Write the URDF → ③ Verify that the TF tree is complete

↓

【Stage two: odometry】

④ Implement reading the encoders and computing the odometry → ⑤ Calibrate the wheel diameter and wheel base (Figure 29-4)

↓

【Stage three: mapping】

⑥ Start SLAM → ⑦ Drive slowly and map → ⑧ Assess the quality → ⑨ Save the map

↓

【Stage four: localization】

⑩ Load the map → ⑪ Start AMCL → ⑫ Set the initial pose → ⑬ Confirm convergence

↓

【Stage five: navigation】

⑭ Start Nav2 → ⑮ Adjust the cost maps → ⑯ Send a goal → ⑰ Watch it execute

↓

【Stage six: debugging and tuning】investigate and optimize in the order of Figure 29-3

Cases

Case 29-1 The Cost of Three Millimeters: an Uncalibrated Wheel Diameter

This happened in a real class and is highly representative.

One group's maps always came out somehow odd: the map looked complete and the wall lines straight enough, but comparing the size of the classroom on the map with the real measurement showed the map's classroom about 8 percent too large. Stranger still, they spent a whole afternoon adjusting every SLAM parameter with no improvement at all.

The trouble lay in the wheel diameter. The wheels they used had a nominal diameter of 65 millimeters, but once fitted, the rubber tire deformed under load and the effective diameter was only about 60. The odometry computed with 65, believed it had travelled further than it had, and the map came out scaled up in proportion.

There are three levels of lesson in this case:

- First, the specification sheet is not the actual value. Tire deformation, manufacturing tolerance, and mounting error all make a difference, and calibration by measurement is required (Section 29.3 and Formula Three).
- Second, a systematic error will not be corrected by SLAM. SLAM can correct random drift (by scan matching and loop closure) but is powerless against a systematic error that magnifies every step by a fixed 8 percent—because all the data are wrong consistently and the system has no way of noticing.

- Third, check the foundations before tuning parameters. This is exactly the order of investigation stressed in Section 29.7: a problem upstream will never be solved by tuning parameters downstream.

Teachers are advised to require the odometry calibration and the recording of its figures at the very beginning of a course—that half hour will save hours of confusion afterwards.

Case 29-2 From Simulation to the Real Thing: an Unexpected Gap

Chapter 28 advised simulate first, then the real thing, but students often meet a frustration: a system that ran perfectly in Gazebo is full of trouble once moved to a real rover. Where does the gap come from?

There are four principal sources:

- Sensor noise: noise may be added to a simulated LiDAR, but a real LiDAR has all the problems discussed in Chapter 8 besides—passing through glass, weak reflection from black objects, random jumps in the measured distance. These are often not modeled fully in simulation.
- The quality of the odometry: wheel speeds in simulation are usually ideal; the real world has slip, changes in the floor material, tire deformation (Case 29-1), and gaps in the data from dropped packets.
- Time synchronization: in simulation every node shares one simulated clock and the time stamps agree perfectly; in a real system the microcontroller, the single-board computer, and the sensors each have a clock, and problems of synchronization surface (the engineering detail stressed repeatedly in Chapters 5, 10, and 27).

- The complexity of the environment: a simulated scene is usually clean and simple; a real classroom has the legs of desks and chairs (the angular resolution problem of Chapter 8), glass windows, and people walking about.

But this does not mean simulation is of no value—quite the contrary. The value of simulation is that it lets you confirm first that the logic and the architecture are right, and settle the TF setup, the topic connections, and the parameter configuration beforehand. What remains on moving to the real thing is only the physics of the real world, and the range of possible problems has narrowed greatly.

The practical advice is this: for a system that passed in simulation, still allow one or two times as long again for debugging on the real machine. This is not failure but the normal state of engineering.

Case 29-3 Why the Robot Plans a Path and Then Does Not Move

This is one of the most puzzling symptoms in navigation work: a handsome green path is plainly visible in RViz and the robot does not stir.

By the order of investigation of Section 29.7, the problem must be at the seventh or eighth checkpoint—the controller or the chassis. The common causes include:

1. The controller is publishing no velocity command: perhaps the controller node did not start, or a parameter is set so that it believes the path cannot be executed. The way to check is to watch the /cmd_vel topic directly for data.
2. The velocity command is published but the chassis does not receive it: the topic names do not match (the remapping problem of Section 28.4), or the chassis driver node is not subscribed.

3. The chassis receives it and does not move: the velocity values are too small, below the motor's starting threshold; or the motor driver is not enabled, or the power is off.
4. The velocity limits are set too conservatively: Nav2's maximum velocity parameter is set too low, or the acceleration limit too strict, so that the robot moves too slowly to look like moving at all.

The value of this case is that it demonstrates the power of systematic debugging: the same symptom guessed at randomly might take hours, while following the order of watch /cmd_vel → decide whether it is the controller or the chassis → investigate down from there usually locates it within ten minutes.

It also answers to the method of reading an open-source project in Section 28.6—trace along the flow of data. Whether reading code or debugging a system, follow the data is the most reliable strategy.

Implementation Activities

Implementation 29-1 A Complete Implementation from Mapping to Autonomous Navigation

Purpose: to complete the book's first end-to-end system implementation. Work through the six stages of Figure 29-5 in order, producing verifiable evidence at every stage. This activity can be completed in simulation; those with hardware are advised to simulate first and then move to the real machine.

Stage One: Setting Up the Coordinate System (everyone)

1. If using a ready-made platform, inspect its URDF, find the transform between base_link and laser, and check it against the actual mounting position. If the platform is your own, write your own URDF.
2. Start the system and generate a picture of the TF tree, and confirm that the chain map → odom → base_link → laser is complete and unbroken.
3. Verify the extrinsics: place the robot at a known distance from a wall (1 meter, say) and check in RViz whether the scan points fall in the right place. If they are off, go back and correct the URDF.
4. An experiment in deliberate breakage: change the LiDAR's position in the URDF wrongly by 20 centimeters on purpose and observe how the scan points go out of register in RViz—understanding the importance of the extrinsics for yourself. Change it back and continue.

Stage Two: Calibrating the Odometry (required with hardware; may be skipped in simulation)

1. Calibrate the wheel diameter by Figure 29-4: drive 3 meters in a straight line, compare the odometry with a tape measure, correct the parameter, and verify again. Record the percentage error before and after.
2. Calibrate the wheel base by Figure 29-4: turn ten times in place, compare the accumulated angle with 3,600 degrees, correct the wheel base, and verify again.
3. Check the smoothness: drive the robot by remote control and observe whether the odometry output is continuous and smooth with no jumps (the quality requirement of Section 29.3).

Stage Three: Mapping (everyone)

1. Start slam_toolbox to map, drive the robot slowly once around the site, and watch the map grow in RViz.
2. Assess the map's quality by Section 13.7: are the walls straight? Does the loop close? Measure key dimensions on the map against the real ones and compute the percentage error.
3. An experiment comparing speeds: build a map once very slowly and once rather faster, compare the quality, and verify the advice of Section 14.6 to drive a little slower.
4. Save the map, inspect the .yaml and .pgm files produced, and open the .pgm in an image viewer to observe the three-state representation of the occupancy grid.

Stage Four: Localization (everyone)

1. Start map_server to load the map just saved and confirm that it can be seen in RViz.
2. Start AMCL and display the particle cloud in RViz. Observe how the particles spread over the whole map when no initial pose is set.
3. Set the initial pose with RViz's pose estimate tool, drive the robot a few steps, and observe the particle cloud converge—Case 11-2 of Chapter 11 reproduced for yourself.
4. A kidnapping test: teleport the robot (or the simulated robot) to another position on the map and observe whether AMCL can converge again, and how long it takes.

Stage Five: Autonomous Navigation (everyone)

1. Start Nav2, display the global and local cost maps in RViz, and observe the inflation about the obstacles.
2. Send a navigation goal with RViz's goal tool, observe the path planned, and watch the robot move along it.
3. A test of dynamic avoidance: place an obstacle in the robot's way as it travels (an object may be added in simulation), and observe how the local cost map updates and how the robot goes around.
4. An experiment on the inflation radius: set the inflation radius in turn to 0.8, 1.2, and 2.5 times the robot's radius, and observe how the path changes and whether no path can be planned (the discussion of Formula Four).
5. Observing the recovery behaviors: trap the robot in a dead end or send it a goal it cannot reach, and observe which recovery behaviors Nav2 triggers.

Stage Six: An Exercise in Debugging (undergraduate)

1. Create faults deliberately: choose three symptoms from Table 29-3, create each by hand (shut down a node, change a topic name wrongly, set the inflation radius enormous), then investigate in the order of Figure 29-3 and record the time from noticing the symptom to locating the problem.
2. Build a checklist of your own: from the experience of this activity, compile a debugging checklist suited to your platform, for your own and your classmates' later use.

Discussion Questions

1. After the extrinsics were changed wrongly by 20 centimeters, how did the scan points go out of register in RViz? If a map were built without correcting it, what would go wrong with the map?

2. (With hardware) What were the errors before and after calibrating the wheel diameter? Uncorrected, how much would that error affect the size of the map? (Compare Case 29-1.)
3. How much did the quality differ between mapping slowly and quickly? Explain the difference from the principle of scan matching in Section 13.3.
4. How long did AMCL take to converge in the kidnapping test? If the map had two regions that looked alike, what would you expect to happen? (Hint: the multi-modal distribution of Chapter 11.)
5. What happened when the inflation radius was too large? And when too small? What value suits your site best?
6. (Undergraduate) In the debugging exercise, how much faster was investigating in systematic order than guessing at random? What does this suggest for your engineering habits hereafter?

Extended Challenge (university level)

First: a complete navigation system for a departmental building—build a map of one floor, set several named goal points (the laboratory door, the elevator, the tea room), and write a Python node that navigates the robot there automatically when the user types a place name. This is in fact a simplified version of the semantic navigation of Chapter 24, and a good starting point for the project of Chapter 31. Second: an experiment in fusing odometry—if the platform has an IMU, configure the EKF node of robot_localization to fuse the wheel speeds with the IMU, compare pure wheel odometry with the fused odometry in fast turns and on slippery ground, and quantify the improvement the IMU brings. This is a direct application of the theory of Chapters 7 and 11.

Further Reading

1. The official Nav2 documentation (docs.nav2.org). (The architecture, the configuration guides, and the tutorials; the principal source for this chapter.)
2. Setup Guides: Setting Up Sensors, and Mapping and Localization, in the official Nav2 documentation. (The complete steps for building a robot from nothing.)
3. The slam_toolbox open-source project (GitHub: SteveMacenski/slam_toolbox). (The default 2D SLAM of ROS 2, of moderate size and suitable for reading.)
4. S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020. (The official paper on Nav2, explaining its design philosophy.)
5. The URDF tutorials in the official ROS 2 documentation (docs.ros.org). (How to write a robot description.)
6. The SLAM and Navigation chapters of the ROBOTIS TurtleBot3 e-Manual. (A complete implementation guide for the teaching platform.)
7. The official documentation of the robot_localization package. (Supplying odometry by fusing several sensors with an EKF, the advanced material of Section 29.3.)

Summary

1. A 2D SLAM rover is made of a chassis and motors, encoders, a 2D LiDAR, a microcontroller, and a single-board computer; the microcontroller handles real-time control at the millisecond scale and the single-board computer the SLAM computation, a division of a piece with the two-chip architecture of Chapter 25.
2. URDF describes the robot's structure and where the sensors are mounted (the extrinsics); the division of labor in the TF tree is that SLAM or AMCL publishes

map→odom, the odometry system publishes odom→base_link, and robot_state_publisher publishes base_link to each sensor from the URDF. The common errors are a broken tree, wrong extrinsics, duplicate publishing, and unsynchronized time stamps.

3. Odometry is computed from the encoder readings by the differential-drive formulas and published; the wheel diameter and the wheel base must be calibrated by measurement, and a wheel base error matters more because it causes an angular error directly; odometry need not be absolutely accurate but must be smooth and continuous, and an EKF may fuse the IMU for greater robustness.
4. slam_toolbox is Nav2's default SLAM algorithm and its parameters are simple enough to suit teaching; the mapping procedure must confirm the TF tree first, drive slowly, and assess the quality throughout; the map is saved as two files, a .yaml and a .pgm.
5. AMCL localizes within a known map with a particle filter and builds no map; it supplies the map→odom transform just as SLAM does and so may be switched seamlessly; the key parameters include the number of particles, the motion noise model, the update thresholds, and the transform tolerance; an initial pose must be set, or the convergence may be slow or wrong.
6. A navigation goal in Nav2 passes through the behavior tree navigator, the planner, the controller, the cost maps, the recovery behaviors, and the velocity command; a cost map represents cost from 0 to 254, 0 free and 254 lethally occupied, built up of several layers—static, obstacle, inflation.
7. Debugging must follow a systematic order: lifecycle, TF, map, localization, cost maps, planner, controller, chassis; the tuning of the planner and the controller

should wait until everything upstream is healthy; and every checkpoint should leave verifiable evidence.

Carrying Forward

This chapter has quantified the three things most apt to hold a project up. On odometry: a wheel base 4 percent wrong together with wheels unequal by 1.5 percent puts the robot 1.13 meters off after a square 2 meters on a side, and the clockwise and counterclockwise errors differ more than threefold (1.13 against 0.32 meters)—driving one direction alone will deceive you. On transforms: a LiDAR position out by 0.20 meters or an angle out by 5 degrees each take the obstacle cells from 992 to over two thousand and the correctness down to a little over seven tenths, and the two symptoms cannot be told apart by eye. On localization: the particle spread of a global localization falls from 3.49 meters to 0.17, but the real convergence happens between steps 5 and 8—the robot must walk a way after start-up before the localization settles.

This chapter has completed the first end-to-end system from sensor to autonomous navigation. Students should feel that the theory of the twenty-seven chapters before has all flowed together in this one chapter into a robot that really moves—the LiDAR of Chapter 8, the coordinate transforms of Chapter 10, the particle filter of Chapter 11, the graph optimization of Chapter 12, the occupancy grid of Chapter 13, the SLAM systems of Chapter 15, and the tool chain of Chapter 28, not one of which could be spared.

In the next chapter we change the sensor to a camera. Setting up a RealSense or a ZED, calibrating the camera, producing an RGB-D point cloud, implementing RTAB-Map and ORB-SLAM3, and—most important of all—comparing visual SLAM and LiDAR

SLAM side by side on the same site. Chapter 30 will let students verify for themselves the complementarity of sensors discussed repeatedly in Chapters 8 and 9.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 30

RGB-D / Stereo Camera SLAM in Practice

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

This is the last technical implementation of Part VIII, and the one place in the book where two sets of sensors are set against each other directly, on the same site and along the same path.

The difficulty of the last chapter's LiDAR rover was in the wiring—the coordinate transforms, the odometry parameters, the integration of one system upon another. With a camera the difficulty changes in kind: a camera does not carry its own light as a LiDAR does, nor does it measure distance directly. Its depth is computed, and anything computed will be computed very accurately under some conditions and wildly wrong under others—and will often not tell you that it is wrong.

The three figures of this chapter are there to measure those three conditions: Figure 30-1 measures how the stereo depth error grows with distance, and answers along the way what an effective range of five meters really means; Figure 30-2 draws the consequence of depth and color not being aligned, and points out that the severity of it varies with distance in exactly the opposite way to the depth error; and Figure 30-4 is the contest this book has been building toward for twenty-odd chapters—four rooms, one covered in posters, one of white walls, one of glass, one dim, with LiDAR and vision each running a circuit.

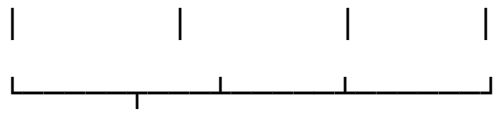
Figure 30-0 The Link Map of Chapter 30

The abilities built up so far

Chapter 9 The camera Chapter 20 Visual SLAM Chapter 21 Epipolar geometry

Chapters 22, 23 ORB-SLAM Chapter 17 Point clouds Chapter 29 LiDAR mapping in

practice



▼

【Chapter 30 Change the sensor to a camera, then set it against the LiDAR】

Choosing ─ calibration ─ point clouds ─ RTAB-Map ─ ORB-SLAM3 ─ failure cases ─ the contest

▼

Chapter 31 Integrating a project and assessing it (both systems examined together)

Learning Objectives

In the last chapter we completed mapping and navigation in full with a LiDAR. This chapter changes the sensor to a camera and implements visual SLAM—and ends by doing something this book has long been building toward: setting the LiDAR and the visual systems against each other directly on the same site. After studying this chapter, students should be able to:

1. Compare the principles and the suitable uses of the three classes of depth camera: RealSense, ZED, and OAK-D.
2. Calibrate a camera and verify that the intrinsics and the distortion coefficients are correct.
3. Produce a colored point cloud from an RGB image and a depth image, and understand the importance of alignment.
4. Run RTAB-Map to build a map and save a three-dimensional map.
5. Run ORB-SLAM3 and complete a trajectory evaluation.

6. Plan a test procedure for an indoor scene and recognize the typical failure cases of visual SLAM.
7. Compare how visual SLAM and LiDAR SLAM perform on the same site.

Core Concepts

中文名詞	English Term	Explanation
深度影像	Depth Image	An image in which every pixel records the distance of that point.
深度對齊	Depth Alignment	Aligning the depth image to the color image's frame.
彩色點雲	Colored Point Cloud	A point cloud in which every three-dimensional point carries an RGB color.
相機資訊	CameraInfo	The ROS message carrying the intrinsics and the distortion coefficients.
視覺里程計	Visual Odometry	Estimating camera motion from images, supplying odom→base_link.
RGB-D 里程計	RGBD Odometry	RTAB-Map's module estimating pose from RGB-D images.
外觀式建圖	Appearance-based Mapping	A mapping method detecting loop closures by the similarity of image appearance.
記憶體管理	Memory Management	RTAB-Map's mechanism for controlling the map's size to stay real time.
資料庫檔	Database File	The file format in which RTAB-Map stores the map and the images.
定位模式	Localization Mode	Loading an existing map to localize only, extending the map no further.
全域快門	Global Shutter	The whole frame exposed at once, which suits vision in motion.

中文名詞	English Term	Explanation
主動紅外投射	IR Projector	A device projecting a textured pattern to assist stereo matching.
近似同步	Approximate Sync	A synchronization allowing the time stamps of different topics to differ slightly.

30.1 An Introduction to the RealSense, ZED, and OAK-D Cameras

Section 9.3 introduced the three ways of obtaining depth; this section focuses on the products actually available and what to consider in choosing one.

The Three Classes of Depth Camera

- The RealSense family: active infrared stereo (Section 9.3). With improved hardware and software it reaches a measuring range of more than ten meters usable both indoors and out, and it has a global shutter and a wide field of view, which is quite useful for robots and drones. Some models carry an IMU and so support visual-inertial SLAM directly (Chapter 23).
- The ZED family: passive stereo, projecting no infrared, so that it still works outdoors in strong sunlight (which is exactly RealSense's weakness). It generally gives better depth accuracy and a wider field of view than RealSense, at the price of needing a GPU—ZED's depth computation depends on CUDA and requires an NVIDIA graphics card.
- The OAK-D family: stereo together with an AI inference chip on board. It can run a neural network such as object detection in the camera itself and offload the computation from the host—of a piece with the edge computing idea of Chapter 25, and of particular value on a platform with limited computing resources.

Four Considerations in Choosing

1. The environment: RealSense will do where the work is mainly indoors; where it must work outdoors in strong sunlight, consider ZED or passive stereo. Section 9.3 noted that an active infrared projection is drowned by sunlight.

2. Computing resources: ZED needs an NVIDIA GPU; RealSense computes the depth within the camera and demands less of the host; OAK-D can even move the AI inference onto the camera.
3. Whether an IMU is needed: for visual-inertial SLAM (Chapter 23), choosing a model with an IMU built in spares the trouble of synchronizing an external IMU and calibrating its extrinsics.
4. The effective range: take the RealSense D435—its measuring range reaches 10 meters, but the accuracy is better within 5, and in practice the effective range is often set at 5 meters. This answers to the discussion in Section 9.6 of depth error growing with the square of distance.

Class	How depth is obtained	In strong light	Computation required	Distinctive feature
RealSense	Active infrared stereo	Weaker	Low (processed in the camera)	Some models carry an IMU
ZED	Passive stereo	Usable	Needs an NVIDIA GPU	Better accuracy and field of view
OAK-D	Stereo + an AI chip	Usable	Very low (inference in the camera)	Can run a neural network on the camera

Table 30-1 A Comparison of the Three Classes of Depth Camera

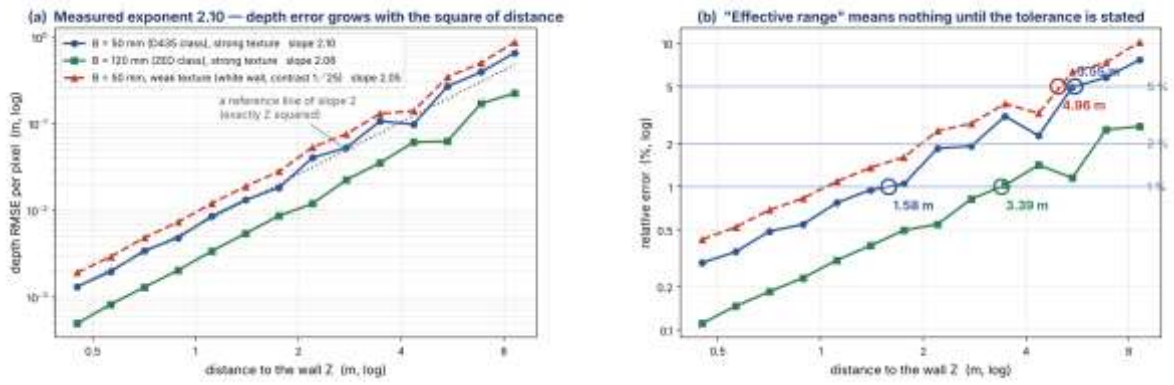
Before going on to calibration and point clouds, let us measure the most crucial sentence of this section on choosing a camera, and the one most often passed over vaguely: an effective range of five meters.

It was said above that the D435's measuring range reaches ten meters but the accuracy

is better within five. That sentence is in fact incomplete—accuracy never suddenly gets worse at some distance; it degrades continuously, so that the words effective range mean something only once it is said how many percent of error is tolerated. The figures below were actually run, not substituted into a formula.

The experiment goes like this: generate a slanted wall with a random speckle texture, compute from the stereo geometry the disparity each pixel ought to have, render the left and right images at 640 × 480, add sensor noise of 2.5 grey levels' standard deviation, hand them to OpenCV's StereoSGBM to match, then convert the disparity it computes back into depth and subtract it from the truth pixel by pixel. Three conditions: a 50 mm baseline (the D435 class), a 120 mm baseline (the ZED class), and the same 50 mm with the texture contrast reduced twenty-five-fold (simulating a white wall).

Figure 30-1 The Measured Growth of Stereo Depth Error: the Work of Distance Squared, Baseline, and Texture



Panel (a) is in logarithmic coordinates. All three lines are straight, and the slopes measure 2.10, 2.06, and 2.05—this is the measured version of the textbook sentence that depth error grows with the square of distance. Note that the square relation comes from no human setting: nowhere in the program is Z squared written, and it grows naturally out of the stereo geometry (depth inversely proportional to disparity) together with the matching error. The depth RMSE at a 50 mm baseline grows from 1.3

millimeters at 0.45 m all the way to 66 centimeters at 8.6 m, about five hundred times.

Panel (b) turns the same figures into relative error, and the words effective range at last take a form that can be answered. For a 50 mm baseline with strong texture: require 1 percent relative accuracy and the effective range is only 1.58 meters; relax it to 2 percent and it is 2.81; relax it to 5 percent and it finally passes the five meters the textbooks speak of (5.55 measured). So the D435's effective range is five meters in fact implies an error of 5 percent is acceptable—at five meters that is 25 centimeters, amply enough for avoiding obstacles and wholly unfit for three-dimensional measurement.

The second line answers the other question of the section on choosing: what a larger baseline really buys. At 1 percent accuracy the 120 mm baseline (2.4 times) has an effective range of 3.39 meters, which is 2.15 times the 50 mm. This is no coincidence: the relative error is proportional to Z / B, so at a fixed requirement of accuracy the effective distance is proportional to the baseline. To double the effective range, the most direct way is to pull the two lenses twice as far apart—which is why stereo cameras for outdoor use are all wide and large.

The third line is the first quantitative evidence for the five typical failure cases later in this chapter. The same camera at the same distance, with only the texture contrast reduced twenty-five-fold, sees the error at five meters worsen from 3.8 percent to 5.0—a third again as large; and the effective range at 1 percent falls from 1.58 meters to 1.04, which is to say to within about one meter. But what really deserves attention is not that the error grew, but what follows.

A White Wall Will Not Give You Invalid—It Will Give You a Wildly Wrong Number

In the weak-texture group, the proportion of valid pixels SGBM reported was still 99.5 percent.

Which is to say that facing a white wall the camera did not decline to answer: it filled the whole depth map just the same, and the values it filled in were wrong by 80 centimeters on average at five meters.

This is the most dangerous trait of a visual sensor: a LiDAR striking a light-absorbing black object reports an invalid point outright and you see it at a glance; a camera will answer wrongly with confidence.

The practical remedy is not to look only for holes in the depth map but at the confidence or the disparity validity mask the driver supplies, and to measure the same wall repeatedly to see whether the values are steady.

A Recommendation for Teaching

For teaching at university and high school, a RealSense model with an IMU built in is advised first: it is relatively affordable, its ROS 2 driver is mature, its community resources are rich, and it needs no separate GPU. Consider ZED later, at the project stage, when outdoor operation or greater accuracy is needed.

30.2 Calibrating the Camera

Section 9.2 explained the principle of camera calibration; this section focuses on the practicalities—the first gate of visual SLAM and the foundation of its accuracy.

Factory Parameters versus Calibrating Yourself

A modern depth camera is calibrated at the factory and supplies its intrinsics and distortion coefficients through the ROS CameraInfo message. For most applications

the factory parameters will serve directly.

But in three situations calibrating yourself is advised:

- The camera has been knocked, or the lens has been adjusted.
- An ordinary webcam or a stereo rig of your own making is used (with no factory parameters).
- A research application demands very high accuracy and you wish to confirm that the factory parameters are accurate.

A Review of the Calibration Procedure

By Zhang's checkerboard method of Section 9.2:

1. Print the checkerboard and mount it on a rigid board (it must be kept flat; any curvature affects the calibration accuracy severely).
2. Take 15 to 20 shots from different angles and distances, covering every region of the frame—and do not forget the corners above all, where radial distortion is worst.
3. Run the calibration program to obtain the intrinsic matrix K and the distortion coefficients.
4. Check the reprojection error, which should be under 1 pixel; if it is larger, add more images or shoot again.
5. Verify: undistort a scene containing straight lines and confirm that the bent lines are pulled straight.

The Calibration Tool in ROS

ROS supplies the camera_calibration package, which completes the calibration interactively: move the camera about the checkerboard and the tool indicates the

coverage live (left-right, up-down, near-far, skew), and computes and saves the parameters once the targets are met. This is far more convenient than writing a program of your own, and is the first choice in practice.

The Extra Calibration for Stereo

Besides the intrinsics of each camera, a stereo rig needs the relative pose between the two (the extrinsics)—which decides the length of the baseline and bears directly on the depth computation (the formula $Z = f \cdot b/d$ of Section 9.6). Stereo calibration can likewise be done with the camera_calibration package, but the left and right images must be captured together.

Common Traps in Calibration

- A checkerboard that is not flat: mounted on card or on a curved surface it makes the calibration wrong. A rigid flat board is essential.
- Insufficient coverage: shooting only in the center of the frame leaves the distortion coefficients at the corners poorly estimated. The checkerboard must appear in every corner of the frame.
- Inconsistent resolution: the image resolution at calibration and in actual use must be the same, or the pixel units of the intrinsics will not match.
- Forgetting to update: after calibrating, confirm that the SLAM system really reads the new parameter file and not an old one or the defaults.

30.3 Producing an RGB-D Point Cloud

With an RGB image, a depth image, and the camera intrinsics, a colored point cloud can be produced—the central form of data in visual SLAM, and one source of the point cloud processing of Chapter 17.

From a Depth Image to a Three-Dimensional Point

The principle is the inverse of the pinhole projection model of Section 9.1. Given a pixel's coordinates and that point's depth, its three-dimensional coordinates follow (the formula is Formula One of this chapter). Do this for every pixel of a depth image and a point cloud results.

Then give every point the RGB color of the corresponding pixel and it becomes a colored point cloud—a simplified version of the point cloud coloring of Section 17.6: because the RGB and the depth come from the same camera, there is no cross-sensor extrinsic or time synchronization to handle.

The Crucial Premise: Depth Alignment

Here is a detail a beginner often overlooks: the color camera and the depth sensor sit at physically different places, so that the same pixel coordinates in the two images do not correspond to the same point in space.

The remedy is depth alignment—project the depth image into the color camera's view so that the pixels of the two correspond one to one. In practice this corresponds to an alignment setting in the camera driver, and the topic produced is a depth image aligned to the color image; RTAB-Map subscribes to exactly that aligned depth topic.

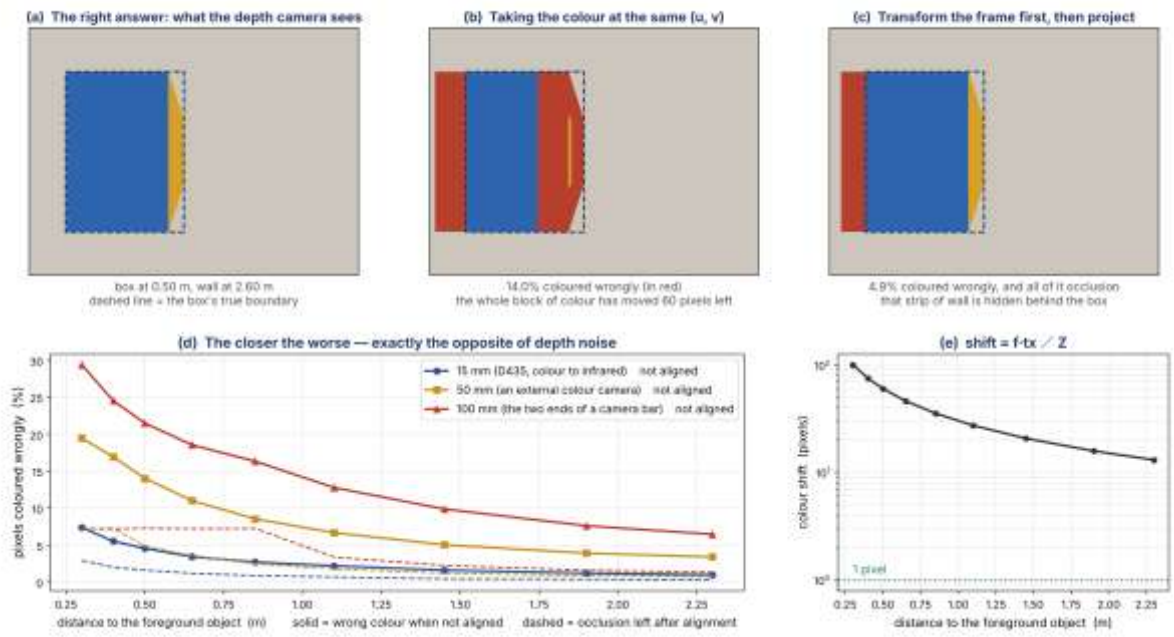
Forget to turn alignment on and the point cloud produced is out of register in color and geometry—the color of the wall ends up on the window, the same symptom as the extrinsic error mentioned in Section 17.6.

Forget to turn alignment on and the point cloud will be out of register in color and geometry sounds like a question of appearance. Measured, it proves to be more than that, and the way its severity varies with distance is exactly the opposite of the

conclusion of Figure 30-1 just now.

The scene of the experiment is simple: a wall 2.6 meters away with a box in front of it, the box's front face painted blue and its right side orange, and the wall off-white. The depth sensor and the color lens are physically offset by tx. Coloring is done two ways: the correct way back-projects the depth pixel into a three-dimensional point, transforms it into the color camera's frame, and projects it back with the color intrinsics to take the color; the wrong way takes the color at the same (u, v) in the color image directly.

Figure 30-2 What a Colored Point Cloud Comes Out Like When Depth Is Not Aligned to Color



Panel (a) is the right answer, with the dashed box marking the box's true boundary. Panel (b) is the result of taking the color at the same (u, v), with red marking the pixels colored wrongly. The whole block of color can plainly be seen shifted 60 pixels to the left: a band along the box's right side is painted the color of the wall, while the wall to the box's left is painted the box's blue. The amount of the shift has a very clean formula,

the f·tx / Z drawn in panel (e)—in this example 600 × 0.050 ÷ 0.50 = 60 pixels. Over the whole frame, 14.0 percent of the pixels are colored wrongly.

Panel (c) is the correct way of transforming the frame first and projecting after. The proportion wrongly colored falls from 14.0 percent to 4.9—but not to zero, and that remaining 4.9 percent deserves a word of its own.

Counting the occlusions separately shows that after alignment, 100 percent of the pixels still colored wrongly fall in the occluded region and the residue outside it is 0.00 percent. The reason is a geometric necessity—the strip of wall close to the box's edge is hidden behind the box in the color camera's view, and there is simply no color for it in the color image. No amount of good alignment can restore that strip; it is the price of the two lenses being in two places. This also explains why a colored point cloud in practice always has a ring of doubtful color at the edges of objects: it is not that your program is wrong.

Panel (d) is the most important of this section. The three solid lines are the proportion wrongly colored at various object distances for three lens separations (the 15 mm of the D435's color to infrared, and 50 mm and 100 mm for an external color camera), and the dashed lines are the lower bound from occlusion remaining after alignment. All three lines are worse the closer the object: the D435's is 7.4 percent wrong at 0.30 meters and only 0.99 at 2.30; the 100 mm line reaches 29.4 percent at 0.30 meters, which is to say nearly a third of the points are colored wrongly.

The Two Errors Pull in Opposite Directions, So There Is No Best Distance

Figure 30-1 says the depth error grows with the square of distance, so the farther the worse.

Figure 30-2 says the color misregistration is f·tx / Z, so the nearer the worse.

Take the D435: backing from 0.5 meters to 2.3, the depth RMSE grows about 38 times while the proportion wrongly colored falls to a quarter.

That the two run opposite ways means the intuition that holding the camera closer is more accurate is only half right: it is right for depth and wrong for color.

The real conclusion is this—decide first what this point cloud is for. For geometric measurement you cannot stand far off, and standing near means the color cannot be trusted; for semantics or recognizing materials it is the reverse. If both are wanted, the only remedy is to turn alignment properly on and cancel the one of the two curves that can be cancelled.

The Data Flow in ROS

Figure 30-3 The Data Flow from an RGB-D Camera to a Point Cloud

The camera driver node

├──→ /color/image_raw the color image

├──→ /aligned_depth_to_color/image_raw the aligned depth image

├──→ /color/camera_info the intrinsics and distortion coefficients

└──→ /imu (if the camera carries one) inertial data

↓

The point cloud node (or inside the SLAM system)

back-projects every pixel into a three-dimensional point by Formula One and gives it a color

↓

└──→ /points a colored point cloud in PointCloud2 format (Section 8.3)

The Reality of the Data Volume

A depth image of 640 × 480 produces about 300,000 points, and at 30 frames per second that is 9 million points a second—which is exactly why Section 17.2 must downsample. In practice a voxel downsampling comes first and the processing after.

Another practical consideration is clipping to the effective range: as Section 30.1 noted, a depth camera's accuracy falls away quickly at a distance, so a maximum effective distance is often set (5 meters, say) and points beyond it discarded—keeping unreliable far points would lower the overall accuracy instead.

30.4 Implementing RTAB-Map

RTAB-Map is the most used open-source system for RGB-D SLAM, and the chief subject of this chapter's implementation.

What RTAB-Map Is

RTAB-Map (Real-Time Appearance-Based Mapping) is an RGB-D graph-optimization SLAM method founded on an incremental appearance-based loop closure detector, designed to be general to indoor and outdoor environments alike.

It performs graph-optimization SLAM with the depth images the camera receives, producing a dense colored point cloud and camera odometry; it can also close loops in real time with a bag of words and a feature detector. It is a graph-optimization SLAM framework supporting RGB-D cameras, using visual appearance and depth information together for three-dimensional mapping and loop closure detection.

Against the structure of this book, students should be able to place it at once: graph optimization is Chapter 12, bag-of-words loop closure Chapters 20 and 22, and the dense point cloud Chapter 17—RTAB-Map is these technologies integrated and realized.

Three Practical Features

- It can use external odometry: it allows you to feed in your own odometry topic rather than producing it itself—so that if you have a more accurate odometry (the wheel odometry of Chapter 29, say, or the result of fusing an IMU), it can be used directly. This is of particular value on a wheeled robot.
- It can fuse a laser: you may also use your own 2D or 3D laser scans to improve the quality of the map. This means RTAB-Map can take in vision and LiDAR at once—exactly the complementarity of sensors stressed in Chapters 8 and 9, embodied in practice.

- Several map formats: it is compatible with data structures such as octrees and voxels, which are more efficient than a raw point cloud for three-dimensional planning and for meshing several maps. This answers to the discussion of the three map representations in Section 17.5.

RGB-D Odometry

The RTAB-Map package also supplies a visual odometry called RGBD odometry, which provides the pose information and the odom→base_link transform, and has shown reliable results indoors.

This matters: even where the robot has no wheel encoders (a handheld camera scan, say), RTAB-Map can produce its own odometry and work in full. This makes it a practical scheme for doing SLAM with one camera.

The Procedure

1. Start the camera driver, and be sure to turn depth alignment on (Section 30.3).
2. Confirm that the three necessary topics carry data: the color image, the aligned depth image, and the camera information.
3. Start RTAB-Map, giving the topic names above and the coordinate frames. On a first run, adding the parameter that deletes the database at start-up is advised, to avoid confusion from loading an old map.
4. Observe in RViz or in RTAB-Map's own visualization: the live point cloud, the camera trajectory, and the loop closures as they happen.
5. Move the camera slowly to scan the whole space, taking care to keep the view rich in features.
6. Save the map when done (RTAB-Map saves to a database file containing the images, the features, and the pose graph).

7. Afterwards it may be switched into localization mode, loading the existing map to localize only and extend no further—the same role as AMCL in Section 29.5.

The Key Parameters

- The synchronization: if the time stamps of the color and the depth images agree exactly, exact synchronization may be used; if they come from different sources or differ slightly, approximate synchronization must be enabled, or the system will never find a pair and will not move at all. This is a common place to get stuck.
- The maximum depth: set the upper bound of the effective range (Section 30.3) and discard the points beyond it.
- Memory management: RTAB-Map has a mechanism to control the size of the map and keep it real time—when the map grows too large the older parts are moved into long-term memory and retrieved only when needed. This lets it handle a large site.

30.5 Implementing ORB-SLAM3

Chapter 23 dissected the principles of ORB-SLAM3 in full; this section focuses on the practicalities.

How It Differs from RTAB-Map in Purpose

Aspect	RTAB-Map	ORB-SLAM3
Chief output	A dense colored point cloud map	A sparse feature point map
Integration with ROS	Natively supported, ready out of the box	Needs a community wrapper or integration of your own

Aspect	RTAB-Map	ORB-SLAM3
Tasks it suits	Three-dimensional reconstruction, navigation maps	High-accuracy localization, trajectory evaluation
Visualization	The point cloud model is plain to see	Feature points and key frames
Chapters of this book	This chapter	The principles in Chapter 23, the implementation here

Table 30-2 A Comparison of the Purposes of RTAB-Map and ORB-SLAM3

In brief: use RTAB-Map for a three-dimensional model you can look at, and ORB-SLAM3 for an accurate trajectory and localization. They are not substitutes for each other but tools for different tasks.

Practical Points

1. Compiling: the dependencies are the same as in Section 22.7 (OpenCV, Eigen3, Pangolin). Mind the version compatibility, and referring to a community-maintained fork or using a container is advised (the advice of Section 28.1).
2. The configuration file: prepare a YAML file carrying the intrinsics and distortion coefficients obtained in Section 30.2, the image size, and the ORB feature parameters (features per frame, pyramid levels).
3. Choosing the mode: choose monocular, stereo, or RGB-D by what the camera can do, and whether to enable the inertial mode (which needs an IMU in the camera).
4. IMU initialization: if the inertial mode is used, be sure to make a motion with acceleration, deceleration, and rotation after starting, or the initialization will not succeed (the excitation of motion stressed repeatedly in Section 23.4).

5. Observing: watch the feature points, key frames, covisibility graph, and loop closures in the Pangolin window—the data structures of Section 22.6 made concrete.
6. Outputting and evaluating the trajectory: output the estimated trajectory and compute ATE and RPE with the evo tool (Section 22.7). In monocular mode, remember to align the scale first.

Comparing the Three Modes by Measurement

Students may test the three modes in turn on the same site and verify the theoretical comparison of Table 20-3 in Section 20.6 for themselves:

- Monocular: hardest to initialize (it needs parallax enough, Section 21.5), the trajectory has no absolute scale, but the hardware is simplest.
- RGB-D: easiest to initialize (there is depth from the first frame), it has absolute scale, and the map is densest, but it is limited by the depth camera's effective range and by strong light.
- Stereo-inertial: the most robust and the most accurate (the millimeter-level results mentioned in Section 23.7), but it needs a camera that supports it and rather more configuration.

30.6 Testing in an Indoor Scene

This section explains how to design a test of teaching value, and particularly the experiment of deliberately seeking the boundary of failure.

The Basic Test Procedure

1. Prepare the environment: begin in a space rich in texture and evenly lit (a classroom with bookshelves and posters, say), to build an experience of success.
2. Scan slowly: handheld or mounted on a rover, move slowly and steadily and cover the whole space.
3. Test the loop closure: go once around back to the start and observe whether the loop closes and whether the map is pulled straight (the striking picture of Chapters 12 and 15 reproduced here).
4. Assess the quality: measure key dimensions on the map against the real ones; check whether the walls have thickened (the three-dimensional quality index of Case 17-1 in Chapter 17).

Five Typical Failure Cases

This is the most valuable part of the section—creating failure deliberately so as to understand the system's boundaries.

1. A white wall and weak texture: scan a large expanse of white wall and observe how the feature count falls to zero and the system loses tracking. This is the curse of the white wall mentioned repeatedly in Chapters 9 and 20, verified here for yourself.
2. Fast motion: the commonest cause of failure in visual SLAM. Moving or turning too fast breaks the sequence of successful point cloud matches and the system loses tracking. Students may raise the speed by degrees and find the critical point at which it begins to fail.

3. Changing light: walk from a bright place into a dim one, or face a window and overexpose, and observe whether the tracking breaks. This is the dependence on light of Section 9.5 made concrete.
4. Glass and mirrors: scan a floor-to-ceiling window or a mirror and observe what goes wrong with the depth data. This is the same class of difficulty as the glass problem of the LiDAR in Chapter 8.
5. Moving objects: have someone walk through the frame and observe whether the localization is affected. Section 24.4 noted that object detection may reject the features of moving objects; here we first observe the consequence of not doing so.

An Important Practical Reminder

On recording data there is one experience worth noting: against recording data and processing them offline, live mapping has an advantage—when RTAB-Map loses tracking, the odometry can be recovered by moving the camera back to where tracking last succeeded.

This live correction with a person in the loop is not available when processing recorded data offline. In teaching it is therefore advised to build the experience and the feel of success live first, and to understand how the camera must be moved for the system not to lose tracking; and to record data later, when algorithms must be compared, for the sake of fairness (the value of rosbag in Section 28.4).

The Feel of Scanning

Visual SLAM has one characteristic LiDAR SLAM does not: the operator's technique affects the result markedly. A few practical suggestions:

- Slow down in the turns: features change gently in a straight line, while a turn changes the view quickly and loses tracking most easily.
- Keep features in view: do not swing the camera suddenly onto a blank wall but move along the direction that has texture.
- Avoid rotating without translating: pure rotation makes triangulation impossible (Section 21.5, Case 21-1), which is fatal in monocular mode above all.
- Look back now and then: glancing back at the region already travelled helps trigger a loop closure and consolidate the map.

30.7 Comparing Visual SLAM and LiDAR SLAM

This is the section the book has long been building toward: Chapters 8 and 9 compared the sensors' properties, Chapter 29 implemented LiDAR SLAM, this chapter implemented visual SLAM—and now the two meet directly on the same site.

Designing a Fair Comparison

A fair comparison requires the variables to be controlled:

- The same site and the same route: best of all is one robot carrying both LiDAR and camera, recording both kinds of data in one pass.
- The same measures of evaluation: trajectory error (ATE), map quality (the 2D indices of Section 13.7 and the wall thickness of Chapter 17), and robustness (the number of times tracking is lost).
- Several environments: this is crucial—a comparison in one environment misleads, and the scenes where each is strong and weak must all be covered.

The Expected Performance in Four Environments

Environment	LiDAR SLAM	Visual SLAM	Why
A bright, textured classroom	Good	Good	Both are in their comfort zone
A dim machine room	Good	Fails	The LiDAR carries its own light; the camera needs ambient light
A corridor of large white walls	Fair	Fails	The LiDAR sees geometry; the camera finds no features
A hall walled in glass	Fails	Fair	The laser passes through glass; the camera sees the reflections and the frames

Table 30-3 The Expected Performance of the Two SLAMs in Four Environments

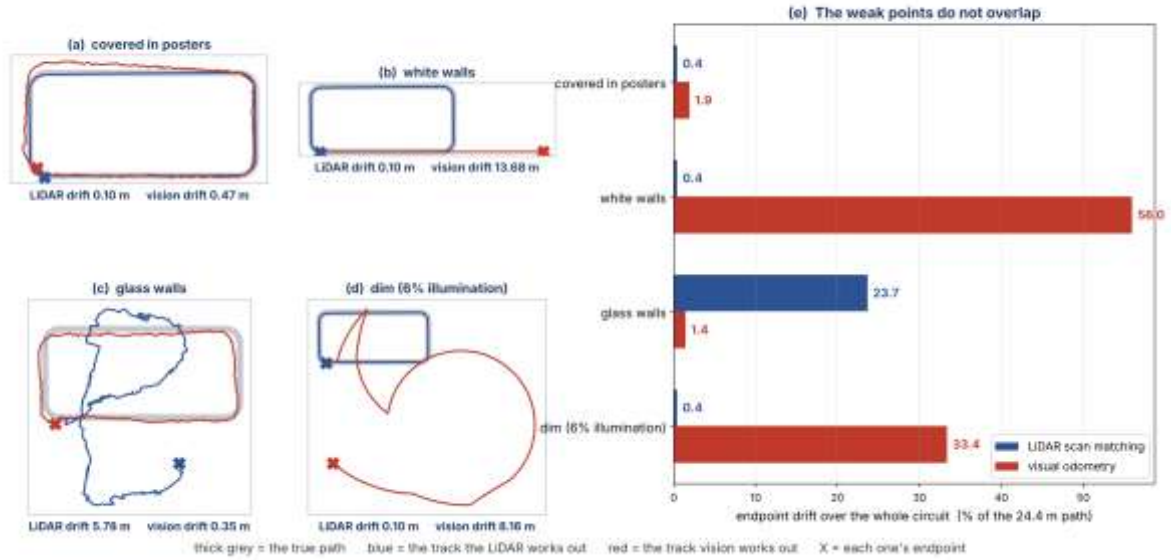
Note the last row particularly—it is the one case where vision beats LiDAR, and it shows best what complementary really means: the weaknesses of the two do not overlap, so that together they cover more situations.

That table is the expected performance. This section runs it for real.

The design of the experiment is deliberately clean: four identical rooms of 12 × 8 meters, with the robot going once around the same rectangular path along the walls (24.4 meters in all) and obtaining a LiDAR scan and a camera observation at every step. The rooms differ only in the character of the walls: covered in posters (texture 1.00, illumination 1.00), white walls (texture 0.04), glass (85 percent of the laser beams pass straight through, and half of those that do not are reflected specularly and report a

wrong distance), and dim (texture 1.00 but illumination only 6 percent). The LiDAR matches scans by point-to-line ICP, while vision solves the relative pose from the bearings and ranges of the landmarks two frames see in common.

Figure 30-4 The Contest on One Site: Four Environments, Two Sensors, One Path



Take the LiDAR first. In three of the four rooms the endpoint drift is exactly the same, 0.10 meters. This is not three independent experiments happening to agree—it is that the LiDAR simulation never reads the texture and illumination parameters at all, so the three runs are the same thing at the level of bits. Put plainly, that is precisely the point: when it is said that LiDAR performs well in the dark, the exact statement is not that it overcomes the darkness but that darkness does not exist for it. The laser carries its own light, and whether the lamps are on has nothing to do with it.

The only room where the LiDAR collapses is the glass one: a drift of 5.78 meters, 23.7 percent of the whole path, with the endpoint flown clean out of the room. The blue line in panel (c) breaking off and shooting upward halfway is the typical look of scan matching that has lost its reference. The usable LiDAR points per frame fall from 360 to 52, a loss of eighty-six percent—and those that remain are mixed with false distances

reflected back. This cell is well worth stopping over in class: it shows that the LiDAR's failure is not a fall in accuracy but suddenly knowing nothing at all.

Now vision. The drift is 13.68 meters in the white-walled room and 8.16 in the dim one, and of 341 steps, 341 and 334 respectively were judged lost—which is to say that nearly the whole circuit was pushed through on a constant-velocity model. The mean usable observations in the two rooms were only 1.7 and 2.6 landmarks, below even the minimum needed to solve a three-degree-of-freedom pose. Note that the white wall and the dim room are in the camera's eyes the same thing: contrast is texture times illumination, and texture 0.04 at full illumination and texture 1.00 at 6 percent illumination give contrasts of 0.04 and 0.06, so no wonder the results are so alike. The observation is useful in practice: putting posters on a white wall and turning the lights up are two forms of the same remedy.

The glass room is the reverse: vision drifts only 0.35 meters, sixteen times better than the LiDAR. The reason is written plainly in the simulation too: glass is transparent to a laser and not to a camera—window frames, smudges, reflections, and stickers are all features that can be tracked. This cell is the one place in the whole book where vision beats LiDAR by a wide margin, and the most concrete evidence of what the word complementary means.

The Most Accurate at Every Step Need Not Be the Most Accurate Over a Circuit

The room covered in posters holds a counterintuitive result worth noting: the LiDAR's per-step relative pose RMSE is 0.42 centimeters against vision's 1.86, so the LiDAR is 4.4 times the more accurate; but over a whole circuit the LiDAR

drifts 0.59 meters and vision only 0.14, so vision wins instead.

The reason is that the errors are of different natures. Scan matching leans a little the same way at every step (because the beams of two neighboring frames strike different points on a surface), and such a systematic bias accumulates all the way; vision's error is larger but more random, and over a circuit a good deal of it cancels.

This matters for marking a project: a per-frame error is not enough to judge an odometry by, and a loop must be closed to see how far from the start it returns. But the loop closure gap has its own blind spot—a scale bias in the odometry scales the whole map up or down in proportion while the loop still closes perfectly, so the loop closure gap must be read together with a dimensional error measured by tape (Section 31.8 has the measurements on this). The marking standard of Chapter 31 is set by exactly this principle of two complementary measures.

A Comparison on Other Dimensions

Dimension of comparison	LiDAR SLAM	Visual SLAM
Cost	Higher	Lower
Information in the map	Accurate geometry, no semantics	Carries color and semantic cues
Computation required	Lower in 2D, higher in 3D	Higher (image processing)
Depth accuracy	Centimeter level and steady	Decays with the square of distance

Dimension of comparison	LiDAR SLAM	Visual SLAM
Sensitivity to technique	Less sensitive	Sensitive (the feel of Section 30.6)
What follows from it	Navigation, obstacle avoidance	Semantic understanding, AR, three-dimensional reconstruction

Table 30-4 The Two SLAMs Compared on Other Dimensions

Fusion: the Most Practical Answer

The conclusion of the comparison is not which is better but how to combine them. Section 30.4 noted that RTAB-Map supports using vision and laser data together, which is exactly the route to implementing a fusion.

- The LiDAR supplies accurate geometry and steady odometry and holds the system up where it is dim and poor in texture.
- The camera supplies color, semantics, and the ability to recognize a place, and comes into its own in a glass environment and where the scene must be understood.
- The IMU (Chapters 7 and 23) holds things up when both fail briefly.

This is exactly the central belief stressed since Part II: there is no all-powerful sensor, only a fitting combination. Having come through twenty-nine chapters, students should be able to verify that sentence with their own hands—and not merely accept it.

But the contest is not over—the rules of that match in fact favored one side in secret. The two sections that follow make the rules fair first, and then consider where

this field is going next.

Taking the Gift Back: What If the Features Are Matched Wrongly

The contest above in fact gave vision an advantage: the program paired landmarks by their index numbers, which amounts to assuming that feature matching never goes wrong. Real ORB descriptor matching takes ten to twenty percent wrong matches as a matter of course where texture repeats, where the viewpoint changes greatly, or where moving objects appear. Take that gift back and the result becomes the following.

Table 30-5 The Drift Over a Circuit of Visual Odometry Once a Rate of False Matches Is Added (meters)

Environment	0% false matches	10% false matches	20% false matches	35% false matches	The LiDAR in the same room
Covered in posters	0.14	0.52	1.22	2.50	0.59
White walls	5.90	5.90	5.90	5.90	0.59
Glass walls	0.36	1.10	3.10	5.50	16.46
Dim	7.51	7.51	7.98	17.32	0.59

The first row is the one to mind. In the ideal world of no false matches, the room covered in posters is a great win for vision (0.14 against 0.59); but add only 10 percent of false matches and vision falls to 0.52 meters, nearly level with the LiDAR's 0.59; at 20 percent it loses outright. In other words, the earlier statement that vision is more accurate in a textured environment holds only if the feature matching is clean enough—which is exactly why ORB-SLAM spends so much effort on RANSAC, on bag-of-words verification, and on checking the consistency of the covisibility graph

(Chapter 22).

The second and fourth rows scarcely change with the rate of false matches, for a very practical reason: in the white-walled and the dim rooms there are only one or two usable landmarks on average, so there is not even the chance to match wrongly—the system was lost long before. The rate of false matches is a problem only where there are features enough; where there are not, the problem lies further upstream.

The third row shows a reassuring fact: even at 20 percent false matches, vision in the glass room (3.10 meters) still far outdoes the LiDAR (16.46). The complementarity does not rest on an assumption that vision is perfect but on the fact that the causes of the two failures have nothing to do with each other.

30.8 2024–2026: Visual SLAM Is Being Rewritten

The RTAB-Map and ORB-SLAM3 taught in this chapter are at present the most reliable choices on a production line and in a classroom, and that will not change soon. But at the same time the main line of research in visual SLAM has moved to two new places, and both are near enough to affect the choice of equipment. This section sets a few concrete numbers together so that students may know where they stand.

One: the Map Has Changed Its Appearance—from Point Clouds to 3D Gaussians

Traditional dense visual SLAM stores the map as a point cloud or as voxels. Since 2024, 3D Gaussian Splatting has begun to displace them: the map becomes millions of ellipsoids with color and opacity, which can be rasterized in real time into a photorealistic picture.

SplaTAM is the representative work that first brought this into the SLAM loop. On the

Replica dataset its camera trajectory error (ATE RMSE) averages 0.36 centimeters, beating Point-SLAM's 0.52, ESLAM's 0.63, and NICE-SLAM's 1.06, and its rendering reaches 400 frames per second at 876 × 584. But on TUM-RGBD, captured with real sensors, its 5.48 centimeters loses to ORB-SLAM2's 1.98—and the paper itself notes that on this benchmark feature-based methods still beat dense ones. The contrast says much for this chapter's choice to teach ORB-SLAM3: first place on the leaderboard is not first place on another dataset.

Progress in speed has been faster still. A Gaussian-plus-SDF SLAM published late in 2025 runs at more than 150 frames per second on real Azure Kinect sequences, where Gaussian SLAM of the same period was mostly still under 20; it uses about half as many Gaussians and about 75 percent fewer optimization iterations.

Two: Running Without Calibration—Vision Foundation Models Enter SLAM

Section 30.2 of this chapter spent some length on camera calibration, because everything downstream is wrong if the intrinsics are wrong. MASt3R-SLAM, at CVPR 2025, challenges that premise directly: it replaces traditional feature matching and triangulation with a pretrained three-dimensional reconstruction foundation model, and can do dense SLAM even where the camera intrinsics are unknown.

The numbers are these: about 15 frames per second on an RTX 4090. With the intrinsics known, the ATE RMSE averages 0.030 meters on TUM-RGBD and 0.047 on 7-Scenes, both slightly ahead of DROID-SLAM's 0.038 and 0.049; with the intrinsics wholly unknown they fall back to 0.060 and 0.066, still comparable with many methods that require calibration. The price is clear too: on an RTX 3080 Ti it falls to 5 to 7 frames per second, it wants 9 to 11 GB of display memory, and on a sequence of fast motion such as EuRoC (0.041) it still loses to DROID-SLAM (0.022).

The other half of the same line is monocular depth estimation. Depth Anything V2 and 2025's Video-Depth-Anything let an ordinary RGB camera output depth that is consistent over time. Take the metric version of the latter: the large model reaches δ1 of 0.908 on NYUv2 and 0.910 on KITTI; the smallest model has only 28.4 million parameters, with a half-precision inference latency of 7.5 milliseconds on an A100. But note that its real-time streaming mode still has a plain price: δ1 on ScanNet falls from 0.926 offline to 0.836.

> **What This Means for a Student Project**
>
> Do not rush to replace RTAB-Map. What the methods above have in common is that their figures in the papers are handsome but they all want a high-end graphics card, and most of them have neither a mature ROS 2 wrapper nor a commitment to long-term maintenance.
>
> What can be used now is monocular depth estimation: it can serve as a fallback where the depth camera fails (a white wall, or beyond the effective range), and it lets a team with no depth camera get the pipeline running first.
>
> What should go into the project report is the comparison: measure the ATE once on your own site by the methods of this chapter, set it beside the figures in the papers, and explain why they differ. The value of doing that is far greater than getting the newest model to run.

Three: the Hardware End Is Changing Too

The advice on choosing a camera in Section 30.1 rests on an assumption: that RealSense is cheap, its ROS 2 driver mature, and its community resources rich. That assumption changed in July 2025—RealSense became a company independent of Intel, taking 50 million dollars of investment (from Intel Capital and the MediaTek Innovation

Fund), with Intel retaining a minority stake and one board seat.

For the classroom this is good news in the short term: the new company has 130 employees and more than three thousand customers, and it names visual SLAM for autonomous mobile robots explicitly as a direction of investment; its new D555 supports power over Ethernet and carries a 5 TOPS AI chip of its own design, which is exactly the road of edge computing in Chapter 25. But its long-term commitment to maintenance and the steadiness of supply need time to prove, so in planning course or project equipment over several years it is advised not to bind the whole pipeline to one manufacturer's SDK—let the nodes depend as far as possible on the standard ROS 2 image and CameraInfo interfaces, so that changing camera changes the driver and not the whole system.

Formulas and Procedures

One: Back-Projecting a Depth Image into Three-Dimensional Points

This is the inverse of the pinhole projection model of Section 9.1. Let a pixel's coordinates be (u, v), that pixel's depth value d, and the camera intrinsics fx, fy, cx, cy; then the corresponding three-dimensional point is:

$$Z = d \,/\, scale$$

$$X = (u - cx) \times Z \,/\, fx$$

$$Y = (v - cy) \times Z \,/\, fy$$

where scale is the depth scaling factor—a depth image is often stored as integers in millimeters while a point cloud is in meters, so one must divide by 1000. Getting this factor wrong is a common beginner's error, and the symptom is that the scale of the point cloud is entirely wrong (a whole room one millimeter across, say).

Apply this to every pixel of the depth image, give each the corresponding RGB color, and a colored point cloud results (Section 30.3).

Two: Depth Quality and the Effective Range

Section 9.6 derived that a stereo depth error grows with the square of distance:

$$\Delta Z \approx (Z^2 / (f \cdot b)) \times \Delta d$$

This explains the practice mentioned in Section 30.1—that although the camera is rated at a ten-meter range, the effective range is often set at five. By this formula: double the distance and the error is four times. With a 1-centimeter error at 2 meters, the error at 10 meters may reach 25 centimeters, which is no longer enough for accurate mapping.

Practical advice: use this formula to work back from your accuracy requirement to the greatest usable distance, and set that as the depth clipping threshold in the system.

Three: the Complete Procedure for Implementing Visual SLAM

Figure 30-5 The Complete Implementation Procedure for RGB-D Visual SLAM

【Stage one: preparing the camera】

① Install the camera driver → ② Confirm the intrinsics (from the factory or calibrated yourself, Section 30.2)

③ Turn depth alignment on → ④ Verify that the color and depth pixels correspond correctly

↓

【Stage two: verifying the data】

⑤ Check the three necessary topics: the color image, the aligned depth, the camera information

⑥ Display the point cloud in RViz and confirm that both the geometry and the color are right

↓

【Stage three: running SLAM】

⑦ Start RTAB-Map or ORB-SLAM3, minding the synchronization setting

⑧ Scan slowly, watching the trajectory, the map, and the loop closures

↓

【Stage four: assessing and comparing】

⑨ Save the map → ⑩ Measure the errors (dimensional comparison, wall thickness, ATE)

⑪ Compare with LiDAR SLAM on the same site (Section 30.7)

Four: the Order of Troubleshooting Visual SLAM

As with the navigation debugging of Section 29.7, visual SLAM should also be investigated in a systematic order:

Figure 30-6 The Debugging Checklist for Visual SLAM

① The camera driver: do all three necessary topics carry data? Are the rates normal?

↓

② The intrinsics: are the values in CameraInfo reasonable? (fx and fy should be close; cx and cy about half the image width and height)

↓

③ Depth alignment: are color and depth aligned? Is the point cloud's color out of register?

↓

④ The depth scale: is the point cloud's scale right? Verify by measuring a known object (the scale of Formula One)

↓

⑤ Synchronization: if the system does not move at all, check whether approximate synchronization must be enabled

↓

⑥ Features: how many feature points are detected in the current frame? Too few means the environment is poor in texture

↓

⑦ Motion: is the movement too fast? Is it rotating without translating? (the feel of Section 30.6)

↓

⑧ Parameters: begin tuning the SLAM parameters only once everything

upstream is healthy

Cases

Case 30-1 Move a Little Faster and It Loses Tracking: the Commonest Frustration of Visual SLAM

This situation very nearly always arises when students implement visual SLAM, and deserves a full account.

The typical scene is this: a student moves a depth camera slowly about the laboratory, the system works well, and the point cloud builds up bit by bit. But move or turn even a little faster and the coordinate axes on screen begin to wander, the point cloud stops recording, and the terminal shows the tracking quality at zero.

The phenomenon is discussed again and again in the community. One user describes it thus: move or rotate the camera very slowly and the map builds but the size is wrong; speed up a little and the camera's TF begins to wander at random and the point cloud stops recording. The official documentation warns plainly too: moving or turning too fast breaks the sequence of successful point cloud matches and the system loses tracking.

The cause is the triple blow discussed in Section 26.4, reappearing here: fast motion causes motion blur so that feature extraction fails, shortens the feature tracks so that matching is hard, and the depth camera's frame rate limit makes the difference between successive frames too great for the registration to converge (the ICP of Section 17.3 needs a good initial value).

There are remedies at three levels:

- At the level of operation: slow down, in the turns above all (the advice on feel in Section 30.6). This is the most direct and effective move of all.
- At the level of configuration: one solution the community offers is to use the camera's two infrared images separately and run SLAM in the infrared configuration rather than the depth one—which amounts to switching to stereo mode and gives steadier feature tracking.
- At the level of the system: add IMU fusion (Chapter 23). The IMU supplies a motion prediction while vision briefly fails and raises the robustness under fast motion greatly—exactly the value of Section 7.6's the IMU fills the gap when vision fails.

The teaching value of this case is that it lets students feel for themselves that the accuracy figures in a paper were obtained under particular conditions. The same system reaches centimeter accuracy on a slow scan and fails outright when moved fast—understanding that gap is a precondition of using a technology responsibly.

Case 30-2 A Three-Meter Room Becomes Three Millimeters: the Trap of the Depth Scale

This is an error that looks absurd and is extremely common, and very nearly every student writing their own point cloud program for the first time meets it.

The symptom is: the point cloud is produced and the shape looks right, but the whole scene is absurdly small—the view must be zoomed in very close in RViz to see it at all, and the measured dimensions are wholly unreasonable.

The cause is the depth scaling factor of Formula One. To save space a depth image

is usually stored as 16-bit integers in millimeters, while a ROS point cloud is in meters. Forget to divide by 1000 in the program and the whole scene shrinks a thousandfold—a three-meter room becomes three millimeters.

Diagnosing this error is extremely simple: measure an object of known size in the point cloud (a sheet of A4 paper or the width of a desk, say), and if the value is out by three orders of magnitude, the scaling factor is the trouble.

The broader lesson is that a units error is one of the commonest and most easily avoided errors in engineering. Section 10.3 mentioned mixing radians and degrees, this chapter millimeters and meters, Chapter 5 degrees of latitude and meters—form the habit of labeling every value with its unit and countless hours of debugging are saved. The habit serves as well in writing programs as in writing reports and in reading a specification sheet.

Case 30-3 The Contest on One Site: When LiDAR and Camera Each Fail

The comparative experiment proposed in Section 30.7 has very high teaching value in a university class. A typical measured result is described here.

The design of the experiment: one rover carrying both a 2D LiDAR and an RGB-D camera travels the same route through four areas—a bright classroom, a dim machine room, a corridor lined with white acoustic panels, and an atrium walled in glass.

The result is quite dramatic:

- The classroom: both perform well, the map clear and the trajectory accurate. Students may feel at this point that the two are much alike.

- The dim machine room: the visual SLAM loses tracking within seconds of entering and the feature count falls to zero; the LiDAR is wholly unaffected, because it carries its own laser light (Chapter 8).
- The white-walled corridor: the visual SLAM fails again (the curse of the white wall); the LiDAR faces the structural degeneracy of a corridor (Section 13.6), but at least keeps running.
- The glass atrium: the situation reverses—the LiDAR's laser passes through the glass and draws on the map a space that does not exist; the camera, on the other hand, can see the glass's reflections, the window frames, and the scenery beyond, and builds a more reasonable map.

This experiment turns the theory of Chapters 8 and 9 from a comparison on a table into a failure seen with one's own eyes. Students generally report that after doing it their understanding of choosing a sensor is far deeper than from reading the table ten times over.

More important is the discussion that follows: since the weaknesses of the two do not overlap, what would combining them do? This leads naturally to the fused implementation in which RTAB-Map uses vision and laser together, and answers to the multi-sensor and multi-robot collaboration of Chapter 27. Teachers are urged to arrange this experiment; it is the most persuasive lesson in Part VIII.

Implementation Activities

Implementation 30-1 Implementing Visual SLAM and Comparing It Across Sensors

Purpose: to complete a full implementation of RGB-D visual SLAM and compare it on

the same site with the LiDAR SLAM of Chapter 29. This is the climax experiment of Part VIII and an important foundation for the project of Chapter 31.

Part One: Setting Up the Camera and Producing a Point Cloud (everyone)

1. Install the depth camera driver, confirm that all three topics—color image, depth image, and camera information—carry data, and record the publishing rate of each.
2. Inspect the intrinsic values in CameraInfo and check them for plausibility by the second step of Figure 30-3 (are fx and fy close? are cx and cy about half the image width and height?).
3. Turn depth alignment on, display the colored point cloud in RViz, and check that the color and the geometry correspond correctly. Turn alignment off deliberately and look again, observing the misregistration.
4. Verify the scale: measure an object of known size in the point cloud (a sheet of A4 paper, the width of a desk) and confirm the value—avoiding the trap of Case 30-2 for yourself.
5. (Advanced) Write it yourself: by Formula One, produce a point cloud from the depth image and the intrinsics without using a ready-made node. Set the scale wrongly on purpose and reproduce the phenomenon of Case 30-2.

Part Two: Mapping with RTAB-Map (everyone)

1. Start RTAB-Map with the right topic names. On a first run, add the parameter that deletes the old database.
2. Move the camera slowly about the classroom to scan, and watch the colored point cloud and the camera trajectory build live.

3. Test the loop closure: go once around back to the start and observe whether loop closure detection fires and whether the map is pulled straight.
4. An experiment on speed: raise the speed of movement by degrees and find the critical speed at which the system begins to lose tracking, reproducing Case 30-1. Record that speed.
5. Save the map, restart in localization mode, and verify that the system can localize within the existing map (compare AMCL in Section 29.5).

Part Three: Exploring the Failure Cases (everyone; the distinctive part of this chapter)

Test each of the five cases of Section 30.6 in turn and record the system's behavior:

1. The white wall test: point the camera at a large expanse of white wall, record how the feature count changes, and how long the system holds before losing tracking.
2. The light test: walk from a bright place into a dim one and record when the tracking breaks; then face a window to overexpose and observe the result.
3. The glass test: scan a glass window or a mirror and observe what goes wrong with the depth data (missing? wrong depth values?).
4. The moving object test: have a classmate walk through the frame and observe whether the localization accuracy and the map quality suffer.
5. The pure rotation test: rotate the camera in place (without translating) and observe the system's response, verifying the triangulation limit of Section 21.5.

Part Four: Comparing Across Sensors (the core experiment; undergraduate)

1. Choosing the site: pick four areas of different character—bright and textured, dim, a large white wall, and glass (see Table 30-3).

2. Collecting the data: carry both the LiDAR and the camera on one rover, move along the same route, and record both kinds of data at once (the rosbag of Section 28.4).
3. Running each: on the same data, run the LiDAR SLAM of Chapter 29 and the visual SLAM of this chapter separately.
4. Comparing quantitatively: record for each of the four areas the number of times tracking was lost, the map quality, and (if there is ground truth) the trajectory error, and enter them in a comparison table of your own.
5. Trying a fusion: configure RTAB-Map to use vision and laser data together (Section 30.4) and observe whether it covers the areas where each alone failed.

Discussion Questions

1. With depth alignment turned off, how was the point cloud's color misregistered? What has the direction of the shift to do with the camera's physical structure?
2. At what speed of movement did your system begin to lose tracking? What does that speed mean for mounting it on a moving robot?
3. Of the five failure cases, which happens most easily? Which has the most serious consequence? Why?
4. (The core question) In the comparison across the four areas, in which did the LiDAR win and in which vision? Does this agree with the expectation of Table 30-3? Was anything unexpected?
5. After fusing vision and laser, how did the system perform? Did it really cover the weaknesses of each? What new costs did the fusion bring (computation, complexity of configuration)?

Extended Challenge (university level)

First: a three-dimensional digital twin of a university site—scan a laboratory or a stretch of corridor with RTAB-Map, produce a complete colored three-dimensional model, and export it as an octree or a mesh by the methods of Section 17.5. The result can serve for departmental display and for space management, and is the direct product of Project Two in Chapter 31. Second: a quantitative study of fusing vision and LiDAR—design a rigorous experiment comparing the three configurations of LiDAR alone, vision alone, and the two fused across several environments, quantified by ATE and by the number of times tracking is lost, and write it up as a complete research report. The topic is complete in its levels and concrete in its result, an excellent main thread for a project, and it answers to Part VIII's insistence on letting the data speak.

Further Reading

1. M. Labbé and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation," Journal of Field Robotics, 2019. (The authoritative paper on RTAB-Map, with an account of fusing vision and laser.)
2. The official RTAB-Map site and the ROS 2 package documentation (introlab.github.io/rtabmap). (Parameter documentation, tutorials, and configuration examples for various cameras.)
3. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021. (The system of Chapter 23, and one subject of this chapter's implementation.)
4. The Intel RealSense ROS 2 package documentation and the SLAM tutorial wiki (GitHub: IntelRealSense/realsense-ros). (The official guidance on camera configuration, depth alignment, and IMU fusion.)

5. The ROS 2 camera_calibration package documentation. (The interactive tool for camera calibration, the basis of the implementation in Section 30.2.)
6. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR Workshops, 2017. (The principles and specifications of the depth cameras, a shared reference for Chapters 9 and 30.)
7. The evo trajectory evaluation tool (GitHub: MichaelGrupp/evo). (Computing ATE and RPE; the quantitative tool for comparing across sensors.)

Summary

1. The three classes of depth camera each have their place: RealSense is active infrared stereo, performs well indoors, and some models carry an IMU; ZED is passive stereo, usable outdoors and more accurate but needing a GPU; OAK-D has an AI chip on board and can infer in the camera. The choice should weigh the environment, the computing resources, whether an IMU is needed, and the effective range.
2. A modern depth camera is calibrated at the factory and supplies its parameters through CameraInfo, but one should calibrate oneself after a knock, with a rig of one's own, or where high accuracy is required; the common traps are a checkerboard that is not flat, insufficient coverage, inconsistent resolution, and forgetting to update the parameters.
3. Back-projecting a depth image into a point cloud is the inverse of the pinhole model, and its crucial premise is depth alignment—the color and the depth sensors sit at different places, and without alignment the color and the geometry are out of register; a wrong depth scaling factor puts the scale of the scene out by three orders of magnitude.

4. RTAB-Map is an RGB-D graph-optimization SLAM founded on incremental appearance-based loop closure detection, producing a dense colored point cloud and RGB-D odometry; it can use external odometry, fuse 2D or 3D laser scans, and support structures such as octrees and voxels, and it is the route to implementing a fusion of vision and LiDAR.
5. RTAB-Map and ORB-SLAM3 are differently aimed: the first outputs a dense colored map suited to three-dimensional reconstruction and navigation, the second a sparse feature map suited to high-accuracy localization and trajectory evaluation; the difficulty and the accuracy of the three modes (monocular, RGB-D, stereo-inertial) can be verified by measurement.
6. The five typical failures of visual SLAM are a white wall and weak texture, fast motion, changing light, glass and mirrors, and moving objects; moving or turning too fast breaks the sequence of point cloud matches and loses tracking, and the advantage of live mapping over offline processing is that tracking can be recovered by moving the camera back by hand to where it last succeeded.
7. The weaknesses of LiDAR and vision do not overlap: the LiDAR wins where it is dim and where the walls are white, vision wins at glass, and that is what complementary truly means; fusion (RTAB-Map using vision and laser together, say) is the most practical answer, and bears out this book's central belief that there is no all-powerful sensor, only a fitting combination.
8. To evaluate an odometry or a SLAM, one measure is not enough: a per-frame error cannot show accumulated drift, and a loop closure gap cannot show a scale bias in the odometry—when the whole map is scaled up or down in proportion, the loop still closes perfectly. The loop closure gap must be read

together with the dimensional error measured by tape, so that their blind spots do not overlap (Section 31.8 has the measured data on this).

The technical implementations of Part VIII close here. Students have built two complete SLAM systems with their own hands and measured on one site where each of them fails, what the failure looks like, and why those weaknesses do not appear together. This is deeper than reading any comparison table—because once you have seen with your own eyes a LiDAR fling its trajectory out of a glass room, and a camera lose tracking for a whole circuit before a white wall, there is no all-powerful sensor, only a fitting combination is no longer a slogan.

In the next chapter we integrate every ability into complete projects: from 2D mapping of a classroom at high school, through three-dimensional scanning of a space at university, path analysis for a robot vacuum, campus inspection by a robot dog, and a vision-navigated drone, to multi-sensor fused localization; and we set out a complete marking standard covering map quality, localization error, system stability, the presentation of the report, and the quality of the code. Chapter 31 is the final examination of this book's practical work.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 31

SLAM Projects and Their Assessment

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

This is the last chapter of Part VIII and the final examination of the book's practical work.

Over thirty chapters we have come all the way from what SLAM is to I have made it run with my own hands: Parts I to III built the ideas, the sensors, and the mathematical foundations; Parts IV to VII dealt in turn with 2D mapping, 3D LiDAR, visual SLAM, and constrained platforms; Chapter 28 got the development environment ready; Chapter 29 made a LiDAR rover really move; and Chapter 30 set a camera and a LiDAR against each other on one site.

This chapter deals with a different question: how these abilities are to be made into a piece of work, and how that work is to be judged good or bad. The two look like course administration and are in fact the hardest things to teach in an engineering training, and the most easily skipped—most students' projects fail not on the technology but on a subject too large, a schedule with no slack, and a report at the end with no data in it.

So this chapter has three figures computed from experiments, each answering a question often passed over vaguely: Figure 31-1 answers how much the efficiency of different strategies differs, using the coverage planning of Project Three; Figure 31-2 answers how much slack a schedule should keep, using a Monte Carlo simulation of four milestones; and Figure 31-3 answers whether the method of evaluation you are using is itself reliable—and its conclusion may have not a few students reconsidering how they evaluate.

Figure 31-0 The Link Map of Chapter 31

The abilities built up over thirty chapters

Chapters 13–16 2D Chapters 17–18 3D Chapters 19–24 Vision Chapters 28–30 Practice

| | | |

└──────┴──────┴──────┘

▼

【Chapter 31 Making the abilities into a piece of work, and saying clearly how good it is】

Six projects (31.1–31.6) ─ the marking rubric (31.7)

─ the reliability of the evaluation method (31.8) ─ schedule and risk (Case 31-3)

▼

Part IX Chapters 32–34: semantics, training in simulation, language navigation

Learning Objectives

Having come through thirty chapters of theory and practice, students now have the full range of SLAM abilities. This chapter is the final examination of the book's practical work: six projects of rising difficulty, a marking standard in five dimensions, and three quantitative experiments explaining the reasons behind how to do it and how it is judged. It is both the capstone of the course and a map for students planning a research direction of their own. After studying this chapter, students should be able to:

1. Choose a project of suitable difficulty for their own abilities and resources and plan how to carry it out.
2. Complete a classroom 2D mapping project and assess the map's quality quantitatively.
3. Complete a three-dimensional scanning project and produce a digital model that can be shown.

4. Design and run a path analysis and coverage assessment for a robot vacuum, and explain by experiment the difference in efficiency among strategies.
5. Plan the system architecture for campus inspection by a robot dog and for a vision-navigated drone.
6. Design the experiment and the method of comparison for multi-sensor fused localization.
7. Check their own work against the five-dimensional marking standard, and prepare the project report and presentation.
8. Judge whether a method of evaluation suffices to support a conclusion, and explain what the loop closure gap, the dimensional error, and the ATE can and cannot each reveal.
9. Plan milestones and slack in probabilistic terms, and estimate the chance of a project finishing on time.

Core Concepts

中文名詞	English Term	Explanation
專題導向學習	Project-Based Learning	A way of learning centered on completing a real task.
評分標準	Rubric	A tool describing the basis of marking by explicit dimensions and grades.
絕對軌跡誤差	ATE	The position error after aligning the estimated trajectory with the truth (Section 22.7).
相對位姿誤差	RPE	The error of the relative motion over a

		fixed interval, reflecting local drift.
閉環落差	Loop Closure Gap	The distance on the map between start and end after going once around.
覆蓋率	Coverage Rate	The proportion of the traversable region actually travelled (Chapter 16).
重複率	Redundancy Rate	The ratio of the distance actually travelled to the theoretical minimum.
地圖品質指標	Map Quality Metric	Items of assessment such as wall thickness, dimensional error, and loop consistency.
系統穩定性	Robustness	The ability to run long without losing tracking and to recover from failure.
可重現性	Reproducibility	The degree to which another person can reproduce your result from your records.
成果證據	Evidence	Verifiable products: map files, trajectory figures, recordings, code.
技術債	Technical Debt	Problems of code quality accumulated for the sake of speed, to be repaid later.
專題里程碑	Milestone	Dividing a long project into staged aims that can be checked.
緩衝	Schedule Buffer	Time kept deliberately to absorb the uncertainty of a schedule.

Table 31-1 The Central Terms of This Chapter

An Overview of the Projects: How to Choose

This chapter offers six projects of rising difficulty. They are not independent alternatives but a path that may be walked in order—the later ones often build on the results of the earlier.

Project	Subject	Difficulty	Whom it suits	Chapters it chiefly draws on
One	Mapping a classroom in 2D	★	High school, introductory undergraduate	Chapters 13–15, 28
Two	Three-dimensional scanning of a space	★★	Undergraduate	Chapters 17, 30
Three	Path analysis for a robot vacuum	★★	Advanced high school, undergraduate	Chapter 16
Four	Campus inspection with a robot dog	★★★★	Project students	Chapters 18, 24, 27
Five	A vision-navigated drone	★★★★	Project students	Chapters 23, 26
Six	Multi-sensor fused localization	★★★★★	Project students, graduate students	Chapters 5–7, 11, 29

Table 31-2 The Difficulty of the Six Projects and Whom Each Suits

Three Criteria for Choosing a Project

- Resources: what hardware is there? how much time? With only a computer, Projects One and Three can be completed wholly in simulation; Projects Four and Five need a real platform.
- Ability: against the five-stage path of learning in Section 28.7, judge honestly

which stage you are at. Taking on a project one stage higher brings the most growth; skipping two stages easily brings frustration.

- Interest and continuity: a project is best if it can grow into a longer research direction. If AI interests you, Projects Four and Five lead naturally into the semantic SLAM of Chapter 32 and the language navigation of Chapter 34.

Principles Common to All

- Divide it into milestones: break the project into three to five stages that can be checked, each with something to show. This avoids finding out at the very end that the direction was wrong.
- Leave evidence: Section 29.7 stressed that every stage should produce verifiable evidence—a map file, error data, a recording, a code repository. It is the basis of debugging and the material of the report alike.
- Simulate before the real thing: the path advised repeatedly in Chapters 28 and 29. Getting the procedure and the parameters working in simulation saves a great deal of debugging on hardware.
- Record the failures: a failed attempt and the reason for it is often worth more than a successful result. A project report that honestly presents what I tried, why it did not work, and how it was finally solved carries far more conviction than one reporting only successes.

31.1 Project One: Mapping a Classroom in 2D

This is the most basic project and the most important. It covers the whole cycle of SLAM and its result is plain to see—which makes it a suitable first project for everyone.

The Aim of the Project

Build an occupancy grid map of a classroom (or a stretch of corridor) with a 2D LiDAR and assess its quality quantitatively. The advanced aim is to complete autonomous navigation on the map afterwards.

Three Stages

- Stage one—mapping in simulation: complete the mapping procedure in a simulated environment and grow familiar with the tool chain and the parameters (Chapters 28 and 29). Product: a simulated map file and a record of the operations.
- Stage two—mapping for real: map a real classroom with a rover. Complete the odometry calibration first without fail (Section 29.3). Product: the map file, a recording of the mapping, and a report assessing the quality.
- Stage three—autonomous navigation: load the map, localize with AMCL, and navigate to given goals with Nav2 (Sections 28.5 and 29.6). Product: a recording of the navigation and statistics on the rate of success.

The Quantitative Assessment Required

- Dimensional error: measure the length and width of the classroom on the map against a tape measure and compute the percentage error. The target is under 3 percent.
- Loop consistency: go once around back to the start and measure the gap between start and end on the map.
- Visual inspection: by Section 13.7, check whether the walls are straight and whether anything is doubled or broken.
- Navigation success rate (if stage three is done): set five goal points, attempt

each three times, and count the proportion reached and the mean time taken.

These four must be done together and not one alone. The reason will be shown by experiment in Section 31.8: the dimensional error and the loop closure gap each see only one kind of error, and measuring only one of them stands a good chance of marking a map that is 3 percent wrong as perfect.

Advanced Challenges

- Comparing algorithms: on one recorded dataset (a rosbag), run slam_toolbox, Cartographer, and Gmapping in turn and compare the map quality and the computational load (Implementation 15-1 in Chapter 15).
- A difficult site: take on a long corridor or an empty hall, reproduce the difficulty of Section 13.6, and try to propose a remedy (add features, adjust parameters, fuse an IMU).
- A study of parameters: vary the number of particles, the resolution, the update thresholds and so on systematically, quantify their effect on quality and performance, and draw it up in figures.

What Is Assessed

What is assessed in this project is not whether a map was built but whether you can say clearly why the map is good or bad. Students should be able to point to the source of the error (odometry? scan matching? the structure of the environment?) and give grounds for an improvement.

> **Three Common Mistakes at the Start**
>
> One: mapping straight away without calibrating the odometry. Section 29.3 measured it: a 2 percent wheel diameter error is enough after one circuit to

distort the map visibly, and the distortion looks like a poor algorithm, which easily sends students tuning parameters in the wrong direction.

Two: pushing too fast. Scan matching needs sufficient overlap between neighboring frames; push the cart faster than the sensor's update rate can bear and the matching fails and jumps. Walk through once slowly first, then speed up by degrees to find your own system's limit.

Three: drawing a conclusion from one pass. Walk the same site at least three times and see whether the result is steady. A large difference among the three means the system is at its boundary, and that fact is itself worth writing into the report.

31.2 Project Two: a 3D Point Cloud Scan of the Intelligent Robotics Laboratory

This project raises the dimension to three, and its result is visually striking enough to serve for departmental display and recruitment material.

The Aim of the Project

Scan a laboratory or a stretch of corridor with an RGB-D camera or a 3D LiDAR, produce a complete colored three-dimensional point cloud model, and assess its accuracy.

Three Stages

- Stage one—preparing and calibrating the equipment: camera calibration, verification of depth alignment, confirmation of the point cloud's scale (Sections 29.2 and 30.3). Product: the calibration parameter file and a record of the

verification.

- Stage two—scanning and modeling: scan the target space with RTAB-Map. Plan the route to ensure complete coverage and sufficient overlap (Section 17.5). Product: the three-dimensional model file and a recording of the scanning.
- Stage three—post-processing and output: downsample, remove outliers, convert to an octree or a mesh, and produce a model that can be shared (Sections 17.2 and 17.5).

The Quantitative Assessment Required

- Wall thickness: the central index of Case 17-1 in Chapter 17. Measure the spread of a stretch of wall point cloud in the perpendicular direction; ideally it should approach the sensor's accuracy (a few centimeters).
- Dimensional error: measure key dimensions in the model (a door's width, a room's length) against the real ones.
- Completeness of coverage: check for plain holes and explain how they arose (occlusion? glass? an insufficient scanning route?).
- The scale of the data: record the raw point count, the count after downsampling, and the processing time at each stage.

Advanced Challenges

- Merging several scans: scan different regions separately and merge them, experiencing the multi-session working of Atlas in Chapter 23 or the map fusion of Chapter 27.
- A comparison across sensors: scan the same space with an RGB-D camera and with a 3D LiDAR and compare the accuracy, the density, and the completeness of the models (the three-dimensional version of Section 30.7).

- An extension into application: import the model into a simulation environment and build a digital twin of the classroom, for the students who come after to test algorithms in (Section 17.7). This road is also the gateway to the Sim-to-Real of Chapter 33—the classroom you have scanned can become a training ground later.

The Long-Term Value of the Result

The product of this project has real uses: managing departmental space, planning accessible routes, recruitment displays, and serving as the basis of a simulation environment. Students are advised to put the result into a shareable format with documentation—far more valuable than a file only its author can read.

31.3 Project Three: Path Analysis for a Robot Vacuum

This project needs no expensive hardware and yet goes deep into the central questions of coverage path planning, which makes it an ideal introduction for high school and the early undergraduate.

The Aim of the Project

Analyze and improve a coverage path planning algorithm, assess it quantitatively by coverage and by the rate of repetition, and (if possible) compare it with the behavior of a real robot vacuum.

Three Stages

- Stage one—analysis on paper and in simulation: do the paper activity of Section 16.7, then implement boustrophedon coverage planning in Python and compute the coverage and the rate of repetition. Product: the code and the

quantitative data.

- Stage two—comparing strategies: implement and compare at least three strategies—random bounce, boustrophedon, and spiral—and compare their efficiency in rooms of different shapes. Product: comparison figures and an analysis.
- Stage three—observing the real thing (optional): observe and record the route of a real robot vacuum and the map in its app, compare with your simulation, and infer what strategy it uses. Product: the observation record and a comparative analysis.

One: What Kind of Figure This Project Should Produce—a Demonstration

What follows is the result this book actually ran for this project, which may serve as a template for the product of students' stage two—note what it looks like: an explicit experimental setting, a control, two complementary measures, and a conclusion that can be stated in one sentence.

The experimental setting: a room of 8 by 6 meters divided into cells of 0.25 meters; obstacles are 3 × 3-cell blocks of furniture placed at random, covering 0, 6, 12, and 20 percent of the area in turn; each setting is run 20 times with random arrangements. Three strategies: random bounce (drive straight, turn at random only on striking something, with a further 5 percent chance of a spontaneous turn at each step to simulate slip), boustrophedon (sweep up and down row by row, going around an obstacle by the shortest path to the next uncovered cell), and spiral (inward from the outside). The horizontal axis is always converted into distance travelled ÷ the theoretical minimum coverage distance, so that the three strategies may be compared fairly.

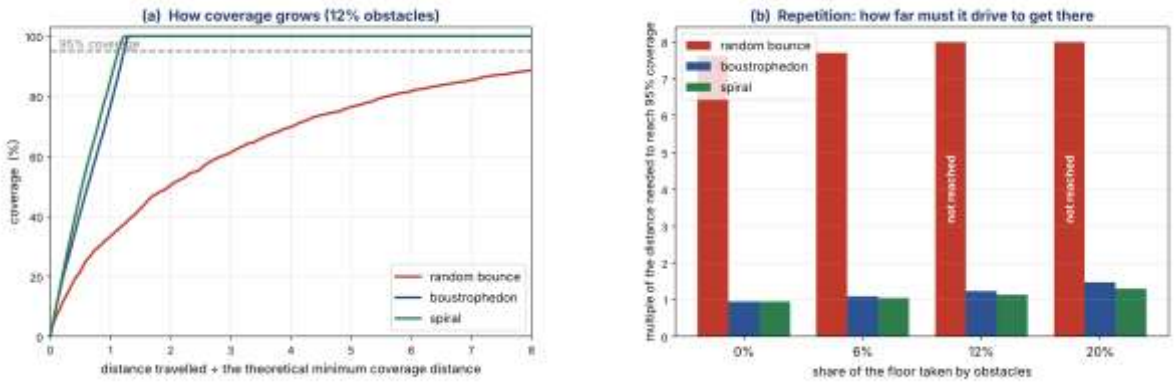


Figure 31-1 The Difference in Efficiency Among Three Coverage Strategies

Panel (a) is the growth of coverage against distance travelled at 12 percent obstacles; panel (b) is the multiple of distance needed to reach 95 percent coverage at four obstacle densities. Random bounce fails to reach the target even at eight times the distance in the 12 and 20 percent settings, and is marked not reached.

Two: Three Conclusions That Can Be Written Straight into a Report

1. A systematic strategy is more than five times as efficient as a random one. At one times the theoretical minimum distance, the spiral has covered 85.5 percent and the boustrophedon 76.7, where random bounce has only 33.3. To reach 95 percent coverage, the boustrophedon needs only 0.95 times the distance in an empty room against random bounce's 7.6—and in a room with 12 percent obstacles or more, random bounce does not get there even at eight times.

2. The real trouble with a random strategy is not that it is slow but that it cannot finish. Look at the red line in panel (a): it climbs not badly at first, but the closer to 100 percent the flatter it goes—those last few cells are forever a matter of luck. This is the mathematical reason an early robot vacuum would clean for ages and always leave a patch undone, and why every manufacturer afterwards changed to mapping and planning.

3. Obstacles make a systematic strategy dearer but do not make it fail. The

boustrophedon's distance to target rises from 0.95 times (no obstacles) to 1.47 (20 percent obstacles) and the spiral's from 0.95 to 1.30—the excess being the cost of going around. In other words, obstacles raise the rate of repetition by about half while the coverage is still guaranteed to be reached; a random strategy loses the guarantee outright.

The third point deserves a further word, because it bears on the theory of Chapter 16: the boustrophedon and the spiral guarantee the target because the path they travel is computed in advance to cover every traversable cell; going around an obstacle only adds cost and does not change what is covered. A random strategy has no such guarantee, and its coverage is a probabilistic quantity—which is why its curve has an asymptote it can never quite reach at 100 percent.

Why the Spiral Slightly Beat the Boustrophedon in This Experiment

The spiral advances along the boundary of the region already covered on every circuit, so in a rectangular room it turns fewer times and detours less, and its distance to target is slightly lower (1.16 times against 1.23 at 12 percent obstacles).

But that advantage depends greatly on the shape of the room. In a long narrow corridor or a layout of several connected rooms, the spiral crosses the region already covered again and again, and the advantage vanishes or even reverses.

This is exactly the place for students to verify for themselves: run it again with another room shape and see whether the conclusion turns over. A conclusion that turns over with the setting is still a good conclusion so long as you state the conditions; it is the conclusion whose conditions are left unstated that is

wrong.

Three: the Quantitative Assessment Required

- Coverage: the proportion of traversable cells actually travelled, with the target near 100 percent.
- The rate of repetition: the total distance travelled divided by the theoretical minimum coverage distance (Formula One of Chapter 16). Using the multiple of distance needed to reach 95 percent coverage is advised instead, because the last 5 percent often takes more than half the time of a random strategy and distorts the mean.
- The number of turns: turning takes time and power, and is an important measure in practice.
- Adaptability to obstacles: test in rooms with different numbers and arrangements of obstacles and observe the robustness of the strategy.

Four: Advanced Challenges

- Room segmentation: implement the distance transform method of Section 16.4 to divide the map into rooms automatically, cover them one by one, and compare the efficiency against covering the whole at once.
- Resuming after a break: implement recording the region already completed on an interruption and covering only the remainder on restart, and verify that the coverage still reaches 100 percent.
- Integrating with the real thing: connect your coverage planner to the map built in Chapter 29 and to Nav2, and have a real rover carry out the coverage cleaning.
- Adding localization error: run this section's experiment again but give the

robot's position a SLAM error (a random offset of 2 centimeters per meter, say). This makes the record of what has been covered unreliable in itself—which is the hardest part of a real robot vacuum, and the reason the first thirty chapters of this book exist.

Five: Its Value in Teaching

What is distinctive about this project is that everyone has an intuition about it and finds on trying that it is not simple. Students may suppose that it is just sweeping row by row, and then meet an irregular room, obstacles, and several connected rooms, and find how complex the problem is. That gap between intuition and rigor is a fine training in algorithmic thinking.

31.4 Project Four: Campus Inspection SLAM with a Robot Dog

This is an integrative project of some difficulty, covering the technology of Chapters 18, 24, and 27, and suited to a project student over one or two semesters.

The Aim of the Project

Have a quadruped or wheeled robot carry out an autonomous inspection on campus, building a three-dimensional map, recognizing the inspection targets, and producing an inspection report.

Five Stages

- Stage one—collecting data: carry a LiDAR and an IMU on a handcart or a robot and record a dataset in corridors, stairwells, and laboratories (Section 18.7). This step needs no algorithmic ability and is a good place to begin.

- Stage two—comparing maps offline: run LeGO-LOAM, LIO-SAM, and Fast-LIO on the recorded data and compare the trajectory accuracy, the map quality, and the computational load (Chapter 18).
- Stage three—studying the degenerate scene: concentrate on the long corridor and compare LiDAR alone with LiDAR tightly coupled to an IMU, quantifying what the IMU contributes. This is the stretch of greatest research value.
- Stage four—semantic perception: train a YOLO model to recognize the inspection targets (fire equipment, doors, anomalies), taking care to collect data from the robot's viewpoint (the low-viewpoint problem of Section 24.4).
- Stage five—autonomous inspection and reporting: integrate localization, navigation, and recognition, set the inspection points to be visited automatically, and produce an inspection report with three-dimensional positions marked.

A particular note on the value of stage one: it is the only stage of the whole project that needs no algorithmic ability at all and is certain to produce something. A well-recorded, clearly labeled campus dataset is itself an asset that can be passed on to the students who come after, and the surest insurance of the whole project—even if every later stage gets stuck, you still have something to write into the report.

The Quantitative Assessment Required

- Trajectory accuracy: assessed by loop consistency or by an approximate ground truth from AprilTags (Section 24.3). Read Section 31.8 alongside this: loop consistency alone is not enough.
- Map quality: wall thickness, whether the floor structure is right, whether the floors are aligned with one another.
- Recognition accuracy: the precision of the YOLO model on a test set from the

robot's viewpoint, set against its performance on a dataset from a human viewpoint.

- Task completion rate: set ten inspection points and count the proportion reached autonomously with the data successfully collected.

Advanced Challenges

- Multi-robot collaboration: have two or more robots inspect separately and fuse their maps (Chapter 27), and compare the efficiency against one machine.
- Detecting change over time: repeat the same inspection route weeks apart and compare automatically to find what has changed in the environment (the multi-session working of Atlas in Chapter 23).
- A language task interface: let the user specify the inspection task in natural language (Section 24.5), leading directly into the language navigation of Chapter 34.

A Reminder on Safety and Ethics

Campus inspection involves public space and people, and before carrying it out one must: obtain permission to use the site, avoid the peak flow of people, mark the robot plainly, have an emergency stop, and observe the rules of privacy for image data (Section 27.7).

The handling of image data deserves particular care: an inspection recording will inevitably capture teachers and students. Blurring faces before the data enter a training set is advised, with an account of your handling in the report—write that passage well and a reviewer will certainly notice.

These are not formalities but the basic professional conduct of an engineer.

31.5 Project Five: a Vision-Navigated Drone

This project combines the visual-inertial navigation of Chapters 23 and 26; it is difficult and its result is extremely attractive.

The Aim of the Project

Have a drone localize and fly autonomously in a GPS-denied indoor environment and complete a given route or exploration task.

Four Stages

- Stage one—analyzing a dataset: run ORB-SLAM3 on EuRoC or TUM-VI first and grow familiar with the system's behavior and the evaluation methods (Section 26.6). No hardware is needed.
- Stage two—a steady hover: test optical flow hovering with a micro drone inside a net and measure the extent of the drift (the safety principles of Section 26.5).
- Stage three—integrating VIO: feed the pose VIO outputs into the flight controller and achieve position holding and simple route flying on visual localization.
- Stage four—an autonomous task: fly a given route, avoid simple obstacles, or explore and map.

The Quantitative Assessment Required

- Hover accuracy: the extent of the horizontal drift over a 60-second hover in place.
- Trajectory error: compute the ATE against ground truth from AprilTags or motion capture.

- Initialization time: the time and the amount of motion needed from start-up to the completion of IMU initialization (Section 23.4).
- The boundary of failure: raise the flight speed by degrees and record the critical speed at which the system begins to be unsteady (Section 26.4).

Advanced Challenges

- Adapting to image quality: implement a mechanism adjusting the visual and inertial weights dynamically by the sharpness of the image (the fifth remedy of Section 26.4).
- Ground-air collaboration: explore together with a ground robot and compare what each can cover (Sections 25.7 and 27.1).
- A dataset of your own: record and publish a visual-inertial dataset of your own campus for the students who come after.

Safety Comes First

The safety principles of Section 26.5 must be observed strictly here: a net or a cage, a micro platform by preference, an emergency stop always to hand, testing in stages, and managing where people stand.

Any test of autonomous flight should first be verified on a tether or in a state where manual control may be taken over at any moment. Skipping this step is the principal cause of accidents.

The success of a project is not worth anyone's safety. Make not flying today a decision that may be taken at any time, and agree beforehand on the conditions under which there will certainly be no flight (high wind, a crowd, low battery, someone without eye protection).

31.6 Project Six: Fused Localization with Geomagnetism, IMU, and Vision

This is the most difficult project and the one that best displays the ability to integrate, covering the sensors of Chapters 5 to 7 and the fusion theory of Chapter 11.

The Aim of the Project

Design and implement a multi-sensor fused localization system, quantify what each sensor contributes, and verify the value of the fusion across several environments.

Four Stages

- Stage one—single-sensor baselines: localize with geomagnetism, with the IMU, and with vision (or LiDAR) separately, and measure the error characteristics and the failure cases of each. This step establishes the basis for comparison.
- Stage two—fusing in pairs: implement IMU with geomagnetism (the AHRS of Section 6.6) and IMU with vision (the VIO of Chapter 23), and quantify the improvement the fusion brings.
- Stage three—the full fusion: integrate all three with an EKF or graph optimization (Chapters 11 and 12), and add a mechanism adjusting the weights by the situation (the switching of trust in Section 5.6).
- Stage four—verification across environments: test in a bright classroom, a dim machine room, beside a steel structure, and outdoors, and verify whether the fused system beats every single sensor in every environment.

The Quantitative Assessment Required

- The trajectory error of each configuration: a table comparing the ATE of each

single sensor and each fused combination in each environment.

- The number of failures: how many times each configuration lost tracking or diverged in each environment.
- The variation of the weights: record how the weight of each sensor varies dynamically in each environment in the fused system, and verify that the mechanism works as intended.
- The computational cost: the computational load of each configuration, assessing the price of the fusion.

The Research Value of This Project

What is most valuable in this project is that it can produce a measured map of which sensor to trust in which environment. This is not merely a class exercise—it is the concrete answer, for your own site, to the principle of fusion discussed repeatedly in Sections 4.7, 5.6, 7.6, and 30.7.

Writing the result up as a complete research report is advised: the experimental design, the data, the figures, the analysis, and the conclusion. A report of that kind is already at the level of a submission to a student conference or material for a graduate application.

The Connection to Part IX

A student who completes this project has every foundation needed to enter Part IX. The semantic SLAM of Chapter 32, the training in simulation and Sim-to-Real of Chapter 33, and the VLM language navigation of Chapter 34 all lay understanding and decision on top of this solid geometric localization—and if the geometric layer is not solid, those layers above it are castles in the air.

31.7 The Marking Rubric: Five Dimensions

This section offers a complete marking standard. It is both the basis of a teacher's assessment and a checklist for students to check themselves against—reading it at the start of a project rather than at the end is advised.

Dimension	Suggested weight	What is assessed
Map quality	25%	Dimensional error, clarity of the walls, loop consistency, completeness
Localization error	25%	Trajectory accuracy, repeatability, the number of failures
System stability	20%	Running long, recovering from failure, tolerance of a changing environment
The report	20%	Definition of the problem, account of the method, presentation of the data, depth of analysis
Code quality	10%	Readability, modularity, documentation, reproducibility

Table 31-3 The Five Dimensions of Project Marking

One: Map Quality (25%)

- Excellent: dimensional error under 2 percent, wall lines clear with nothing doubled, the loop perfectly closed, coverage complete with no unexplained holes, and the source of the residual error explained.
- Good: dimensional error under 5 percent, wall lines broadly clear, the loop closed, with only a few explicable holes.
- Pass: the map allows the structure of the environment to be recognized, but with plain distortion or doubling, and the problem can be pointed out.

- Needs improvement: the map is severely distorted or incomplete, and the reason cannot be explained.

Two: Localization Error (25%)

- Excellent: quantitative data against ground truth (ATE / RPE), the error within a reasonable range, with an analysis of how it varies with time and with environment.
- Good: a quantitative assessment (a loop closure gap, a dimensional comparison), with reasonable figures.
- Pass: a qualitative assessment (the trajectory looks reasonable), but no quantities.
- Needs improvement: no assessment of localization at all.

Three: System Stability (20%)

- Excellent: runs steadily for a long time, recovers automatically after losing tracking, and the boundary of failure has been tested deliberately (the five cases of Section 30.6, say).
- Good: completes the whole task, failing occasionally but recoverable by hand.
- Pass: completes the task under ideal conditions.
- Needs improvement: works only in fragments and cannot complete the whole procedure.

Four: the Report (20%)

- Excellent: defines the problem clearly, explains the reasons for the choices of method, presents the data in figures, analyzes the results deeply, and discusses honestly the limitations and the attempts that failed.
- Good: complete in structure, clear in its data, with a basic analysis.

- Pass: explains what was done, but without analysis or reflection.
- Needs improvement: only a record of operations, from which the depth of understanding cannot be seen.

Five: Code Quality (10%)

- Excellent: clearly structured and modular, with comments and documentation, well version-controlled, and another person can reproduce the results from the documentation.
- Good: the program runs and has basic comments and documentation.
- Pass: the program runs but is hard to read or lacks documentation.
- Needs improvement: the program does not run or cannot be obtained.

Six: With Particular Emphasis—Report Failures Honestly

One principle in assessment is often overlooked and extremely important: reporting a failed attempt honestly should earn marks, not lose them.

A report saying we tried method A, which failed for such and such a reason; on analysis we changed to method B and verified the improvement shows a far deeper understanding than one saying only we used method B and got a good result. The first displays engineering judgment; the second may be merely copying a tutorial.

That this book has discussed common failures from its first chapter is in exactly this spirit. A teacher should convey this standard explicitly in assessment and encourage students to face difficulty honestly.

Seven: Advice on Presenting

- The core should take three minutes: say first what problem was solved, how, and with what result, and leave the details for later.

- Use a video rather than a live demonstration: a live demonstration is risky (hardware failure, a different environment), while a recording made beforehand is more reliable and can be cut to the essentials.
- Data need a comparison: an error of 5 centimeters means nothing; an error of 5 centimeters, against 30 before calibration carries conviction.
- Prepare an if I did it again: being able to say what you would improve with more time displays the ability to keep thinking.

31.8 Is the Method of Evaluation Itself Reliable

The first two dimensions of Section 31.7 carry half the marks between them, and both rest on an assumption: that the figures a student reports really reflect how good the system is. This section examines that assumption.

This is not pedantry. In real project reviews the commonest situation is not that students did no assessment but that they did one that looks scientific and proves nothing. The two experiments below each expose one such trap.

One: the First Trap—a Loop Closure Gap of Zero Does Not Mean the Map Is Right

The loop closure gap—going once around back to the start and seeing how far apart start and end are on the map—is the measure students like best, because it needs no ground truth equipment and a tape measure will do. The trouble is that it is sensitive only to one class of error.

The experimental setting: simulate two kinds of odometry error on an elliptical loop of 12 by 8 meters. Family A is a scale bias (the wheel diameter measured wrongly, so the

distance travelled is consistently 0.5 to 6 percent too large or too small); family B is a heading random walk (the angle measured at every step carries a random error that accumulates). Each family is run 150 times, and three numbers are computed for each: the loop closure gap, the true ATE (compared point by point with the true trajectory), and the dimensional error of the map.

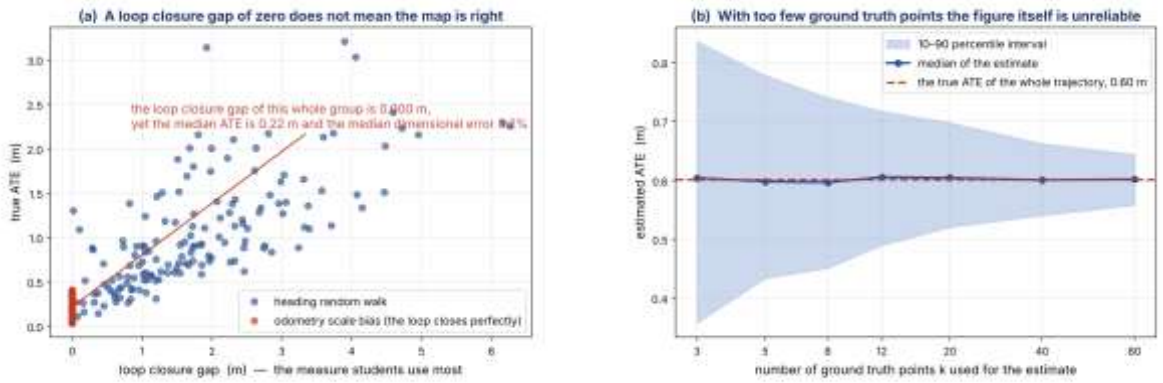


Figure 31-2 What Each of Two Methods of Evaluation Can See

In panel (a) each point is one trial. The red points are the scale bias family and the blue the heading random walk family. Panel (b) is one trajectory with the number k of ground truth points used to estimate the ATE varied, each repeated over 800 samplings.

The red points of panel (a) are all crowded at zero on the horizontal axis: the loop closure gap of the scale bias family is numerically 0.000 meters—exactly zero, down to the last bit of the floating-point arithmetic. The reason is simple—scale the whole map up or down in proportion and the loop still closes. But the true ATE of these maps has a median of 0.22 meters, and the dimensional error a median of 3.2 percent and a maximum of 6.0. By the standard of Section 31.7 a map 6 percent wrong does not reach even good, and yet its loop consistency is full marks.

The blue points show another aspect: the correlation between the loop closure gap and the ATE is 0.70, which is indeed a useful measure. Taking the two families together the

correlation of 0.84 looks handsomer still, but that number is false—it comes wholly from the blue group, while the vertical row of red points contributes almost nothing to it and is precisely the error most easily missed.

The Conclusion: Use Both Measures, Because They Are Complementary

What the loop closure gap can see: heading error, the accumulated drift of scan matching, a failure to detect the loop. What it cannot see: a scale bias.

What the dimensional error (compared by tape) can see: a scale bias. What it cannot see: local distortion and doubling—the map may be right in length and width and crooked in the middle.

This is why Section 31.1 requires all four of dimensional error, loop consistency, visual inspection, and navigation success rate to be done together. Each has a blind spot, and the blind spots do not overlap.

If ground truth equipment can be borrowed (motion capture, a total station, or the AprilTag scheme of Section 24.3), compute the ATE / RPE directly—but read the next subsection first.

Two: the Second Trap—with Too Few Ground Truth Points, the ATE Itself Wobbles

Suppose you have borrowed the equipment and can lay a few ground truth points about the site. The question is: how many are enough?

Panel (b) of Figure 31-2 uses a representative trajectory from family B whose true ATE is 0.60 meters (computed over all 1,200 samples). Then the case of measuring only k ground truth points is simulated: k points are drawn at random from the trajectory and the ATE computed, repeated 800 times for each k, and the 10th to 90th percentile

interval of the estimate drawn.

The result: at k = 3 the 10–90 interval of the estimate runs from 0.36 to 0.84 meters—the same system may be reported as 0.4 or as 0.8, a difference of more than twofold. At k = 8 the interval narrows to 0.45–0.74, at k = 20 to 0.52–0.70, and only at k = 80 to 0.56–0.64.

What does this mean? If you measure ATE = 0.25 meters with three ground truth points and the next group measures 0.55 with three, you cannot say your system is better—both numbers lie in the same distribution and the difference may be entirely the luck of the sampling. For my system beats the control to stand up, the ground truth points must be numerous enough that the intervals do not overlap.

Ground truth points k	The 10–90 interval of the ATE estimate (m)	Departure from the true 0.60 m
3	0.36 – 0.84	−40% to +40%
5	0.43 – 0.78	−28% to +30%
8	0.45 – 0.74	−25% to +23%
12	0.49 – 0.72	−18% to +20%
20	0.52 – 0.70	−13% to +17%
40	0.54 – 0.66	−10% to +10%
80	0.56 – 0.64	−7% to +7%

Table 31-4 The Number of Ground Truth Points and the Uncertainty of the ATE Estimate

Practical advice: for a comparison between groups or between methods, at least 20 ground truth points; to report only a rough order of magnitude, 8 to 12 are acceptable,

but the number must be stated in the report together with the note that the figure carries about ±25 percent of uncertainty. Writing the uncertainty down will not lower your mark—by the standard of Section 31.7 it is a mark of the excellent grade.

Three: What Students Should Take Away from These Two Experiments

1. A method of evaluation has blind spots, and the blind spots are a property of the method and not a matter of luck. Before choosing a measure, ask what class of error it is insensitive to.
2. Using two or more measures whose blind spots do not overlap is more effective than refining one measure finely.
3. Every estimate carries an uncertainty, including the number with which you prove your system good. With too small a sample, the difference between two figures may mean nothing at all.
4. Writing in the report what my evaluation method cannot see shows that you really understand, better than three more decimal places would.

Advice to Teachers on Planning the Course

This section offers project arrangements for three lengths of course, for teachers' reference.

Kind of course	Hours	Projects advised	How to adjust
High school elective	18 weeks × 1–2 hours	Project One (in simulation) or Three	Mainly simulation and paper activities, with no source code
Undergraduate	18 weeks × 3	Project One +	Walk the whole of mapping,

	hours	Two, or Three	localization, navigation, and assessment
Project / research	One or two semesters	Project Four, Five, or Six	Stress research method, quantitative comparison, and writing the report

Table 31-5 Project Arrangements Suggested for Three Kinds of Course

Principles of Scheduling

- The first third on theory and tools: let students get the environment ready and the basics in hand, with small simulation exercises interspersed to keep up the motivation.
- The middle third on basic implementation: complete the guided implementations of Chapters 29 and 30 and build an experience of success.
- The last third on the project: by now students can explore on their own and the teacher turns adviser.
- Keep slack: a practical course has a great many accidents (hardware failure, environment problems, dependency errors). At least two weeks of slack is advised—and the next section will show that at least two weeks is in fact still optimistic.

Advice on Grouping

- Three or four to a group is ideal: fewer and the workload is too heavy, more and someone easily rides along.
- Divide the work explicitly but rotate: four roles of mapping, navigation, assessment, and reporting, say, rotated part way, so that everyone meets the whole procedure.

- Make assessment a role of its own: it is the work most often sacrificed, because it does not bring the immediate satisfaction that mapping does. Appoint someone to it and require data from them at every milestone.

Cases

Case 31-1 A Map Worth Full Marks, 4 Percent Wrong

One group's Project One result: a classroom map with straight wall lines, nothing doubled, and start and end exactly coincident after a circuit—flawless on visual inspection, and the group wrote in their report that the loop closure error is zero and the map quality excellent.

The reviewer measured the classroom with a tape: 9.2 meters long in fact, 8.83 on the map, an error of 4.0 percent. The cause was that the wheel diameter parameter used the nominal value from the specification sheet while the tire, underinflated, had a smaller real radius—which is the real-world version of the red family in Figure 31-2.

The lesson of this case is not that the students were lazy; quite the contrary: they did an assessment, and the one they did simply has no feeling for this kind of error. The remedy was simple too—run the odometry calibration of Section 29.3 once, and after remapping the dimensional error fell to 0.8 percent. The whole thing took less than an afternoon, but had nobody measured with a tape, that map would have carried its 4 percent error on into the navigation stage, and then had them doubting their navigation parameters while Nav2 missed the goal again and again.

Case 31-2 An Improvement with No Control

One group implemented an improved coverage planner in Project Three and wrote in

their report that the coverage reached 97 percent, a good result. The reviewer asked: what was it with the original method? They could not say.

This is the commonest way of losing marks. The number 97 percent carries no information in itself—Figure 31-1 shows that in a room with 12 percent obstacles the plainest boustrophedon passes 95 percent by 1.23 times the distance, while random bounce does not get there at eight times. Whether 97 percent is remarkable or a matter of course depends on what you compare it with and under what conditions.

The right way to write it is: under the same room arrangements and the same budget of distance travelled, the baseline boustrophedon covered 92.4 percent and this method 97.1, and over 20 random arrangements this method was the better in 17. That sentence has a control, a controlled variable, and a number of repetitions; lacking any one of the three, the conclusion is only an impression.

Case 31-3 Three Typical Modes of Project Failure

Mode one: the aim is too large

Setting out confidently at the start of term to build a robot dog that can inspect the whole campus autonomously, and handing in at the end a report containing only simulations. The trouble is not the ambition but the failure to define a minimum viable result—when the first stage sticks, there is nothing to fall back on.

Mode two: environment problems eat the time

Dependency conflicts, an incompatible driver, hardware arriving late, a site that cannot be booked. Each of these alone is a small thing, and together they can eat half a semester. The simulation in the next section will tell you this is not bad luck but the normal state.

Mode three: no quantitative assessment

A great deal was done and it really ran, but the whole report contains only photographs and a record of operations. By the standard of Section 31.7 such a project bleeds badly across the 50 percent carried by map quality and localization error together—and those fifty percent are the easiest to make good: a tape measure and an afternoon will take a pass to a good.

The remedy common to all three modes is to think clearly at the start of the project about what must be handed in at the end. Keep the marking standard of Section 31.7 and the checklist of Figure 31-4 before you and work backward to what each stage must do—the basic skill of project management, and one of the most practical abilities a student takes into industry or graduate school.

Case 31-4 How Much Slack a Schedule Needs: a Simulation

Keep two weeks of slack is common advice, but where do two weeks come from? This section gives an answer with grounds, by Monte Carlo simulation.

The model: a project is divided into four milestones planned at 3, 4, 4, and 3 weeks, 14 weeks in all. The actual duration of each milestone follows a log-normal distribution whose median equals the planned value—which is to say the plan does not systematically underestimate, it merely has uncertainty. Let σ stand for the size of that uncertainty: σ = 0.25 is a software project that goes smoothly, σ = 0.45 the ordinary case, and σ = 0.65 a project involving hardware, a site, and outside dependencies. Each setting is run 40,000 times.

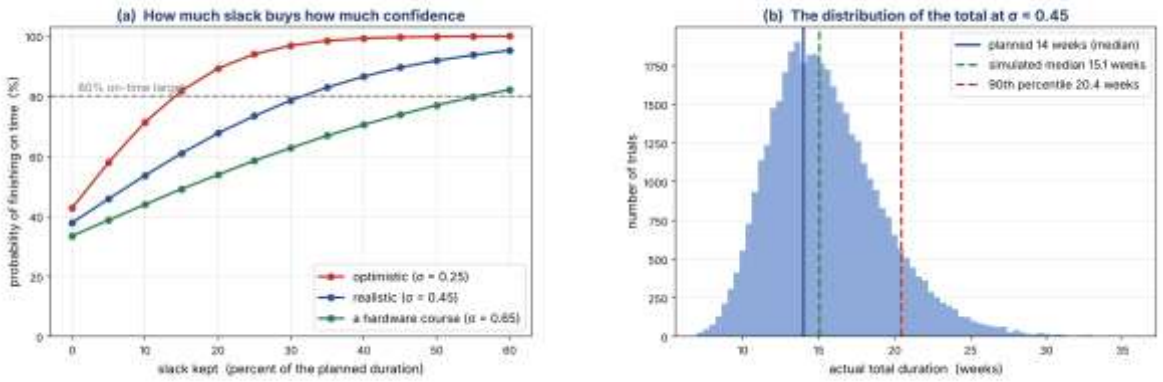


Figure 31-3 The Proportion of Slack and the Rate of Finishing on Time

In panel (a) the horizontal axis is the slack kept as a percentage of the planned duration and the vertical axis the probability of finishing within it. Panel (b) is the distribution of the total duration at σ = 0.45.

One: Three Counterintuitive Numbers

1. Keep no slack and the chance of finishing on time is only three or four in ten. At σ = 0.45 it is 37.9 percent and at σ = 0.65 only 33.5. Note that the median of every milestone here equals the planned value—nobody underestimated, and together they still overrun with high probability.

2. The reason is that the distribution is asymmetric. Look at the histogram in panel (b): the left is blocked at 0 and the right trails a long tail. Finishing a stage early saves a few days at most, while getting stuck can cost three weeks. So the median total duration of 15.1 weeks is already above the planned 14, and the 90th percentile is 20.4 weeks—1.46 times the plan.

3. To reach 80 percent on time, σ = 0.45 needs about 32 percent slack (14 weeks becoming 18.5) and σ = 0.65 about 56. Converted back: for a 14-week project, two weeks of slack buys only a 60 percent chance of being on time; for eight tenths of confidence a software project needs four and a half weeks and a hardware project nearly eight.

Two: Four Implications for Planning a Project

- Put the slack at the end and do not spread it among the stages. Slack spread about is eaten by Parkinson's law (work expands to fill the time available); only slack gathered at the end lets you see how much is left.
- Define a minimum viable result, and reach it at the first milestone. The simulation tells us that overrunning is normal, so it must be guaranteed that there is something in hand when the overrun comes.
- Put the stages of greatest uncertainty first. Hardware arriving, booking a site, installing dependencies—these things of large σ should be stumbled over early, so that the time remaining can respond.
- Re-estimate at the end of every milestone. If the first stage took 1.5 times as long, that is evidence that σ is large for this project of yours, and the expectation for the three stages after must be revised upward too, rather than hoping they will make it back.

The Limits of This Model (and students should write them so in their own reports)

The model assumes the durations of the four stages independent. In reality they are often correlated—unstable hardware slows several stages at once, which makes the long tail heavier, so the figures above are an optimistic estimate.

The model also assumes the median of each stage equals the planned value. Where the plan is itself optimistic (as most people's are), the reality will be worse than Figure 31-3.

The log-normal is a convenient choice and not fitted from data. If you have

records of the actual durations of earlier years' projects, substituting the real distribution and running it again would make a valuable report on improving the course.

Implementation Activities

Implementation 31-1 Planning and Proposing a Project

Purpose: to complete a full planning proposal before committing to a project. The exercise is itself important training—most projects are decided in the planning stage.

Part One: Choosing a Subject and Assessing Its Feasibility

- Choose two or three candidate projects by Table 31-2 and assess each: can the hardware needed be obtained? do you have the ability needed (against the five stages of Section 28.7)? is there time enough?
- For the project that interests you most, list the minimum viable result and the ideal complete result. The first is the floor that must be reached, the second the aim if there is capacity to spare.
- Assess the risks: list the three links most likely to fail and prepare a fallback for each. For instance, if the real rover will not work, complete stage two in simulation.

Part Two: Planning the Milestones

- Divide the project into three to five milestones, each defining explicitly what is to be completed, what evidence it produces, and how long it is expected to take.
- Mark the checkpoints on the schedule. For the proportion of slack follow Case

31-4: keep 30 percent for a project of software or simulation alone, 50 percent or more for one involving hardware, a site, or outside dependencies, and gather the slack at the end.

- For every milestone, list what I will do if this step sticks for more than a week.

Part Three: Designing the Method of Evaluation (crucial)

- By the five dimensions of Section 31.7, design concrete methods of evaluation for your project. What is to be measured for map quality, say, and what will serve as ground truth for localization error.
- Check the measures you chose by Section 31.8: what class of error are they insensitive to? Have you a second measure whose blind spot does not overlap?
- Estimate the number of ground truth points: how many do you intend to measure? By Table 31-4, how much uncertainty does your ATE estimate carry at that number? Is that uncertainty small enough to support the conclusion you want to draw?
- Design a control: what will your result be compared with? (A different method? different parameters? the presence or absence of some component?) Without a control the data mean nothing (Case 31-2).
- Plan the data collection: what data are to be collected at every milestone? In what format recorded? This avoids finding at the end that the data are insufficient (mode three of Case 31-3).

Part Four: Presenting the Proposal

- Put the above into a two-page proposal and prepare a five-minute oral report.
- Present to your classmates and your teacher, with the aim of convincing them that the subject is worthwhile, that you can complete it, and that you know what

difficulties will come.

- Take questions and revise the proposal. Other people's doubts often point out your blind spots—far more efficient than working head-down for three weeks and finding the problem yourself.

Discussion Questions

1. What is your minimum viable result? If only half the time were left, what would you give up?
2. Where does the ground truth in your method of evaluation come from? Without ground truth, how will you argue for the quality of your result?
3. What class of error can your measure not see? What makes it good? (Compare Figure 31-2.)
4. What is your control? Why does that control bring out the value of your result?
5. By the three failure modes of Case 31-3, which is your project likeliest to fall into? What is your precaution?
6. By Case 31-4, about which band does your project's σ fall in? Is the slack you kept enough?

Extended Challenge

Write your proposal up in the formal shape of a research plan: the background and motivation, a literature review (the further reading of this book's chapters may be cited directly), the method, the expected results, the schedule, and the references. The document is not only the starting point of the project but the basis of a graduate application, a National Science and Technology Council undergraduate research proposal, or a competition entry later on. The act of writing it will itself force your ideas clear—the greatest value of this activity.

Implementation 31-2 Stress-Testing Your Method of Evaluation

Purpose: to examine, by the methods of Section 31.8, the way you yourself intend to evaluate. This activity needs only a computer and can be done in one class, but students who have done it usually go back and change their original design of evaluation.

1. Take a trajectory of your own (or generate one going round a circuit in simulation to serve as ground truth).
2. Add two errors by hand: (a) multiply every step length by 1.03, simulating a wheel diameter measured wrongly; (b) add a zero-mean random error to the heading at every step and accumulate it. Generate 50 trajectories of each.
3. For each, compute three numbers: the loop closure gap, the dimensional error, and the ATE against the ground truth.
4. Draw the two scatter plots of loop closure gap against ATE and dimensional error against ATE, and compute the correlation of each.
5. Answer: which error is the measure you originally intended to use blind to? What measure must be added to make it good?
6. Advanced: draw k points at random from your trajectory to estimate the ATE (k of 3, 5, 10, 20, 50), repeat 500 times for each k, and draw the distribution of the estimates. How many ground truth points do you actually intend to lay? At that number, could you confidently distinguish a system with an error of 0.3 meters from one with 0.5?

The product of this activity—two scatter plots and a paragraph—can go straight into the evaluation methods section of your project report. By the standard of Section 31.7, a report that can say I know what my measure cannot see already stands at excellent on the dimension of the report.

Further Reading

- J. Sturm et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," IROS, 2012. (The authoritative source for the ATE and RPE methods, and the paper describing the TUM RGB-D dataset.)
- The evo trajectory evaluation tool (GitHub: MichaelGrupp/evo). (An essential tool for project assessment, supporting many dataset formats and figure outputs.)
- Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," IROS, 2018. (A complete tutorial on trajectory evaluation, including details such as scale alignment; the concerns of Section 31.8 agree with this paper.)
- C. Cadena et al., "Past, Present, and Future of SLAM," IEEE T-RO, 2016. (The authoritative review cited as early as Chapter 1, and a good starting point for a project's literature review.)
- Section 13.7 of this book (assessing map quality), Case 17-1 in Chapter 17 (the wall thickness index), and Section 22.7 (trajectory evaluation). (Internal references for the methods of project assessment.)
- S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020. (The Nav2 paper, whose methodology for assessing system performance is worth consulting.)
- E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013. (A review of coverage path planning, the theoretical basis of Project Three.)

- Collections of outstanding undergraduate research proposals and project competition entries from various universities. (To see how others organize a complete project report.)

Two Figures Worth Pinning Above Your Desk

Figure 31-4 The Complete Flow of a Project from Planning to Presentation

【The planning stage】

① Choose the project by Table 31-2 → ② Assess the resources and the ability → ③ Divide it into three to five milestones

④ Decide the proportion of slack by Case 31-4 (30% for software, 50% or more for hardware, gathered at the end)

↓

【The execution stage】

⑤ Verify in simulation (get the procedure and the parameters working first)

⑥ Implement for real (allow twice the time for debugging)

⑦ Produce verifiable evidence at every milestone (Section 29.7), and re-estimate the time remaining

↓

【The assessment stage】

⑧ Check yourself against the five dimensions of Section 31.7

⑨ Assess quantitatively: ATE / RPE / dimensional error / coverage (as the project requires)

⑩ Check by Section 31.8 whether the measures have blind spots and whether the ground

truth points suffice

⑪ Record honestly the attempts that failed and why

↓

【The presentation stage】

⑫ Write the report: the problem, the method, the data, the analysis, the limitations

⑬ Prepare a three-minute core account + a demonstration video

⑭ Prepare a reflection on what I would do differently

Figure 31-5 The Checklist Before Handing In

□ Map quality: is there quantitative data on the dimensional error? Can the source of the error be explained?

□ Localization accuracy: is there data on the ATE or the loop closure gap? Were both measured?

□ The measure's blind spot: what class of error can my measure not see? Has a second measure been added?

□ Ground truth points: are there enough to support the conclusion I want to draw? Is the uncertainty written down?

□ System stability: has the boundary of failure been tested? Is it known under what conditions the system breaks?

□ The control: is there a comparison with another method or another setting of the parameters? Are the controlled variables stated?

□ A record of failures: are the failed attempts and how they were solved recorded honestly?

□ Complete evidence: are the map files, the trajectory figures, the recordings, and the code all there?

□ Reproducibility: can another person reproduce the result from your documentation?

□ Reflection: can you say what you would do differently?

Summary

- The six projects rise in difficulty: 2D mapping of a classroom (★), three-dimensional scanning (★★), path analysis for a robot vacuum (★★), campus inspection with a robot dog (★★★★), a vision-navigated drone (★★★★), and multi-sensor fused localization (★★★★★); in choosing, weigh the resources, the ability, and the continuity.
- Project One covers the whole cycle of SLAM and its result is plain to see; what is assessed is not whether a map was built but whether it can be said clearly why the map is good or bad. The product of Project Two has long-term value for managing departmental space and for a simulation environment, and may become the scene for the training in simulation of Chapter 33 directly.
- The experimental result of Project Three: at one times the theoretical minimum distance, the spiral covers 83.2 percent, the boustrophedon 75.6, and random bounce only 35.3; to reach 95 percent coverage the boustrophedon needs 0.95 times the distance in an empty room and random bounce 6.9, while in a room with 12 percent obstacles or more random bounce does not get there even at eight times. The advantage of a systematic strategy is not merely speed but its guarantee of coverage.
- Marking takes five dimensions: map quality (25%), localization error (25%), system stability (20%), the report (20%), and code quality (10%); reading the standard at the start of the project and planning backward from it is advised.
- The method of evaluation has blind spots of its own: a map error from a scale bias is wholly invisible to the loop closure gap—in the experiment this family's loop closure gap was 0.000 meters while the median ATE was 0.25, the median dimensional error 3.2 percent, and the maximum 6.0. The loop closure gap and the dimensional error must be used together, because their blind spots do not

overlap.

- The figures of an evaluation carry uncertainty themselves: for one trajectory whose true ATE is 0.60 meters, the 10–90 interval estimated from 3 ground truth points is 0.20–0.88 meters and only narrows to 0.46–0.73 at 20 points. For a comparison between groups, use at least 20 ground truth points and state the number and the uncertainty in the report.
- The uncertainty of a schedule is asymmetric: a project of four milestones planned at 14 weeks, under ordinary uncertainty and with no slack, finishes on time only 37.9 percent of the time, with a median total duration of 15.1 weeks and a 90th percentile of 20.4. For eight tenths of confidence a software project needs about 30 percent of slack and a hardware project about 55, and the slack should be gathered at the end rather than spread about.
- The three typical modes of project failure are an aim too large, environment problems eating the time, and no quantitative assessment; the remedy common to all is to think clearly at the start about what must be handed in at the end, and to plan each stage backward from the marking standard.
- Reporting a failed attempt honestly should earn marks rather than lose them—displaying the whole course of finding the problem, analyzing the cause, and verifying the improvement says far more about the depth of understanding than presenting a handsome result alone; and writing what my evaluation method cannot see is likewise a mark of the excellent grade.
- Advice on presenting: the core should take three minutes, use a recorded video rather than a live demonstration, the data need a control, and prepare a reflection on what I would do differently.

The Close of Part VIII

Part VIII, Implementation in ROS / Python / C++, is now complete. From the environment of Chapter 28, through the LiDAR mapping and navigation of Chapter 29 and the visual SLAM and cross-sensor comparison of Chapter 30, to this chapter's integration and assessment of a project, students have walked the whole path from theory to practice.

And remember the two scatter plots of Section 31.8. The thirty chapters before taught how to let a robot know where it is; this chapter adds how to know that it really knows—and the latter is the more easily skipped, and the better at telling the quality of a piece of work.

In the last part we turn our eyes to the future. Traditional SLAM solves where am I and what does the map look like; AI and the VLM are to solve what am I looking at, what does this place mean, and what should be done next. Part IX has three chapters: Chapter 32, Semantic SLAM: Giving the Map Meaning, raises a geometric map into a semantic one; Chapter 33, Training in Simulation and Sim-to-Real, lets a robot learn a skill in a virtual environment and deploy it on a real body; and Chapter 34, VLM, Language Navigation, and Embodied Intelligence, lets a robot understand an instruction such as please go to the registrar's office, and brings the book to its close.

The habits of evaluation you have built in this chapter will matter more in Part IX, not less. A failure at the layer of semantics and language usually raises no error on the spot—it merely takes the robot quietly to the wrong room.